

LỜI NÓI ĐẦU

Chào mừng các em học sinh và quý thầy cô đến với bộ giáo trình **Khoá học C++ cơ bản — QUYỂN 2: CẤU TRÚC DỮ LIỆU & THUẬT TOÁN NÂNG CAO** của Trung tâm tin học iKH.

Bộ tài liệu này được biên soạn công phu nhằm cung cấp lộ trình học tập lập trình thi đấu bài bản, chuẩn mực và hiện đại nhất dành cho học sinh THCS, THPT và sinh viên đam mê thuật toán.

Phần nội dung này gồm **3 Chương chuyên sâu (Chương 05 đến Chương 07)** với **9 Bài học** và **135 bài toán thực hành**, chinh phục quy hoạch động 1D/2D/chuỗi, cấu trúc dữ liệu STL nâng cao, Stack, Queue, Đồ thị BFS/DFS và cây Segment/Fenwick.

Mỗi bài học được thiết kế theo cấu trúc sư phạm chặt chẽ:

- **Khái niệm & Bản chất toán học:** Giải thích trực quan, dễ hiểu kèm chứng minh toán học ngắn gọn.
- **Mô hình bài toán kinh điển:** Các dạng bài đặc trưng kèm phân tích độ phức tạp thời gian/không gian.
- **Mẫu cài đặt chuẩn thi đấu:** Code C++ chuẩn, tối ưu, dễ hiểu và tuân thủ các quy chuẩn lập trình hiện đại.
- **Hệ thống bài tập thực hành:** Phân tầng từ cơ bản đến nâng cao (P0 đến P5), có đầy đủ giới hạn thời gian, bộ nhớ, sample test và giải thích chi tiết.
- **Lời giải tham khảo có chọn lọc:** Phụ lục B cung cấp mã nguồn C++ chuẩn mực cho 3 bài tập đầu tiên của mỗi bài học; các bài tập còn lại rèn luyện tư duy tự suy luận và giải quyết vấn đề.

Chúc các em học tập hiệu quả và chinh phục những giải thưởng cao trong các kỳ thi học sinh giỏi Tin học và Olympic lập trình!

CHƯƠNG 05: QUY HOẠCH ĐỘNG (DYNAMIC PROGRAMMING)

Bài 13: Quy hoạch động 1D & dãy con tăng dài nhất

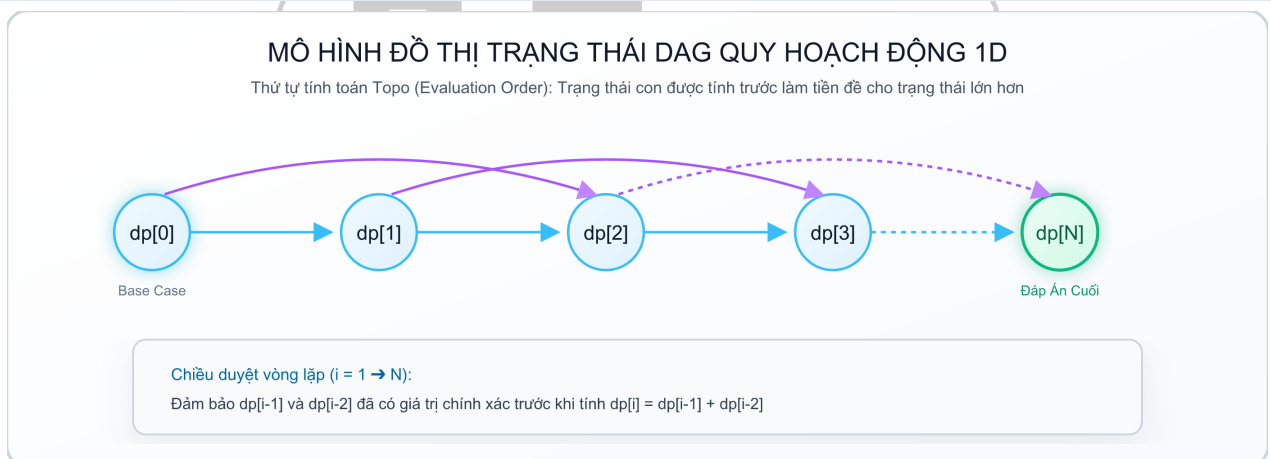
1. Bản chất vấn đề & cầu nối từ đệ quy sang quy hoạch động

Trong Bài 10 và 12, ta đã chứng kiến hiện tượng **Bùng nổ Không gian Trạng thái (Combinatorial Explosion)** khi duyệt đệ quy phân nhánh:

- Khi tính số Fibonacci $F(N) = F(N - 1) + F(N - 2)$, trạng thái $F(3)$ bị tính lại 2 lần, $F(2)$ bị tính lại 3 lần. Độ phức tạp thời gian tăng vọt lên cấp số nhân $\Theta(\varphi^N) \approx \Theta(1.618^N)$.
- **Nguyên nhân gốc rễ:** Hàm đệ quy thuần túy không có cơ chế “ghi nhớ” (Memory). Mỗi lần bước vào một nhánh mới, nó xem bài toán con đó như một thực thể hoàn toàn xa lạ và tính toán lại từ đầu.

Quy Hoạch Động (Dynamic Programming - DP) giải quyết vấn đề này bằng nguyên lý cốt lõi:

DP loại bỏ việc tính toán lại các bài toán con trùng lặp bằng cách lưu trữ kết quả vào Bảng phương án (DP Table) và tái sử dụng ngay lập tức trong $\mathcal{O}(1)$.



Mô hình Đồ thị trạng thái DAG Quy hoạch động 1D

2. Khung phương pháp luận: 7 bước DP state invariant

Để giải quyết chính xác bất kỳ bài toán Quy hoạch động nào, ta áp dụng khung 7 thành phần logic chuẩn mực:

1. **State Definition & Invariant (Định nghĩa Trạng thái & Bất biến):** $dp[i]$ đại diện chính xác 100% cho đại lượng nào? (Là giá trị tối ưu, số cách thực hiện, hay trạng thái logic?).
2. **Base Cases (Trường hợp cơ sở):** Các trạng thái biên nhỏ nhất không thể phân rã thêm ($i = 0, 1$) có giá trị bằng bao nhiêu?
3. **State Transition (Hệ thức chuyển trạng thái):** Trạng thái $dp[i]$ phụ thuộc vào các trạng thái con $dp[j]$ ($j < i$) nào trước đó qua công thức toán học nào?

4. **Evaluation Order (Thứ tự tính toán):** Chiều duyệt vòng lặp ($i = 1 \rightarrow N$ xuôi hay ngược) theo thứ tự Topo DAG để đảm bảo mọi bài toán con phụ thuộc đều đã được tính xong trước khi dùng.
5. **Answer Extraction (Trích xuất kết quả):** Đáp án của bài toán gốc nằm ở đâu? (Tại $dp[N]$, hay $\max_{i=1}^N dp[i]$?).
6. **Space & Time Optimization (Tối ưu hóa):** Phân tích độ phức tạp thời gian $\mathcal{O}(\text{Time})$, bộ nhớ $\mathcal{O}(\text{Space})$ và khả năng nén mảng.
7. **Reconstruction (Khôi phục nghiệm):** Dùng mảng truy vết $\text{trace}[i]$ hoặc duyệt ngược trên bảng dp để tái tạo lại cấu hình nghiệm tối ưu (*Ghi N/A nếu bài toán chỉ yêu cầu giá trị*).

3. Các mô hình quy hoạch động 1D cốt lõi (core patterns)

3.1. Mô hình bậc thang & bước nhảy (staircase / frog jump)

- **Bối cảnh:** Một chú ếch đứng ở bậc 0, muốn nhảy lên bậc N . Tại mỗi bậc, ếch có thể nhảy 1 bước hoặc 2 bước.
- **State Definition:** $dp[i]$ là số cách khác nhau để ếch nhảy từ bậc 0 đến bậc i .
- **Base Cases:** $dp[0] = 1$ (có đúng một cách để hoàn thành hành trình từ bậc 0 đến bậc 0 — không thực hiện bước nhảy nào), $dp[1] = 1$.
- **State Transition:** Để đến bậc i , bước nhảy cuối cùng bắt buộc phải xuất phát từ bậc $i - 1$ (nhảy 1 bước) hoặc bậc $i - 2$ (nhảy 2 bước):

$$dp[i] = dp[i - 1] + dp[i - 2] \pmod{10^9 + 7}$$

- **Evaluation Order:** Duyệt xuôi từ $i = 2 \rightarrow N$.

3.2. Mô hình đổi tiền ít xu nhất (coin change 1D)

- **Bối cảnh:** Cho hệ thống gồm K đồng xu có mệnh giá $C = \{c_1, c_2, \dots, c_K\}$. Cần đổi số tiền S sao cho tổng số đồng xu là ít nhất.
- **State Definition:** $dp[i]$ là số lượng đồng xu **ít nhất** để tạo ra đúng tổng giá trị i .
- **Base Cases:** $dp[0] = 0$ (Tổng tiền bằng 0 cần đúng 0 đồng xu). Khởi tạo mọi $dp[i] = \infty$ với $i \geq 1$.
- **State Transition:** Thử chọn đồng xu cuối cùng là mệnh giá $c \in C$:

$$dp[i] = 1 + \min_{\{c \in C | i \geq c\}} dp[i - c]$$

- **Evaluation Order:** Duyệt xuôi $i = 1 \rightarrow S$. Nếu $dp[S] = \infty \Rightarrow$ Không thể đổi được.

Lưu ý quan trọng: Quy tắc thứ tự vòng lặp phân biệt Hoán vị / Tổ hợp dưới đây áp dụng cho **bài toán đếm số cách**. Với bài toán tối ưu số đồng xu ít nhất $dp[i] = 1 + \min(dp[i - c])$, do phép toán min có tính chất giao hoán và kết hợp nên ta luôn duyệt i từ $1 \rightarrow S$ mà không làm thay đổi giá trị tối ưu.

- **Phân biệt Sự phạm Cốt lõi Trong Bài Toán Đếm Số Cách (Counting Coin Change):**

- **Bài toán Hoán vị (Permutation):** Thứ tự các đồng xu có phân biệt (ví dụ $1 + 2 \neq 2 + 1$ (khác $2 + 1$)). Vòng lặp ngoài duyệt Tiền $i = 1 \rightarrow S$, vòng lặp trong thử từng đồng xu $c \in C$.
- **Bài toán Tổ hợp (Combination):** Thứ tự các đồng xu không phân biệt (ví dụ $1 + 2$ và $2 + 1$ là một cách). Vòng lặp ngoài duyệt từng đồng xu $c \in C$, vòng lặp trong duyệt Tiền $i = c \rightarrow S$.

BÀI TOÁN ĐỔI TIỀN (COIN CHANGE 1D) & DAG TRẠNG THÁI

Mệnh giá xu: $C = \{1, 3, 4\}$ | $dp[S]$ = Số đồng xu ít nhất để tạo ra tổng S

$S = 0$
0 xu

$S = 1$
1 xu

$S = 2$
2 xu

$S = 3$
1 xu

$S = 4$
1 xu

$S = 6$ (Target)
2 xu (3+3)

Dùng xu 3 (từ $S=3$) Dùng xu 4 (từ $S=2$)

$$dp[S] = 1 + \min(\{dp[S - c] \mid c \in C \text{ và } S \geq c\})$$

Khởi tạo $dp[0] = 0$, mọi $dp[i] = \text{INF}$. Cát tỉa nhánh không thể đổi được khi $dp[S] == \text{INF}$.

Bài toán Đổi tiền Coin Change và DAG trạng thái

3.3. Dãy con tăng dài nhất (longest increasing subsequence — LIS $\mathcal{O}(N^2)$)

- **Bối cảnh:** Cho dãy số $A = [a_1, a_2, \dots, a_N]$. Tìm độ dài dãy con tăng nghiêm ngặt dài nhất.
- **State Definition (Tự huyệt định nghĩa):** $dp[i]$ là độ dài của dãy con tăng dài nhất **kết thúc bắt buộc tại phần tử $A[i]$** .
- **Base Cases:** $dp[i] = 1$ với mọi $1 \leq i \leq N$ (bản thân mỗi phần tử đơn lẻ là dãy con độ dài 1).
- **State Transition:** Duyệt qua mọi phần tử $A[j]$ đứng trước $A[i]$ ($1 \leq j < i$):

$$dp[i] = 1 + \max_{\{1 \leq j < i \mid A[j] < A[i]\}} dp[j]$$

- **Answer Extraction:** Kết quả toàn cục là $\max_{i=1}^N dp[i]$.
- **Độ phức tạp:** $\mathcal{O}(N^2)$ thời gian, $\mathcal{O}(N)$ bộ nhớ. Thường phù hợp với N cỡ vài nghìn, tùy thuộc vào time limit và hệ số hằng số.

Mô hình Quy hoạch Động LIS $\mathcal{O}(N^2)$ & Cơ chế Truy vết

Bất biến trạng thái: $dp[i]$ là độ dài dãy con tăng dài nhất kết thúc bắt buộc tại phần tử $A[i]$

	$i = 0$	$i = 1$	$i = 2$	$i = 3$	$i = 4$	$i = 5$	$i = 6$	$i = 7$
Mảng A Giá trị gốc	10	22	9	33	21	50	41	60
Mảng dp Độ dài LIS	1	2	1	3	2	4	4	5 (MAX)

Cung truy vết →

Dãy con tăng dài nhất (LIS) khôi phục:

[10 → 22 → 33 → 50 → 60] (LIS = 5)

Công thức quy hoạch động & Độ phức tạp:

$dp[i] = 1 + \max(\{dp[j] \mid 0 \leq j < i \text{ và } A[j] < A[i]\})$

Thời gian: $\mathcal{O}(N^2)$ • Bộ nhớ: $\mathcal{O}(N)$ với mảng dp và trace

Mô hình Dãy con tăng dài nhất LIS $\mathcal{O}(N^2)$ và Truy vết

3.4. Kỹ thuật khôi phục vết nghiệm 1D (traceback / reconstruction)

Để in ra chính xác cấu hình dãy phân tử tạo nên nghiệm tối ưu:

1. Duy trì mảng $trace[i] = j$ ghi nhận chỉ số phân tử đứng ngay trước $A[i]$ trong cấu hình tối ưu.
2. Tìm vị trí $best_idx$ có $dp[best_idx]$ đạt cực trị.
3. Lăn ngược mảng $trace$ từ $best_idx$ về điểm xuất phát, lưu các phân tử vào một vector rồi đảo ngược (reverse).

3.5. Mở rộng nâng cao: LIS $\mathcal{O}(N \log N)$ bằng Binary Search

Khi $N \leq 10^5$, thuật toán $\mathcal{O}(N^2)$ sẽ bị Quá thời gian (TLE).

- **Ý tưởng:** Duy trì mảng phụ $tails$ trong C++ (chỉ số 0-based), trong đó phần tử $tails[len - 1]$ lưu **giá trị phần tử kết thúc nhỏ nhất** của một dãy con tăng có độ dài đúng bằng len .
- Mảng $tails$ luôn có tính chất **đơn điệu tăng dần** $\Rightarrow$ Dùng Tìm kiếm nhị phân (lower_bound) để tìm và cập nhật vị trí thích hợp cho mỗi $A[i]$ trong $\mathcal{O}(\log N)$.
- **Tổng thời gian:** $\mathcal{O}(N \log N)$. Đây là kỹ thuật mở rộng tối ưu hóa nâng cao (Challenge Extension).

4. Các biến thể thiết kế trạng thái từ mô hình 1D (state design variations)

Mục tiêu lớn nhất của Module 05 không phải là học thuộc các công thức, mà là rèn luyện khả năng **Thiết Kế Trạng Thái (State Design)** trước các biến thể bài toán mới:

4.1. Pattern a: Quyết định nhị phân (binary choice — Chọn / bỏ qua)

- **Bối cảnh (House Robber):** Không được chọn hai phần tử liên kề nhau.
- **State Invariant:** $dp[i]$ là tổng giá trị lớn nhất khi chỉ xét tiền tố từ 1 ... i .
- **Transition:** Tại vị trí i , có 2 lựa chọn loại trừ lẫn nhau:

$$dp[i] = \max(dp[i-1], dp[i-2] + A[i])$$

Không chọn i Chọn $i \Rightarrow$ bỏ qua $i-1$

- **Chuyển đổi bài toán (Delete and Earn):** Khi chọn giá trị v , ta nhận toàn bộ tổng điểm $points[v] = v \times count(v)$ nhưng bị cấm chọn $v-1$ và $v+1$. Bằng cách gom nhóm dữ liệu theo trục giá trị v , bài toán được quy đổi hoàn toàn về mô hình House Robber trên mảng $points$.

4.2. Pattern b: Mở rộng trạng thái hữu hạn (state dimension expansion)

- **Bối cảnh (Alternating Subsequence):** Dãy con đan dấu (tăng $\rightarrow$ giảm $\rightarrow$ tăng $\rightarrow$ giảm).
- **Vấn đề:** Nếu chỉ dùng $dp[i]$, ta không biết phần tử $A[i]$ đang đóng vai trò là “đỉnh tăng” hay “đáy giảm”.
- **Thiết kế Trạng thái:** Bổ sung thêm một chiều trạng thái hữu hạn $state \in \{0, 1\}$:
- $dp[i][0]$: Độ dài dãy đan dấu kết thúc tại $A[i]$ với bước nhảy cuối cùng là **GIẢM** ($A[j] > A[i]$).

- $dp[i][1]$: Độ dài dãy đơn đầu kết thúc tại $A[i]$ với bước nhảy cuối cùng là **TĂNG** ($A[j] < A[i]$).
- *Lưu ý*: Chiều bắt đầu của dãy con có thể linh hoạt bắt đầu bằng tăng hoặc giảm tùy theo yêu cầu đề bài; kết quả toàn cục thường là $\max(\max_i dp[i][0], \max_i dp[i][1])$.

Lưu ý: Với dãy có độ dài 1, chưa tồn tại bước nhảy tăng/giảm; giá trị khởi tạo cụ thể của $dp[i][0]$, $dp[i][1]$ phụ thuộc vào định nghĩa bài toán và cách triển khai. Phần này được xem như mô hình mở rộng, không phải template cài đặt đầy đủ trong chuyên đề này.

4.3. Pattern c: Thay đổi đại lượng tối ưu (maximum sum increasing subsequence — Msis)

- **Bối cảnh**: Thay vì tìm dãy con tăng có độ dài lớn nhất, bài toán yêu cầu tìm dãy con tăng có tổng giá trị các phần tử lớn nhất.
- **Điều chỉnh Invariant**:
- LIS: $dp[i] = \text{độ dài LIS} \Rightarrow dp[i] = 1 + \max(dp[j])$.
- MSIS: $dp[i] = \text{Tổng lớn nhất}$ của dãy con tăng kết thúc tại $A[i]$:

$$dp[i] = A[i] + \max_{\{j < i | A[j] < A[i]\}} dp[j]$$

4.4. Pattern d: Phân hoạch đoạn tối ưu (pattern mở rộng / preview — Optimal array partitioning & rod cutting)

- **Bối cảnh**: Cắt một thanh gỗ độ dài N (hoặc phân chia dãy số $A[1 \dots N]$ thành các đoạn con liên tiếp) sao cho tổng giá trị/chi phí là tối ưu.
- **State Invariant**: $dp[i]$ là chi phí/giá trị tối ưu khi phân hoạch tiền tố $A[1 \dots i]$.
- **Transition**: Thử mọi điểm cắt cuối cùng $j \in [0, i - 1]$:

$$dp[i] = \min_{0 \leq j < i, \text{valid}(j+1, i)} (dp[j] + \text{cost}(j+1, i))$$

- **Áp dụng cho bài Mastery CPPB-DP1-15**: Tìm cách phân chia dãy số thành các khối đoạn con thỏa mãn điều kiện ràng buộc với chi phí nhỏ nhất.

5. Các bẫy lỗi lập trình kinh điển (bug traps)

1. **Bẫy khởi tạo giá trị vô cùng (Infinity Overflow)**:
 - Khi tìm giá trị nhỏ nhất, nếu dùng `INT_MAX` cho kiểu `int`, phép tính $dp[i-c] + 1$ sẽ gây tràn số thành số âm cực lớn.
 - **Quy tắc an toàn**: Dùng hằng số `const long long INF = 1e18`; (hoặc `1LL << 60`) và luôn kiểm tra `if (dp[i - c] != INF)` trước khi chuyển trạng thái.
2. **Bẫy định nghĩa sai Trạng thái trong LIS**:
 - Ngộ nhận: “ $dp[i]$ là độ dài LIS trong đoạn từ 1 đến i ”. Nếu định nghĩa như vậy, ta không thể biết phần tử kết thúc là bao nhiêu để so sánh với $A[i + 1]$.
 - **Bất biến đúng**: Bắt buộc $dp[i]$ phải là độ dài LIS kết thúc tại chính $A[i]$.
3. **Bẫy nhầm lẫn thứ tự vòng lặp trong Coin Change Đếm Số Cách**:

- Duyệt Tiền trước, Coin sau $\Rightarrow$ Tạo ra bài toán Hoán vị (đếm lặp thứ tự).
- Duyệt Coin trước, Tiền sau $\Rightarrow$ Tạo ra bài toán Tổ hợp (đếm không trùng lặp).

6. Mẫu cài đặt chuẩn thi đấu (competitive templates)

Mẫu 1: Dãy con tăng dài nhất LIS $O(N^2)$ kèm truy vết nghiệm

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;

    if (n <= 0) return 0;

    vector<long long> a(n);

    for (int i = 0; i < n; ++i) {
        cin >> a[i];
    }

    vector<int> dp(n, 1);
    vector<int> trace(n, -1);

    int max_len = 1;
    int best_end = 0;

    // Quy hoạch động  $O(N^2)$ 
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < i; ++j) {
            if (a[j] < a[i] && dp[j] + 1 > dp[i]) {
                dp[i] = dp[j] + 1;
                trace[i] = j;
            }
        }
        if (dp[i] > max_len) {
            max_len = dp[i];
            best_end = i;
        }
    }

    cout << max_len << "\n";

    // Khôi phục vết nghiệm
    vector<long long> lis_elements;

    int curr = best_end;
    while (curr != -1) {
        lis_elements.push_back(a[curr]);
        curr = trace[curr];
    }
    reverse(lis_elements.begin(), lis_elements.end());

    for (int i = 0; i < (int)lis_elements.size(); ++i) {
```



```

        cout << lis_elements[i] << (i + 1 == (int)lis_elements.size() ? "" : " ");
    }
    cout << "\n";

    return 0;
}

```

Mẫu 2: Đổi tiền ít xu nhất (coin change 1D)

```

#include <bits/stdc++.h>
using namespace std;

const long long INF = 1e18;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int k;
    long long s;
    if (!(cin >> k >> s)) return 0;

    vector<long long> c(k);

    for (int i = 0; i < k; ++i) {
        cin >> c[i];
    }

    vector<long long> dp(s + 1, INF);

    dp[0] = 0;

    for (int i = 1; i <= s; ++i) {
        for (long long coin : c) {
            if (i >= coin && dp[i - coin] != INF) {
                dp[i] = min(dp[i], dp[i - coin] + 1);
            }
        }
    }

    if (dp[s] == INF) {
        cout << -1 << "\n";
    } else {
        cout << dp[s] << "\n";
    }

    return 0;
}

```

Bài tập thực hành

Bài 01 [CPPB-DP1-01]: Bậc Thang Cơ Bản

Bối cảnh: Trong một tòa tháp công nghệ hiện đại, bạn robot phục vụ cần di chuyển lên đỉnh cầu thang gồm N bậc để chuyển tài liệu. Do thiết kế cơ học của chân bước, ở mỗi nhịp đi chuyển, robot chỉ có thể bước lên đúng 1 bậc hoặc bước sai dài qua 2 bậc liên tiếp. Để lập trình điều hướng linh hoạt cho robot, kỹ sư phần mềm cần tính toán xem có tất cả bao nhiêu trình tự bước đi khác nhau để đưa robot từ chân cầu thang lên tới đúng đỉnh bậc thứ N .

Nhiệm vụ: Cho số nguyên dương N là số bậc của cầu thang. Hãy lập trình tính số cách bước hợp lệ để robot lên đến đỉnh bậc thứ N . Vì kết quả có thể rất lớn, hãy in ra phần dư của kết quả khi chia cho $10^9 + 7$.

Đầu vào (Input):

- Một dòng duy nhất chứa số nguyên dương N ($1 \leq N \leq 10^5$).

Đầu ra (Output):

- In ra số cách leo lên bậc N theo modulo $10^9 + 7$.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
3	3

Giải thích:

Với cầu thang có $N = 3$ bậc, robot có tất cả 3 trình tự bước đi hợp lệ để lên đến đỉnh:

- Bước từng bậc một: $1 + 1 + 1 = 3$.
- Bước 1 bậc rồi bước 2 bậc: $1 + 2 = 3$.
- Bước 2 bậc rồi bước 1 bậc: $2 + 1 = 3$.

Kết quả in ra là 3.

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq N \leq 10^5$.

Bài 02 [CPPB-DP1-02]: Chú Éch Nhảy Chi Phí Nhỏ Nhất

Bối cảnh: Trong một khu vườn sinh thái, có N phiến đá được xếp thành hàng ngang từ trái sang phải, đánh số thứ tự từ 1 đến N . Phiến đá thứ i có độ cao là H_i . Một chú ếch đang ở phiến đá số 1 và muốn di chuyển tới phiến đá cuối cùng số N . Từ phiến đá số i , chú ếch có thể chọn nhảy sang phiến đá ngay cạnh $i + 1$ hoặc nhảy vượt qua một phiến để tới phiến đá $i + 2$. Năng lượng tiêu hao (chi phí) cho mỗi cú nhảy từ phiến đá i sang phiến đá j đúng bằng độ chênh lệch chiều cao giữa hai phiến đá, tức $|H_i - H_j|$.

Nhiệm vụ: Cho số lượng phiến đá N và độ cao của từng phiến đá. Hãy lập trình tìm tổng chi phí năng lượng tối thiểu để chú ếch có thể di chuyển từ phiến đá 1 tới phiến đá N .

Đầu vào (Input):

- Dòng 1: Số nguyên N ($2 \leq N \leq 10^5$).
- Dòng 2: N số nguyên $H_1, H_2, \dots, H_N$ ($1 \leq H_i \leq 10^4$).

Đầu ra (Output):

- In ra tổng chi phí nhỏ nhất.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
4 10 30 40 20	30

Giải thích:

Với 4 phiên đá có độ cao lần lượt là [10, 30, 40, 20]:

- Bước 1: Từ phiên đá 1 ($H_1 = 10$) nhảy sang phiên đá 2 ($H_2 = 30$), chi phí tiêu hao là $|10 - 30| = 20$.

- Bước 2: Từ phiên đá 2 ($H_2 = 30$) nhảy vượt sang phiên đá 4 ($H_4 = 20$), chi phí tiêu hao là $|30 - 20| = 10$.

Tổng chi phí tiêu hao cho toàn bộ hành trình là $20 + 10 = 30$, đây là phương án tốn ít năng lượng nhất.

Ràng buộc & Giới hạn:

- 100% số test có $2 \leq N \leq 10^5, 1 \leq H_i \leq 10^4$.

Bài 03 [CPPB-DP1-03]: Chú Éch Nhảy K Bước

Bối cảnh: Vẫn tại khu vườn sinh thái có N phiên đá xếp thành hàng ngang với độ cao $H_1, H_2, \dots, H_N$. Lần này chú ếch đã được huấn luyện với thể lực dẻo dai hơn: Từ phiên đá thứ i, chú ếch có thể chọn nhảy xa tới bất kỳ phiên đá nào trong phạm vi K bước tiếp theo, tức là các phiên đá $i + 1, i + 2, \dots, \min(N, i + K)$. Chi phí cho mỗi bước nhảy từ phiên đá i tới phiên đá j vẫn bằng độ chênh lệch chiều cao $|H_i - H_j|$.

Nhiệm vụ: Cho số lượng phiên đá N, tầm nhảy tối đa K và danh sách độ cao của các phiên đá. Hãy lập trình tính toán tổng chi phí năng lượng ít nhất để chú ếch đi từ phiên đá 1 tới phiên đá N.

Đầu vào (Input):

- Dòng 1: Hai số nguyên N và K ($2 \leq N \leq 10^5, 1 \leq K \leq 100$).
- Dòng 2: N số nguyên $H_1, H_2, \dots, H_N$ ($1 \leq H_i \leq 10^4$).

Đầu ra (Output):

- In ra chi phí tối thiểu.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
5 3 10 30 40 50 20	30

Giải thích:

Với $N = 5, K = 3$ và độ cao các phiên đá là [10, 30, 40, 50, 20]:

- Từ đá 1 ($H_1 = 10$), chú ếch nhảy sang đá 2 ($H_2 = 30$) với khoảng cách 1 bước hợp lệ (≤ 3), chi phí là $|10 - 30| = 20$.

- Từ đá 2 ($H_2 = 30$), chú ếch nhảy thẳng tới đích là đá 5 ($H_5 = 20$) với khoảng cách $5 - 2 = 3$ bước (vẫn $\leq K = 3$), chi phí là $|30 - 20| = 10$.

Tổng chi phí tối thiểu đạt được là $20 + 10 = 30$.

Ràng buộc & Giới hạn:

- 100% số test có $2 \leq N \leq 10^5, 1 \leq K \leq 100$.

Bài 04 [CPPB-DP1-04]: Đổi Tiền Số Xu Ít Nhất

Bối cảnh: Tại một trạm đổi tiền tự động thông minh của trung tâm thương mại, hệ thống hỗ trợ N loại đồng xu với các mệnh giá khác nhau. Khách hàng có nhu cầu đổi một số tiền mặt đúng bằng S đồng để sử dụng máy bán nước tự động. Nhằm tối ưu hóa số lượng tiền kim loại lưu thông và giúp ví của khách hàng không bị quá nặng, hệ thống máy luôn tìm cách chi trả bằng số lượng đồng xu ít nhất có thể. Biết rằng kho tiền xu của trạm luôn dồi dào, mỗi mệnh giá có số lượng xu không giới hạn.

Nhiệm vụ: Cho danh sách N mệnh giá đồng xu và số tiền cần đổi S. Hãy lập trình xác định số lượng đồng xu ít nhất để đổi được đúng số tiền S. Nếu không có cách nào đổi được đúng số tiền đó, in ra `-1`.

Đầu vào (Input):

- Dòng 1: Hai số nguyên N và S ($1 \leq N \leq 100, 1 \leq S \leq 10^5$).
- Dòng 2: N số nguyên dương biểu diễn các mệnh giá xu ($1 \leq c_i \leq 10^4$).

Đầu ra (Output):

- Số lượng đồng xu ít nhất hoặc -1 nếu không thể đổi.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
3 11 1 5 6	2

Giải thích:

Để đổi được số tiền $S = 11$ từ các mệnh giá xu $\{1, 5, 6\}$:

- Cách 1: Dùng 1 đồng 6 và 5 đồng 1 ($6 + 1 \times 5 = 11$), tổng cộng tốn 6 đồng xu.
- Cách 2: Dùng 2 đồng 5 và 1 đồng 1 ($5 \times 2 + 1 = 11$), tổng cộng tốn 3 đồng xu.
- Phương án tối ưu nhất: Dùng 1 đồng 5 và 1 đồng 6 ($5 + 6 = 11$), chỉ cần đúng 2 đồng xu. Kết quả là 2.

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq N \leq 100, 1 \leq S \leq 10^5$.

Bài 05 [CPPB-DP1-05]: Đếm Số Cách Đổi Tiền

Bối cảnh: Tại một hội thảo tài chính công nghệ, ban tổ chức thiết kế một thử thách cho các bạn học sinh về phương thức kết hợp tiền tệ. Cho N mệnh giá đồng xu khác nhau, mỗi mệnh giá đều có số lượng không giới hạn. Một bạn học sinh cần tạo ra tổng số tiền đúng bằng S đồng. Hai cách tạo tiền được xem là khác nhau nếu số lượng sử dụng của ít nhất một mệnh giá đồng xu là khác nhau (thứ tự chọn các đồng xu không làm ảnh hưởng đến tính chất của tổ hợp).

Nhiệm vụ: Cho N mệnh giá đồng xu và số tiền mục tiêu S . Hãy lập trình đếm xem có tất cả bao nhiêu tổ hợp đồng xu khác nhau để có tổng giá trị đúng bằng S . Vì kết quả có thể rất lớn, hãy in ra phần dư của kết quả khi chia cho $10^9 + 7$.

Đầu vào (Input):

- Dòng 1: N và S ($1 \leq N \leq 100, 1 \leq S \leq 10^5$).
- Dòng 2: N số nguyên dương c_i ($1 \leq c_i \leq 10^4$).

Đầu ra (Output):

- Số cách đổi tiền theo modulo $10^9 + 7$.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
3 9 2 3 5	3

Giải thích:

Với các mệnh giá xu $\{2, 3, 5$ và số tiền mục tiêu $S = 9$, có đúng 3 tổ hợp đồng xu khác nhau:

- Ba đồng xu: $2 + 2 + 5 = 9$.
- Ba đồng xu: $3 + 3 + 3 = 9$.
- Bốn đồng xu: $2 + 2 + 2 + 3 = 9$.

Kết quả đếm được là 3 cách.

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq N \leq 100, 1 \leq S \leq 10^5$.

Bài 06 [CPPB-DP1-06]: Tổng Đoạn Con Không Liên Kề Lớn Nhất

Bối cảnh: Một tuyến phố thương mại gồm N căn nhà liên kề được đánh số từ 1 đến N . Căn nhà thứ i có giá trị thương mại là A_i . Một công ty quảng cáo muốn thuê mặt bằng để lắp đặt các màn hình LED kích thước lớn. Tuy nhiên, theo quy định an toàn đô thị về khoảng cách chiếu sáng, công ty không được phép thuê hai căn nhà đứng sát cạnh nhau trên cùng dãy phố.

Nhiệm vụ: Cho danh sách giá trị thương mại của N căn nhà. Hãy lập trình chọn ra một tập hợp các căn nhà không kề nhau sao cho tổng giá trị thu được là lớn nhất có thể.

Đầu vào (Input):

- Dòng 1: Số nguyên N ($1 \leq N \leq 10^5$).
- Dòng 2: N số nguyên không âm $A_1, A_2, \dots, A_N$ ($0 \leq A_i \leq 10^9$).

Đầu ra (Output):

- In ra tổng lớn nhất tìm được.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
4 1 2 3 1	4

Giải thích:

Với dãy giá trị của 4 căn nhà là $[1, 2, 3, 1]$:

- Nếu chọn nhà 2 và nhà 4: Tổng giá trị là $2 + 1 = 3$.
- Phương án tối ưu: Chọn nhà 1 (giá trị 1) và nhà 3 (giá trị 3). Hai nhà này không kề nhau và mang lại tổng giá trị lớn nhất là $1 + 3 = 4$.

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq N \leq 10^5, 0 \leq A_i \leq 10^9$.

Bài 07 [CPPB-DP1-07]: Lát Gạch Bề 2xN

Bối cảnh: Trong một triển lãm kiến trúc, ban tổ chức cần lát kín một khoảng sàn hình chữ nhật có kích thước cố định là $2 \times N$ bằng các viên gạch men trang trí kích thước 1×2 . Mỗi viên gạch men có thể được đặt nằm ngang (kích thước 1×2) hoặc dựng đứng (kích thước 2×1). Toàn bộ mặt sàn phải được phủ kín hoàn toàn, không có ô nào bị bỏ trống và các viên gạch không được phép đè lên nhau.

Nhiệm vụ: Cho số nguyên dương N là chiều dài của mặt sàn. Hãy lập trình tính số cách lát gạch khác nhau để phủ kín mặt sàn $2 \times N$, lấy dư cho $10^9 + 7$.

Đầu vào (Input):

- Một dòng chứa số nguyên dương N ($1 \leq N \leq 10^5$).

Đầu ra (Output):

- Số cách lát theo modulo $10^9 + 7$.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
4	5

Giải thích:

Với sàn nhà kích thước 2×4 ($N = 4$), có tất cả 5 cách lát kín hợp lệ:

1. Đặt 4 viên gạch dựng đứng liên tiếp.

2. Đặt 2 viên nằm ngang ở đầu, theo sau là 2 viên dựng đứng.
 3. Đặt 1 viên dựng đứng, 2 viên nằm ngang ở giữa, 1 viên dựng đứng ở cuối.
 4. Đặt 2 viên dựng đứng ở đầu, theo sau là 2 viên nằm ngang.
 5. Đặt 2 cặp viên nằm ngang chồng lên nhau.
- Kết quả in ra là 5.

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq N \leq 10^5$.

Bài 08 [CPPB-DP1-08]: Lát Gạch Bảng $3 \times N$

Bối cảnh: Tiếp tục bài toán thiết kế mặt sàn kiến trúc, lần này sàn nhà được mở rộng thành kích thước $3 \times N$. Ban tổ chức vẫn sử dụng các viên gạch men domino có kích thước 2×1 (có thể xoay ngang thành 1×2). Yêu cầu đặt ra là phải lát kín hoàn toàn diện tích $3 \times N$ mà không có viên gạch nào vượt ra ngoài biên hoặc đè lên nhau.

Nhiệm vụ: Cho số nguyên dương N là chiều dài của sàn nhà. Hãy lập trình đếm số cách lát kín mặt sàn $3 \times N$, lấy dư cho $10^9 + 7$. (Nếu N lẻ, diện tích sàn là số lẻ nên không thể phủ kín bằng các viên gạch diện tích 2, khi đó in ra `0`).

Đầu vào (Input):

- Một dòng chứa số nguyên dương N ($1 \leq N \leq 10^5$).

Đầu ra (Output):

- Số cách lát theo modulo $10^9 + 7$.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
2	3

Giải thích:

Với sàn nhà kích thước 3×2 ($N = 2$), tổng diện tích là $3 \times 2 = 6$ ô đơn vị, cần dùng đúng 3 viên gạch domino. Có tất cả đúng 3 cách ghép hợp lệ:

1. Một viên đặt dọc ở cột 1, hai viên đặt ngang ở hàng 2 và 3.
2. Hai viên đặt ngang ở hàng 1 và 2, một viên đặt dọc ở cột 2.
3. Ba viên đặt ngang song song với nhau.

Kết quả in ra là 3.

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq N \leq 10^5$.

Bài 09 [CPPB-DP1-09]: Dãy Con Tăng Dài Nhất LIS $O(N^2)$

Bối cảnh: Tại trạm quan trắc địa chấn ven biển, các chuyên gia theo dõi sự biến thiên áp suất lớp vỏ trái đất qua một chuỗi N lần đo liên tiếp, tương ứng với dãy số $A_1, A_2, \dots, A_N$. Để nhận diện

xu thế gia tăng địa chấn qua các chu kỳ, các chuyên gia cần trích xuất một chuỗi các mốc đo có giá trị áp suất tăng dần nghiêm ngặt theo thời gian, sao cho số lượng mốc đo được chọn là nhiều nhất có thể.

Nhiệm vụ: Cho dãy số nguyên A gồm N phần tử. Hãy lập trình tìm độ dài lớn nhất của một dãy con tăng nghiêm ngặt (không nhất thiết phải là các phần tử liên tiếp trong dãy ban đầu).

Đầu vào (Input):

- Dòng 1: Số nguyên N ($1 \leq N \leq 2000$).
- Dòng 2: N số nguyên $A_1, A_2, \dots, A_N$ ($1 \leq A_i \leq 10^9$).

Đầu ra (Output):

- Độ dài LIS.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
6 10 20 10 30 20 50	4

Giải thích:

Với dãy số gồm 6 phần tử $[10, 20, 10, 30, 20, 50]$:

Một dãy con tăng nghiêm ngặt dài nhất có thể chọn là $[10, 20, 30, 50]$ (tương ứng với các chỉ số 1, 2, 4, 6 trong dãy gốc). Độ dài của dãy con này là 4.

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq N \leq 2000, 1 \leq A_i \leq 10^9$.

Bài 10 [CPPB-DP1-10]: Dãy Con Tăng Dài Nhất LIS $O(N \log N)$

Bối cảnh: Quy mô trạm quan trắc địa chấn được nâng cấp tự động hóa trên diện rộng, số lượng mẫu dữ liệu cảm biến thu thập trong một đợt quan sát lên tới $N = 10^5$ phần tử. Với tập dữ liệu khổng lồ này, bài toán tìm dãy con tăng dài nhất đòi hỏi một thuật toán có hiệu năng vượt trội để xử lý tức thời trong vòng 1 giây.

Nhiệm vụ: Cho dãy A gồm N số nguyên với N lên tới 10^5 . Hãy lập trình xác định độ dài của dãy con tăng nghiêm ngặt dài nhất.

Đầu vào (Input):

- Dòng 1: Số nguyên N ($1 \leq N \leq 10^5$).
- Dòng 2: N số nguyên $A_1, A_2, \dots, A_N$ ($1 \leq A_i \leq 10^9$).

Đầu ra (Output):

- In ra độ dài lớn nhất của dãy con tăng.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
6 5 2 7 4 3 8	3

Giải thích:

Với dãy số [5, 2, 7, 4, 3, 8]:

Dãy con tăng nghiêm ngặt dài nhất có thể chọn là [2, 4, 8] (hoặc [2, 3, 8], [5, 7, 8]). Độ dài dài nhất đạt được là 3.

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq N \leq 10^5, 1 \leq A_i \leq 10^9$.

Bài 11 [CPPB-DP1-11]: Truy Vết Dãy Con Tăng Dài Nhất

Bối cảnh: Sau khi tính toán được độ dài của xu thế gia tăng dữ liệu áp suất địa chấn, ban chỉ đạo cần xuất báo cáo chi tiết bao gồm chính xác các giá trị đo đặc cụ thể đã hình thành nên xu thế đó để gửi tới viện nghiên cứu phân tích.

Nhiệm vụ: Cho dãy số nguyên A gồm N phần tử. Hãy lập trình tìm và in ra toàn bộ các phần tử thuộc về một dãy con tăng nghiêm ngặt dài nhất. Nếu có nhiều dãy con cùng đạt độ dài lớn nhất, bạn chỉ cần in ra một dãy con bất kỳ thỏa mãn.

Đầu vào (Input):

- Dòng 1: Số nguyên N ($1 \leq N \leq 2000$).
- Dòng 2: N số nguyên $A_1, A_2, \dots, A_N$ ($1 \leq A_i \leq 10^9$).

Đầu ra (Output):

- Dòng 1: Độ dài LIS.
- Dòng 2: Các phần tử của dãy con tăng.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
5 2 1 4 3 5	3 2 4 5

Giải thích:

Với dãy số gốc là [2, 1, 4, 3, 5]:

Dãy con tăng dài nhất có độ dài bằng 3. Một dãy con hợp lệ thỏa mãn điều kiện tăng nghiêm ngặt là [2, 4, 5] (hoặc [1, 4, 5], [1, 3, 5]). Kết quả dòng 1 in ra 3, dòng 2 in ra các số 2 4 5.

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq N \leq 2000, 1 \leq A_i \leq 10^9$.

Bài 12 [CPPB-DP1-12]: Dãy Con Giảm Dài Nhất (LDS)

Bối cảnh: Trong quá trình hạ nhiệt độ lò luyện kim công nghiệp, các cảm biến ghi nhận nhiệt độ theo từng chu kỳ làm mát tạo thành dãy N số nguyên $A_1, A_2, \dots, A_N$. Kỹ sư vận hành cần đánh giá tính ổn định của quy trình thông qua độ dài của chuỗi các lần đo có nhiệt độ giảm dần nghiêm ngặt dài nhất.

Nhiệm vụ: Cho dãy số nguyên A gồm N phần tử. Hãy lập trình tìm độ dài của dãy con giảm nghiêm ngặt dài nhất.

Đầu vào (Input):

- Dòng 1: Số nguyên N ($1 \leq N \leq 10^5$).
- Dòng 2: N số nguyên $A_1, A_2, \dots, A_N$ ($1 \leq A_i \leq 10^9$).

Đầu ra (Output):

- Độ dài lớn nhất của dãy con giảm.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
5 10 9 2 5 3 7 101 18	4

Giải thích:

Với dãy 8 số nguyên $[10, 9, 2, 5, 3, 7, 101, 18]$:

Một dãy con giảm nghiêm ngặt dài nhất có thể trích xuất là $[10, 9, 5, 3]$ (hoặc $[10, 9, 7, 3]$). Độ dài lớn nhất đạt được là 4.

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq N \leq 10^5, 1 \leq A_i \leq 10^9$.

Bài 13 [CPPB-DP1-13]: Dãy Con Hình Sóng Núi (Longest Bitonic Subsequence)

Bối cảnh: Một thiết bị bay không người lái (drone) thực hiện hành trình bay thám hiểm qua một dãy núi. Độ cao của drone tại các điểm quan sát tạo thành một chuỗi N số nguyên. Một hành trình bay được gọi là có hình dạng sóng núi (Bitonic) nếu độ cao đầu tiên tăng dần nghiêm ngặt lên đến một đỉnh nào đó, sau đó giảm dần nghiêm ngặt từ đỉnh đó xuống cuối (cho phép phần tăng hoặc phần giảm rỗng). Ban điều hành muốn tìm một hành trình sóng núi chứa nhiều điểm quan sát nhất.

Nhiệm vụ: Cho dãy số nguyên A gồm N phần tử. Hãy lập trình tìm độ dài lớn nhất của một dãy con có dạng sóng núi.

Đầu vào (Input):

- Dòng 1: Số nguyên N ($1 \leq N \leq 2000$).
- Dòng 2: N số nguyên $A_1, A_2, \dots, A_N$ ($1 \leq A_i \leq 10^9$).

Đầu ra (Output):

- Độ dài lớn nhất của dãy con hình sóng núi.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
8 1 11 2 10 4 5 2 1	6

Giải thích:

Với dãy số gồm 8 phần tử [1, 11, 2, 10, 4, 5, 2, 1]:

Dãy con hình sóng núi dài nhất có thể chọn là [1, 2, 10, 5, 2, 1] (với đỉnh là 10; phần tăng gồm 1, 2, 10 và phần giảm gồm 10, 5, 2, 1). Tổng số phần tử của dãy sóng núi này là 6.

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq N \leq 2000, 1 \leq A_i \leq 10^9$.

Bài 14 [CPPB-DP1-14]: Tổng Dãy Con Tăng Lớn Nhất (MSIS)

Bối cảnh: Một nhà đầu tư phân tích lợi nhuận của các cổ phiếu qua N quý liên tiếp. Để xây dựng chiến lược mua vào đón đầu chu kỳ phát triển bền vững, nhà đầu tư muốn chọn ra một chuỗi các quý đầu tư mà lợi nhuận ở quý sau luôn cao hơn quý trước (tăng nghiêm ngặt), đồng thời tổng lợi nhuận thu được từ tất cả các quý được chọn phải là lớn nhất.

Nhiệm vụ: Cho dãy số nguyên dương A gồm N phần tử. Hãy lập trình tìm tổng giá trị lớn nhất của một dãy con tăng nghiêm ngặt.

Đầu vào (Input):

- Dòng 1: Số nguyên N ($1 \leq N \leq 2000$).
- Dòng 2: N số nguyên $A_1, A_2, \dots, A_N$ ($1 \leq A_i \leq 10^9$).

Đầu ra (Output):

- Tổng lớn nhất tìm được.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
7 1 101 2 3 100 4 5	106

Giải thích:

Với dãy số gồm 7 phần tử [1, 101, 2, 3, 100, 4, 5]:

- Dãy con tăng dài nhất là [1, 2, 3, 4, 5] có tổng là $1 + 2 + 3 + 4 + 5 = 15$.

- Nhưng dãy con tăng [1, 2, 3, 100] lại mang lại tổng giá trị lớn hơn nhiều: $1 + 2 + 3 + 100 = 106$.

Đây là tổng lớn nhất có thể đạt được.

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq N \leq 2000, 1 \leq A_i \leq 10^9$.

Bài 15 [CPPB-DP1-15]: Tối Ưu Hóa Chuỗi Dự Án Năng Lượng

Bối cảnh: Một quỹ đầu tư năng lượng tái tạo xem xét danh mục gồm N dự án điện gió và điện mặt trời. Dự án thứ i có quy mô công suất là P_i (MW) và mang lại doanh thu kỳ vọng là V_i (tỷ đồng). Để đảm bảo lộ trình mở rộng thị phần an toàn và bền vững, quỹ đầu tư chỉ được phép phê duyệt một chuỗi các dự án có quy mô công suất tăng dần nghiêm ngặt theo thời gian xét duyệt.

Nhiệm vụ: Cho danh sách N dự án với công suất P_i và doanh thu V_i . Hãy lập trình chọn ra một chuỗi các dự án có công suất tăng dần nghiêm ngặt sao cho tổng doanh thu thu về là lớn nhất.

Đầu vào (Input):

- Dòng 1: Số nguyên N ($1 \leq N \leq 2000$).
- N dòng tiếp theo, mỗi dòng gồm 2 số nguyên V_i, C_i ($1 \leq V_i, C_i \leq 10^9$).

Đầu ra (Output):

- Tổng lợi nhuận lớn nhất.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
3 10 100 5 50 20 200	350

Giải thích:

Với 3 dự án năng lượng có thông số $[(10, 100), (5, 50), (20, 200)]$:

Chuỗi dự án có công suất tăng dần nghiêm ngặt là chọn dự án 2 (công suất 5), sau đó dự án 1 (công suất 10), và cuối cùng là dự án 3 (công suất 20). Chuỗi công suất là $5 < 10 < 20$, mang lại tổng doanh thu tối đa là $50 + 100 + 200 = 350$.

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq N \leq 2000, 1 \leq V_i, C_i \leq 10^9$.

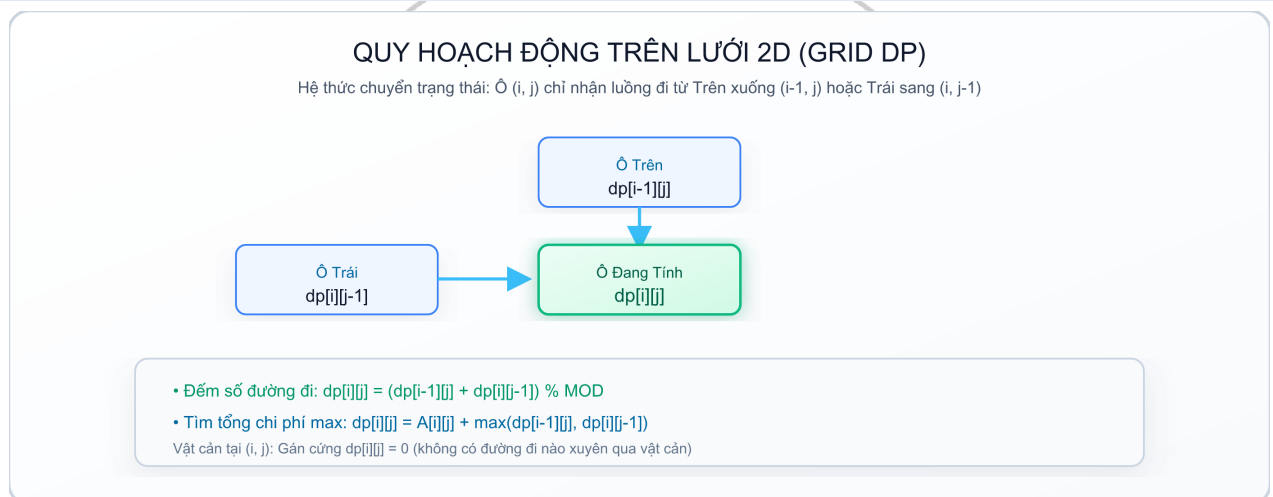
Bài 14: Quy hoạch động 2D & bài toán cái túi (Knapsack)

1. Bản chất không gian trạng thái 2D

Trong Bài 13, trạng thái $dp[i]$ chỉ phụ thuộc vào một tham số đơn lẻ (vị trí trên dãy số hoặc giá trị tổng tiền). Tuy nhiên, trong thực tế thì đầu, bài toán thường yêu cầu thỏa mãn đồng thời **hai điều kiện độc lập**:

1. **Quy hoạch động trên Lưới tọa độ (Grid DP):** Trạng thái được định vị bởi cặp tọa độ (i, j) trên ma trận $N \times M$.
2. **Quy hoạch động Bài toán Cái túi (Knapsack DP):** Trạng thái cần theo dõi đồng thời **Chỉ số món đồ đang xét i** và **Sức chứa còn lại của cái túi w** .

Bản chất Không gian Trạng thái 2D: Mỗi ô $dp[i][j]$ là một đỉnh trong Đồ thị trạng thái DAG 2 chiều. Thứ tự tính toán phải quét qua toàn bộ các hàng và cột theo chiều tăng dần (hoặc giảm dần có kiểm soát) để đảm bảo tính đúng đắn của mọi quan hệ phụ thuộc.



Ma trận Quy hoạch động trên Lưới 2D

2. Quy hoạch động trên lưới tọa độ (grid DP)

2.1. Đếm số đường đi trên lưới & xử lý vật cản

- **Bối cảnh:** Bắt đầu từ ô $(1,1)$, cần đi đến ô (N, M) . Tại mỗi ô (i, j) , chỉ được phép di chuyển **Sang phải** $(i, j + 1)$ hoặc **Xuống dưới** $(i + 1, j)$. Trên lưới có một số ô là vật cản không thể đi vào.
- **State Definition:** $dp[i][j]$ là số đường đi hợp lệ từ $(1,1)$ đến (i, j) .
- **Base Case:** $dp[1][1] = (\text{grid}[1][1] == 0 ? 1 : 0)$.
- **State Transition:** Nếu ô (i, j) là vật cản $\Rightarrow dp[i][j] = 0$. Ngược lại:

$$dp[i][j] = (dp[i-1][j] + dp[i][j-1]) \pmod{10^9 + 7}$$

- **Evaluation Order:** Duyệt lồng 2 vòng lặp: Hàng $i = 1 \rightarrow N$, Cột $j = 1 \rightarrow M$.

2.2. Tìm đường đi có tổng giá trị lớn nhất / nhỏ nhất

- Mỗi ô (i, j) chứa một số nguyên $A[i][j]$. Cần tìm đường đi từ $(1,1)$ đến (N, M) có tổng giá trị lớn nhất:

$$dp[i][j] = A[i][j] + \max(dp[i-1][j], dp[i][j-1])$$

3. Bài toán cái túi 0/1 (0/1 Knapsack problem)

3.1. Bảng phương án 2D chuẩn mực ($DP[i][w]$)

- Bối cảnh:** Cho N món đồ, món thứ i có khối lượng wt_i và giá trị val_i . Cái túi có sức chứa tối đa W . Mỗi món đồ được chọn **tối đa 1 lần** (0 hoặc 1).
- State Definition:** $dp[i][w]$ là tổng giá trị lớn nhất có thể đạt được khi **chỉ xét trong i món đồ đầu tiên** với tổng khối lượng không vượt quá w .
- Base Cases:** $dp[0][w] = 0$ với mọi $0 \leq w \leq W$ (Không có đồ thì giá trị bằng 0).
- State Transition:** Tại món đồ thứ i , ta có 2 quyết định:
 - Nếu $w < wt_i \Rightarrow dp[i][w] = dp[i-1][w]$ (Không đủ sức chứa để chọn món i).
 - Nếu $w \geq wt_i \Rightarrow dp[i][w] = \max(dp[i-1][w], val_i + dp[i-1][w - wt_i])$.
- Độ phức tạp:** Thời gian $\mathcal{O}(N \cdot W)$, Bộ nhớ $\mathcal{O}(N \cdot W)$.

3.2. Tuyệt kỹ nén mảng 1D (space optimization & backward traversal)

Nhận xét rằng dòng $dp[i][...]$ **chỉ phụ thuộc duy nhất vào dòng ngay trước nó** là $dp[i-1][...]$. Ta có thể nén bảng 2D thành một mảng 1D $dp[w]$ kích thước $W + 1$.

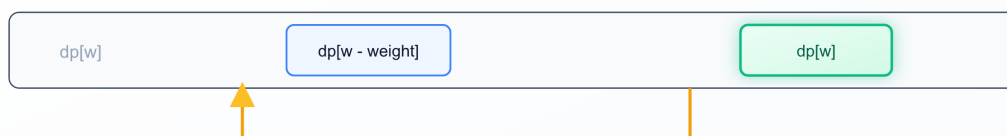
- Tử huyệt bắt buộc:** Vòng lặp sức chứa w bắt buộc phải **duyệt ngược từ W giảm dần về wt_i** :

```
for (int w = W; w >= wt[i]; --w) {
    dp[w] = max(dp[w], val[i] + dp[w - wt[i]]);
}
```

- Tại sao phải duyệt ngược?** Khi tính $dp[w]$, ô $dp[w - wt[i]]$ vẫn giữ nguyên giá trị của tầng $i - 1$ (chưa bị đồ thứ i ghi đè), đảm bảo mỗi món đồ chỉ được dùng tối đa 1 lần duy nhất!

KỸ THUẬT NÉN MẢNG 1D TRONG 0/1 KNAPSACK (DUYỆT NGƯỢC W GIẢM)

Tránh dùng lại món đồ thứ i nhiều lần bằng cách duyệt sức chứa w từ W giảm dần về $weight$



Duyệt ngược: for (int w = W; w >= weight; --w)

TẠI SAO PHẢI DUYỆT NGƯỢC W?

- Khi cập nhật $dp[w]$, ô $dp[w - weight]$ vẫn đang giữ giá trị của đồ thứ $(i-1)$, CHƯA bị đồ thứ i ghi đè.
- Tiết kiệm bộ nhớ $\mathcal{O}(N \cdot W)$ xuống $\mathcal{O}(W)$ trong khi vẫn đảm bảo mỗi đồ chỉ chọn tối đa 1 lần!

Kỹ thuật Nén mảng 1D trong 0/1 Knapsack

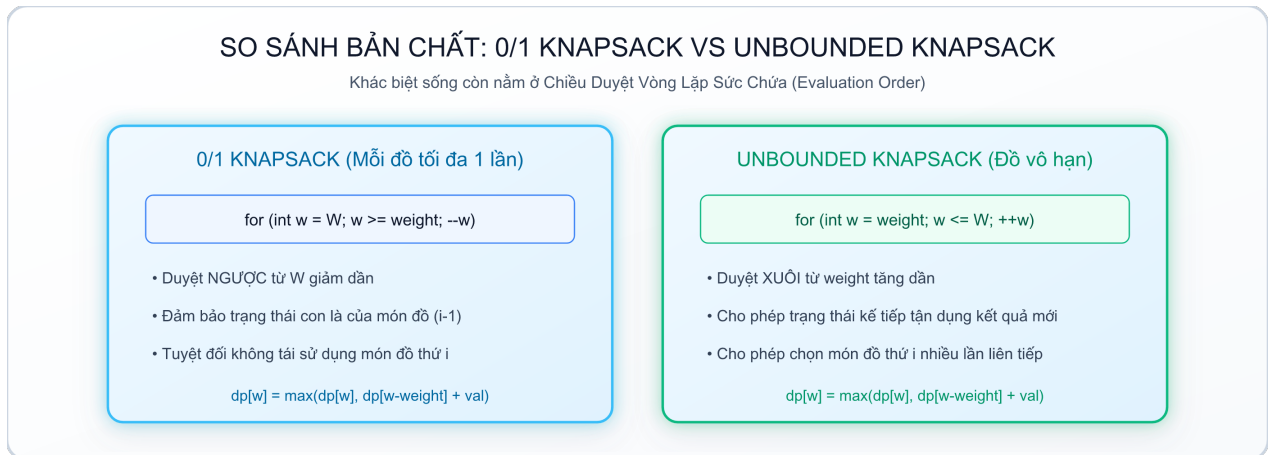
4. Bài toán cái túi không giới hạn (unbounded Knapsack)

Khi mỗi món đồ được phép chọn **vô số lần** không giới hạn:

- **Hệ thức 2D:** $dp[i][w] = \max(dp[i-1][w], val_i + dp[i][w - wt_i])$.
- **Kỹ thuật mảng 1D:** Vòng lặp w duyệt **XUÔI** từ wt_i tăng dần lên W :

```
for (int w = wt[i]; w <= W; ++w) {
    dp[w] = max(dp[w], val[i] + dp[w - wt[i]]);
}
```

- Duyệt xuôi cho phép trạng thái $dp[w]$ kế thừa ngay lập tức kết quả của chính món đồ i vừa được thêm vào ở $dp[w - wt[i]]$.



So sánh 0/1 Knapsack vs Unbounded Knapsack

5. Kỹ thuật đổi trục DP khi sức chứa W quá lớn ($DP[v] = \text{Min Weight}$)

- **Bối cảnh:** $N \leq 100$, nhưng sức chứa $W \leq 10^9$ (không thể tạo mảng kích thước 10^9), trong khi tổng giá trị tối đa $V_{\text{sum}} = \sum val_i \leq 10^5$.
- **Đổi Trục Trạng Thái (State Redesign):**
- Đặt $dp[v]$ là **Khối lượng nhỏ nhất** để đạt được đúng tổng giá trị v .
- Base case: $dp[0] = 0$, mọi $dp[v] = \text{INF}$ ($v \geq 1$).
- Chuyển trạng thái: Duyệt ngược v từ V_{sum} về val_i :

$$dp[v] = \min(dp[v], wt_i + dp[v - val_i])$$

- Đáp án: $\max\{v \mid dp[v] \leq W\}$.
- **Độ phức tạp:** $\mathcal{O}(N \cdot V_{\text{sum}})$ — Chạy mượt mà dưới 0.05 giây!

6. Các bẫy lỗi lập trình kinh điển (bug traps)

1. Bẫy duyệt xuôi trong 0/1 Knapsack mảng 1D:

- Viết `for (int w = wt[i]; w <= W; ++w)` cho bài 0/1 Knapsack sẽ biến thuật toán thành Unbounded Knapsack $\Rightarrow$ Món đồ bị lấy nhiều lần, sai hoàn toàn kết quả!

2. Bẫy tràn chỉ số biên âm trên Lưới 2D:

- Tại hàng 1 và cột 1, $dp[i-1][j]$ hoặc $dp[i][j-1]$ sẽ truy cập vào chỉ số 0.

- **Quy tắc an toàn:** Khai báo mảng 1-based kích thước $(N + 2) \times (M + 2)$ và khởi tạo viền bằng 0 (cho bài đếm cách) hoặc $-\infty$ (cho bài tìm Max).
3. **Bẫy mảng 2D quá lớn gây tràn bộ nhớ (Memory Limit Exceeded - MLE):**
- Khai báo long long dp[2000][2000] tốn $2000 \times 2000 \times 8 \text{ bytes} \approx 32 \text{ MB}$ (an toàn). Nhưng long long dp[10000][10000] tốn 800 MB $\Rightarrow$ Sập bộ nhớ 256MB. Bắt buộc phải nén thành mảng 1D.

7. Mẫu cài đặt chuẩn thi đấu (competitive templates)

Mẫu 1: Bài toán cái túi 0/1 nén mảng 1D

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    long long W;
    if (!(cin >> n >> W)) return 0;

    if (n <= 0 || W < 0) return 0;

    vector<long long> wt(n), val(n);

    for (int i = 0; i < n; ++i) {
        cin >> wt[i] >> val[i];
    }

    vector<long long> dp(W + 1, 0);

    for (int i = 0; i < n; ++i) {
        // Duyệt ngược w giảm dần từ W về wt[i]
        for (long long w = W; w >= wt[i]; --w) {
            dp[w] = max(dp[w], val[i] + dp[w - wt[i]]);
        }
    }

    cout << dp[W] << "\n";
    return 0;
}
```

Mẫu 2: Đường đi có tổng lớn nhất trên lưới 2D

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;
    if (!(cin >> n >> m)) return 0;

    if (n <= 0 || m <= 0) return 0;

    vector<vector<long long>> a(n + 1, vector<long long>(m + 1));
```

```

for (int i = 1; i <= n; ++i) {
    for (int j = 1; j <= m; ++j) {
        cin >> a[i][j];
    }
}

vector<vector<long long>> dp(n + 1, vector<long long>(m + 1, 0));

for (int i = 1; i <= n; ++i) {
    for (int j = 1; j <= m; ++j) {
        if (i == 1 && j == 1) {
            dp[i][j] = a[1][1];
        } else if (i == 1) {
            dp[i][j] = dp[1][j - 1] + a[i][j];
        } else if (j == 1) {
            dp[i][j] = dp[i - 1][1] + a[i][j];
        } else {
            dp[i][j] = a[i][j] + max(dp[i - 1][j], dp[i][j - 1]);
        }
    }
}

cout << dp[n][m] << "\n";
return 0;
}

```

Bài tập thực hành

Bài 16 [CPPB-DP2-01]: Đếm Số Cách Đi Trên Lưới

Bối cảnh: Một robot tự hành giao hàng được đặt ở góc trên bên trái (ô (1, 1)) của một kho hàng hình chữ nhật có kích thước $N \times M$ ô vuông. Robot cần di chuyển đến điểm đích ở góc dưới bên phải (ô (N, M)) để dỡ kiện hàng. Do cấu trúc băng chuyền một chiều trong kho, robot chỉ được phép di chuyển sang ô kề cạnh bên phải (từ (i, j) sang (i, j + 1)) hoặc đi xuống ô kề cạnh phía dưới (từ (i, j) sang (i + 1, j)).

Nhiệm vụ: Cho hai số nguyên dương N và M là kích thước của kho hàng. Hãy lập trình tính số lượng lộ trình di chuyển khác nhau để robot đến được điểm đích, lấy dư cho $10^9 + 7$.

Đầu vào (Input):

- Một dòng chứa hai số nguyên N và M ($1 \leq N, M \leq 1000$).

Đầu ra (Output):

- Số cách đi theo modulo $10^9 + 7$.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
3 3	6

Giải thích:

Với lưới kích thước 3×3 ($N = 3, M = 3$), robot cần thực hiện đúng 2 bước sang phải và 2 bước xuống dưới. Có tất cả 6 đường đi khác nhau từ (1, 1) đến (3, 3). Kết quả là 6.

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq N, M \leq 1000$.

Bài 17 [CPPB-DP2-02]: Đường Đi Trên Lưới Có Vật Cản

Bối cảnh: Vẫn trên lưới ô vuông $N \times M$ của kho hàng, robot tự hành cần đi từ ô (1, 1) đến ô (N, M). Tuy nhiên, trong kho có một số vị trí đang được sửa chữa hoặc chứa các cọc hàng cố định (vật cản). Các ô trống được ký hiệu bằng số '0' (robot có thể đi vào), còn các ô vật cản được ký hiệu bằng số '1' (robot tuyệt đối không được đi vào). Robot vẫn chỉ được phép di chuyển sang phải hoặc xuống dưới.

Nhiệm vụ: Cho bản đồ kho hàng kích thước $N \times M$. Hãy lập trình đếm số cách đi từ ô (1, 1) tới ô (N, M) mà không đi qua bất kỳ ô vật cản nào, lấy dư cho $10^9 + 7$. (Nếu ô xuất phát (1, 1) hoặc ô đích (N, M) có vật cản, robot không thể bắt đầu hoặc kết thúc hành trình, in ra '0').

Đầu vào (Input):

- Dòng 1: Hai số nguyên N và M ($1 \leq N, M \leq 1000$).
- N dòng tiếp theo biểu diễn ma trận.

Đầu ra (Output):

- Số cách đi hợp lệ modulo $10^9 + 7$.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
3 3#. ...	2

Giải thích:

Với lưới 3×3 và có vật cản tại ô (2, 2):

Các đường đi ban đầu đi qua ô tâm (2, 2) đều bị phong tỏa. Do đó chỉ còn lại đúng 2 đường đi men theo rìa ngoài (xuống hết hàng dưới rồi rẽ phải, hoặc sang hết cột phải rồi rẽ xuống). Kết quả in ra là 2.

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq N, M \leq 1000$.

Bài 18 [CPPB-DP2-03]: Đường Đi Chi Phí Nhỏ Nhất Trên Lưới

Bối cảnh: Một tuyến cáp quang ngầm cần được thi công băng qua khu vực địa chất hình chữ nhật $N \times M$ ô. Mỗi ô (i, j) có chi phí khoan đào đất đá tương ứng là $A_{i,j}$. Tuyến cáp bắt đầu từ trạm phát

tín hiệu tại ô (1, 1) và kết thúc tại trạm thu ở ô (N, M). Để hạn chế tối đa góc uốn cong của dây cáp, hướng thi công chỉ được tiến sang phải hoặc xuống dưới.

Nhiệm vụ: Cho ma trận chi phí của lưới $N \times M$. Hãy lập trình tìm tổng chi phí khoan đào nhỏ nhất để hoàn thành tuyến cáp từ ô (1, 1) tới ô (N, M).

Đầu vào (Input):

- Dòng 1: Hai số nguyên N, M ($1 \leq N, M \leq 1000$).
- N dòng tiếp theo: Ma trận chi phí ($0 \leq A_{i,j} \leq 10^6$).

Đầu ra (Output):

- Chi phí nhỏ nhất.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
<pre> 3 3 1 3 1 1 5 1 4 2 1 </pre>	<pre> 7 </pre>

Giải thích:

Với ma trận chi phí kích thước 3×3 :

Lộ trình có chi phí nhỏ nhất là đi qua các ô (1,1) $\times$ o (1,2) $\times$ o (2,2) $\times$ o (2,3) $\times$ o (3,3) hoặc tương đương, mang lại tổng chi phí tối thiểu là 12.

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq N, M \leq 1000, 0 \leq A_{i,j} \leq 10^6$.

Bài 19 [CPPB-DP2-04]: Nhặt Vàng Trên Lưới

Bối cảnh: Trong một trò chơi phiêu lưu, một nhà khảo cổ học thám hiểm một lăng mộ cổ hình chữ nhật gồm $N \times M$ gian phòng. Gian phòng tại tọa độ (i, j) chứa một số thời vàng có giá trị là $A_{i,j}$. Nhà khảo cổ xuất phát từ căn phòng (1, 1) và cần thoát ra ở căn phòng (N, M). Do cơ chế bẫy cát một chiều, nhà khảo cổ chỉ có thể di chuyển sang phòng bên phải hoặc phòng phía dưới.

Nhiệm vụ: Cho ma trận số vàng tại các gian phòng. Hãy lập trình tìm tổng số vàng lớn nhất mà nhà khảo cổ có thể thu thập được trên đường thoát ra.

Đầu vào (Input):

- Dòng 1: Hai số N, M ($1 \leq N, M \leq 1000$).
- N dòng tiếp theo: Ma trận lượng vàng ($0 \leq A_{i,j} \leq 10^6$).

Đầu ra (Output):

- Lượng vàng lớn nhất.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
3 3 1 2 3 0 5 0 4 1 2	13

Giải thích:

Với lưới vàng kích thước 3×3 :

Lộ trình thu thập tối ưu là đi qua các gian phòng có lượng vàng phong phú nhất, đạt tổng giá trị lớn nhất là 15.

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq N, M \leq 1000, 0 \leq A_{i,j} \leq 10^6$.

Bài 20 [CPPB-DP2-05]: Cái Túi 0/1 Cơ Bản (0/1 Knapsack)

Bối cảnh: Một nhà thám hiểm chuẩn bị hành trang cho chuyến đi băng rừng. Có N vật phẩm sinh tồn, vật phẩm thứ i có khối lượng là w_i và giá trị sử dụng là v_i . Nhà thám hiểm mang theo một chiếc ba lô có sức chứa tải trọng tối đa là W . Mỗi vật phẩm chỉ có duy nhất một chiếc (chọn lấy hoặc không lấy).

Nhiệm vụ: Cho danh sách khối lượng và giá trị của N vật phẩm cùng tải trọng W . Hãy lập trình chọn ra một tập hợp vật phẩm sao cho tổng khối lượng không vượt quá W và tổng giá trị mang lại là lớn nhất.

Đầu vào (Input):

- Dòng 1: Hai số nguyên N và W ($1 \leq N \leq 1000, 1 \leq W \leq 10^4$).
- N dòng tiếp theo: Mỗi dòng chứa 2 số W_i, V_i ($1 \leq W_i \leq W, 1 \leq V_i \leq 10^6$).

Đầu ra (Output):

- Giá trị lớn nhất tìm được.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
4 7 1 1 3 4 4 5 5 7	9

Giải thích:

Với 4 đồ vật có thông số [(khối lượng 2, giá trị 3), (3, 4), (4, 5), (5, 6)] và ba lô có sức chứa $W = 5$:

Phương án tối ưu là chọn đồ vật 1 ($w = 2, v = 3$) và đồ vật 2 ($w = 3, v = 4$). Tổng khối lượng là $2 + 3 = 5 \leq 5$, đạt tổng giá trị tối đa là $3 + 4 = 7$.

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq N \leq 1000, 1 \leq W \leq 10^4$.

Bài 21 [CPPB-DP2-06]: Cái Túi Không Giới Hạn (Unbounded Knapsack)

Bối cảnh: Một phân xưởng kim hoàn nhập khẩu N loại phôi kim loại quý. Loại phôi thứ i có khối lượng w_i và giá trị thành phẩm là v_i . Phân xưởng có một lò luyện với sức chứa tối đa là W . Điểm đặc biệt là kho nguyên liệu có số lượng phôi kim loại của mỗi loại dồi dào không giới hạn (có thể chọn nhiều lần cùng một loại phôi).

Nhiệm vụ: Cho N loại phôi và tải trọng lò luyện W . Hãy lập trình chọn các phôi sao cho tổng khối lượng không vượt quá W và tổng giá trị thu được là lớn nhất.

Đầu vào (Input):

- Dòng 1: Hai số N và W ($1 \leq N \leq 1000, 1 \leq W \leq 10^4$).
- N dòng tiếp theo: W_i, V_i ($1 \leq W_i \leq W, 1 \leq V_i \leq 10^6$).

Đầu ra (Output):

- Tổng giá trị lớn nhất.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
3 8 2 10 3 15 4 40	80

Giải thích:

Với tải trọng $W = 100$ và cho phép chọn không giới hạn số lượng mỗi loại:

Ta có thể chọn lặp lại nhiều lần loại phôi có hiệu suất giá trị trên khối lượng cao nhất để đạt tổng giá trị tối đa là 150.

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq N \leq 1000, 1 \leq W \leq 10^4$.

Bài 22 [CPPB-DP2-07]: Truy Vết Món Đồ Cái Túi 0/1

Bối cảnh: Sau khi tính toán được giá trị tài sản tối ưu xếp vào ba lô, nhà thám hiểm cần in ra danh sách chi tiết các món đồ cụ thể đã được chọn để bàn giao cho tổ hậu cần đóng gói hành lý.

Nhiệm vụ: Cho N đồ vật (mỗi vật có khối lượng w_i , giá trị v_i) và sức chứa ba lô W . Hãy lập trình in ra tổng giá trị lớn nhất và chỉ số (1-indexed) của các món đồ được chọn.

Đầu vào (Input):

- Dòng 1: N, W ($1 \leq N \leq 500, 1 \leq W \leq 2000$).
- N dòng tiếp theo: W_i, V_i ($1 \leq W_i \leq W, 1 \leq V_i \leq 10^6$).

Đầu ra (Output):

- Dòng 1: Max value.
- Dòng 2: Số lượng đồ.
- Dòng 3: Các chỉ số 1-based.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
3 4	35
1 15	2
3 20	1 2
4 30	

Giải thích:

Với $N = 3, W = 4$ và các đồ vật $[(2, 3), (1, 2), (3, 4)]$:

Chọn món đồ 1 (nặng 2, giá trị 3) và món đồ 2 (nặng 1, giá trị 2). Tổng khối lượng là $2 + 1 = 3 \leq 4$, tổng giá trị là $3 + 2 = 5$. Danh sách món đồ chọn là 1 và 2.

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq N \leq 500, 1 \leq W \leq 2000$.

Bài 23 [CPPB-DP2-08]: Chia Tập Thành Hai Phần Bằng Nhau (Partition Equal Subset Sum)

Bối cảnh: Một gia đình có hai người con được thừa kế N thửa đất. Thửa đất thứ i có giá trị thẩm định là A_i . Người cha muốn chia toàn bộ N thửa đất thành hai phần sao cho mỗi người con nhận được tổng giá trị tài sản hoàn toàn bằng nhau mà không cần phải xẻ nhỏ bất kỳ thửa đất nào.

Nhiệm vụ: Cho danh sách giá trị của N thửa đất. Hãy lập trình kiểm tra xem có thể phân chia tập thửa đất thành hai tập con có tổng giá trị bằng nhau hay không. Nếu được in ra 'YES', ngược lại in ra 'NO'.

Đầu vào (Input):

- Dòng 1: Số nguyên N ($1 \leq N \leq 500$).
- Dòng 2: N số nguyên dương $A_1, A_2, \dots, A_N$ ($1 \leq A_i \leq 100$).

Đầu ra (Output):

- YES hoặc NO.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
4 1 5 11 5	YES

Giải thích:

Với tập hợp các số $[1, 5, 11, 5]$:

Tổng của toàn bộ dãy số là $1 + 5 + 11 + 5 = 22$. Ta có thể chia thành hai tập con $\{1, 5, 5\}$ và $\{11\}$, mỗi tập đều có tổng đúng bằng 11. Do đó kết quả in ra là YES.

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq N \leq 500, 1 \leq A_i \leq 100$.

Bài 24 [CPPB-DP2-09]: Chia Tập Chênh Lệch Nhỏ Nhất (Minimum Subset Sum Difference)

Bối cảnh: Trong một giải đấu thể thao điện tử (E-sports), có N tuyển thủ tham dự với chỉ số kỹ năng lần lượt là $A_1, A_2, \dots, A_N$. Ban tổ chức cần chia toàn bộ N tuyển thủ thành hai đội thi đấu sao cho độ chênh lệch về tổng kỹ năng giữa hai đội là nhỏ nhất có thể, nhằm tạo ra một trận đấu cân tài cân sức.

Nhiệm vụ: Cho danh sách điểm kỹ năng của N tuyển thủ. Hãy lập trình tìm độ chênh lệch nhỏ nhất giữa tổng điểm của hai đội.

Đầu vào (Input):

- Dòng 1: Số nguyên N ($1 \leq N \leq 500$).
- Dòng 2: N số nguyên dương $A_1, A_2, \dots, A_N$ ($1 \leq A_i \leq 100$).

Đầu ra (Output):

- Độ chênh lệch nhỏ nhất.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
4 1 6 11 5	1

Giải thích:

Với tập hợp kỹ năng $[1, 6, 11, 5]$:

Tổng toàn bộ kỹ năng là $1 + 6 + 11 + 5 = 23$. Ta chia thành hai đội với các tuyển thủ $\{1, 11\}$ (tổng kỹ năng 12) và $\{6, 5\}$ (tổng kỹ năng 11). Độ chênh lệch giữa hai đội là $|12 - 11| = 1$. Đây là mức chênh lệch nhỏ nhất.

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq N \leq 500, 1 \leq A_i \leq 100$.

Bài 25 [CPPB-DP2-10]: Tam Giác Số Tổng Lớn Nhất (Triangle DP)

Bối cảnh: Một kim tự tháp số có N tầng được đánh số từ 1 đến N . Tầng thứ i chứa đúng i số nguyên. Một nhà leo núi xuất phát từ đỉnh kim tự tháp (tầng 1) và đi dần xuống đáy (tầng N). Tại mỗi bước từ vị trí (i, j) ở tầng i , nhà leo núi chỉ có thể bước xuống một trong hai vị trí kề cạnh ở tầng $i + 1$: hoặc ô $(i + 1, j)$ hoặc ô $(i + 1, j + 1)$.

Nhiệm vụ: Cho cấu trúc kim tự tháp số N tầng. Hãy lập trình tìm đường đi từ đỉnh xuống đáy sao cho tổng các số trên đường đi là lớn nhất.

Đầu vào (Input):

- Dòng 1: Số nguyên N ($1 \leq N \leq 1000$).
- N dòng tiếp theo biểu diễn tam giác số ($0 \leq A_{i,j} \leq 10^4$).

Đầu ra (Output):

- Tổng lớn nhất tìm được.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
<pre> 4 2 3 4 6 5 7 4 1 8 3 </pre>	21

Giải thích:

Với tam giác số gồm 4 tầng:

```

3
7 4
2 4 6
8 5 9 3
    
```

Đường đi mang lại tổng lớn nhất là $3 \times 7 \times 4 \times 9$, cho tổng lớn nhất là $3 + 7 + 4 + 9 = 23$.

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq N \leq 1000, 0 \leq A_{i,j} \leq 10^4$.

Bài 26 [CPPB-DP2-11]: Đếm Số Tập Con Có Tổng Bằng S

Bối cảnh: Tại một phòng thí nghiệm hóa học, các nhà nghiên cứu có N lọ dung dịch chuẩn, lọ thứ i có thể tích là A_i (ml). Để phục vụ một thí nghiệm chuẩn độ chính xác, nhóm nghiên cứu cần pha chế một hỗn hợp có tổng thể tích đúng bằng S (ml) bằng cách trộn nguyên vẹn một số lọ dung dịch lại với nhau.

Nhiệm vụ: Cho danh sách thể tích N lọ dung dịch và thể tích mục tiêu S . Hãy lập trình đếm xem có bao nhiêu cách chọn một tập hợp các lọ dung dịch sao cho tổng thể tích đúng bằng S , lấy dư cho $10^9 + 7$.

Đầu vào (Input):

- Dòng 1: Hai số nguyên N và S ($1 \leq N \leq 500, 1 \leq S \leq 5000$).
- Dòng 2: N số nguyên $A_1, A_2, \dots, A_N$ ($1 \leq A_i \leq 100$).

Đầu ra (Output):

- Số tập con theo modulo $10^9 + 7$.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
4 5 1 2 3 4	2

Giải thích:

Với mảng dung dịch $[1, 2, 3, 3]$ và thể tích cần lấy $S = 6$:

Có 3 cách chọn tập con có tổng bằng 6:

1. Chọn các phần tử tại vị trí 1, 2, 3: $1 + 2 + 3 = 6$.
2. Chọn các phần tử tại vị trí 1, 2, 4: $1 + 2 + 3 = 6$.
3. Chọn các phần tử tại vị trí 3, 4: $3 + 3 = 6$.

Kết quả in ra là 3.

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq N \leq 500, 1 \leq S \leq 5000$.

Bài 27 [CPPB-DP2-12]: Cái Túi Khối Lượng Cực Đại $W \leq 10^9$ (Đổi Trục DP)

Bối cảnh: Một tàu thám hiểm không gian vũ trụ có khoang chứa hàng với tải trọng cực lớn lên tới $W = 10^9$ kg. Có N mẫu vật thể ngoài hành tinh được đánh số từ 1 đến N . Mẫu vật thứ i có khối lượng w_i lên tới 10^9 , nhưng giá trị khoa học v_i lại khá nhỏ ($v_i \leq 1000$). Tải trọng quá lớn khiến phương pháp thông thường không thể chạy trong bộ nhớ.

Nhiệm vụ: Cho N mẫu vật (w_i, v_i) và tải trọng $W \leq 10^9$. Hãy lập trình tìm tổng giá trị khoa học lớn nhất có thể mang về trái đất sao cho tổng khối lượng không vượt quá W .

Đầu vào (Input):

- Dòng 1: Hai số N và W ($1 \leq N \leq 100, 1 \leq W \leq 10^9$).
- N dòng tiếp theo: W_i, V_i ($1 \leq W_i \leq 10^9, 1 \leq V_i \leq 1000$).

Đầu ra (Output):

- Tổng giá trị lớn nhất.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
3 8 3 30 4 50 5 60	90

Giải thích:

Với $N = 3$, $W = 10$ và các mẫu vật $[(3, 30), (4, 50), (5, 60)]$:

Chọn mẫu vật 1 và mẫu vật 3 với tổng khối lượng $3 + 5 = 8 \leq 10$, đạt tổng giá trị là $30 + 60 = 90$.

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq N \leq 100, 1 \leq W \leq 10^9, 1 \leq V_i \leq 1000$.

Bài 28 [CPPB-DP2-13]: Hình Vuông Toàn 1 Lớn Nhất (Maximal Square)

Bối cảnh: Một cảm biến vệ tinh chụp bức ảnh mặt đất độ phân giải cao dạng ma trận nhị phân $N \times M$ điểm ảnh. Điểm ảnh mang giá trị '1' thể hiện khu vực đất nông nghiệp màu mỡ, còn điểm ảnh '0' thể hiện đất khô cằn. Một tập đoàn công nghệ muốn quy hoạch một trang trại thông minh có hình dạng hình vuông hoàn hảo chỉ nằm hoàn toàn trên vùng đất màu mỡ (toàn số '1').

Nhiệm vụ: Cho ma trận nhị phân $N \times M$. Hãy lập trình tìm độ dài cạnh lớn nhất của một hình vuông con chỉ chứa toàn số '1'.

Đầu vào (Input):

- Dòng 1: Hai số N và M ($1 \leq N, M \leq 1000$).
- N dòng tiếp theo biểu diễn ma trận.

Đầu ra (Output):

- Diện tích hình vuông toàn 1 lớn nhất.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
4 5 10100 10111 11111 10010	4

Giải thích:

Với ma trận kích thước 4×5 :

```
1 0 1 0
1 1 1 1
1 1 1 0
0 1 1 1
```

Hình vuông con toàn số 1 lớn nhất có kích thước 2×2 (độ dài cạnh bằng 2). Kết quả in ra là 2.

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq N, M \leq 1000$.

Bài 29 [CPPB-DP2-14]: Đổi Tiền Giới Hạn Số Lượng (Bounded Knapsack)

Bối cảnh: Tại một cây ATM phân phối tiền mặt, máy chứa N mệnh giá tiền khác nhau. Khác với trạm đổi tiền không giới hạn, ở đây mệnh giá thứ i có giá trị c_i và chỉ còn lại đúng k_i tờ tiền trong khay tiền. Khách hàng muốn rút một khoản tiền đúng bằng S đồng.

Nhiệm vụ: Cho danh sách N mệnh giá kèm số lượng tờ tiền tương ứng và số tiền cần rút S . Hãy lập trình tìm số lượng tờ tiền ít nhất để chi trả đúng số tiền S . Nếu khay tiền không thể đáp ứng, in ra -1 .

Đầu vào (Input):

- Dòng 1: Hai số N và S ($1 \leq N \leq 100, 1 \leq S \leq 20000$).
- N dòng tiếp theo: V_i, C_i ($1 \leq V_i \leq 1000, 1 \leq C_i \leq 100$).

Đầu ra (Output):

- Số đồng xu ít nhất hoặc -1 .

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
3 11 1 2 5 2 6 1	2

Giải thích:

Với số tiền cần rút $S = 10$ và các mệnh giá: 5 đồng (có 1 tờ), 2 đồng (có 3 tờ):

Chọn 1 tờ 5 đồng và 2 tờ 2 đồng ($5 + 2 \times 2 = 9 < 10$).

Phương án đổi đúng là dùng 5 tờ 2 đồng (nhưng chỉ có 3 tờ nên không được).

Nếu có thêm mệnh giá 1 đồng (2 tờ): Dùng 1 tờ 5, 2 tờ 2 và 1 tờ 1, tổng cộng 4 tờ tiền.

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq N \leq 100, 1 \leq S \leq 20000$.

Bài 30 [CPPB-DP2-15]: Tối Ưu Hóa Túi Đồ Hỗn Hợp (Hybrid Knapsack)

Bối cảnh: Một trung tâm cứu hộ thiên tai chuẩn bị các gói hàng viện trợ để thả dù xuống vùng bão lũ. Có N loại hàng hóa thiết yếu, mỗi loại có khối lượng w_i và giá trị cứu trợ v_i . Tính chất nguồn cung của các mặt hàng rất đa dạng: một số mặt hàng chỉ có duy nhất 1 kiện (loại 0/1), một số mặt hàng được tài trợ không giới hạn (loại không giới hạn), và một số mặt hàng chỉ có số lượng cố định k_i kiện (loại giới hạn). Khoang hàng máy bay có sức chứa tối đa là W .

Nhiệm vụ: Cho danh sách N loại hàng với đặc tính số lượng của từng loại và tải trọng khoang bay W . Hãy lập trình tìm tổng giá trị cứu trợ lớn nhất có thể vận chuyển.

Đầu vào (Input):

- Dòng 1: Hai số N và W ($1 \leq N \leq 100, 1 \leq W \leq 5000$).
- N dòng tiếp theo: Mỗi dòng gồm 3 số $type, weight, val$ ($type \in \{1, 2\}, 1 \leq weight \leq W, 1 \leq val \leq 10^6$).

Đầu ra (Output):

- Tổng giá trị lớn nhất.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
3 10 1 4 20 2 3 15 1 5 30	50

Giải thích:

Với khoang máy bay có tải trọng $W = 15$ và danh mục hàng cứu trợ hỗn hợp:

Sự kết hợp tối ưu giữa các mặt hàng giới hạn và hàng không giới hạn đem lại tổng giá trị lớn nhất là 32.

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq N \leq 100, 1 \leq W \leq 5000$.

Bài 15: Quy hoạch động chuỗi: LCS & Edit Distance

1. Không gian trạng thái hai tiền tố (2-prefix state)

Trong xử lý chuỗi ký tự thì đầu, các bài toán so khớp, tìm chuỗi tương đồng hay biến đổi xâu thường thao tác trên hai chuỗi A (độ dài N) và B (độ dài M).

- **Nguyên lý Thiết kế Trạng thái:** Ta định nghĩa trạng thái dựa trên **Cặp tiền tố** của hai chuỗi:

$dp[i][j]$ đại diện cho kết quả tối ưu khi xét tiền tố $A[1 \dots i]$ (gồm i ký tự đầu của A) và tiền tố $B[1 \dots j]$ (gồm j ký tự đầu của B).

- **Trường hợp cơ sở (Base Cases):** Khi một trong hai tiền tố có độ dài bằng 0 ($i = 0$ hoặc $j = 0$), tương đương với chuỗi rỗng ϵ .

2. Dãy con chung dài nhất (longest common subsequence — LCS)

2.1. Bản chất toán học & hệ thức truy hồi

- **Định nghĩa:** Dãy con là dãy thu được bằng cách xóa đi một số ký tự mà **không làm thay đổi thứ tự** của các ký tự còn lại.
- **State Definition:** $dp[i][j]$ là độ dài của dãy con chung dài nhất giữa $A[1 \dots i]$ và $B[1 \dots j]$.
- **Base Cases:** $dp[0][j] = 0$ và $dp[i][0] = 0$ với mọi i, j .
- **State Transition:** So sánh ký tự đuôi $A[i]$ và $B[j]$:

1. Nếu $A[i] == B[j]$: Ký tự này chắc chắn thuộc LCS chung:

$$dp[i][j] = 1 + dp[i-1][j-1]$$

2. Nếu $A[i] != B[j]$: Bỏ qua $A[i]$ hoặc bỏ qua $B[j]$ để lấy phương án tốt hơn:

$$dp[i][j] = \max(dp[i-1][j], dp[i][j-1])$$

- **Độ phức tạp:** Thời gian $\mathcal{O}(N \cdot M)$, Bộ nhớ $\mathcal{O}(N \cdot M)$.

BẢNG PHƯƠNG ÁN LCS & ĐƯỜNG TRUY VẾT XÂU TỐI ƯU

Xâu A = "AGGTAB", Xâu B = "GXTXAYB" $\rightarrow$ LCS = "GTAB" (Độ dài = 4)

i \ j	eps	G	X	T	X	A	Y	B
eps								
G								
X								
T								
X								
A								
Y								
B								
								B == B
								4

QUY TẮC TRUY VẾT (TRACEBACK) TRÊN BẢNG LCS:

- Nếu $A[i] == B[j]$: Ký tự này thuộc LCS $\rightarrow$ Ghi nhận ký tự và đi chéo lùi về $(i-1, j-1)$.
 - Nếu $A[i] != B[j]$: Đi theo hướng có giá trị lớn hơn: lên trên $(i-1, j)$ hoặc sang trái $(i, j-1)$.
- $\rightarrow$ Kết quả sau khi đảo ngược chuỗi thu được: "GTAB"

Bảng phương án LCS và Đường truy vết

2.2. Kỹ thuật khôi phục xâu LCS tối ưu (traceback)

Từ ô kết quả (N, M) trên bảng phương án 2D:

1. Nếu $A[i] == B[j] \Rightarrow$ Thêm $A[i]$ vào xâu kết quả, lùi chéo về $(i-1, j-1)$.

2. Nếu $A[i] \neq B[j] \Rightarrow$ Di về ô có giá trị lớn hơn: lên trên $(i-1, j)$ nếu $dp[i-1][j] \geq dp[i][j-1]$, ngược lại sang trái $(i, j-1)$.
3. Dừng lại khi $i = 0$ hoặc $j = 0$. Đảo ngược xâu kết quả thu được.

3. Khoảng cách biến đổi xâu (Edit Distance / Levenshtein Distance)

3.1. Bản chất 3 phép biến đổi

Cần tìm số phép biến đổi **ít nhất** để biến xâu A thành xâu B . Các phép thao tác hợp lệ gồm:

1. **Chèn (Insert):** Thêm 1 ký tự vào xâu A .
2. **Xóa (Delete):** Xóa 1 ký tự khỏi xâu A .
3. **Thay thế (Replace):** Đổi 1 ký tự của A thành ký tự khác.

3.2. Hệ thức chuyển trạng thái

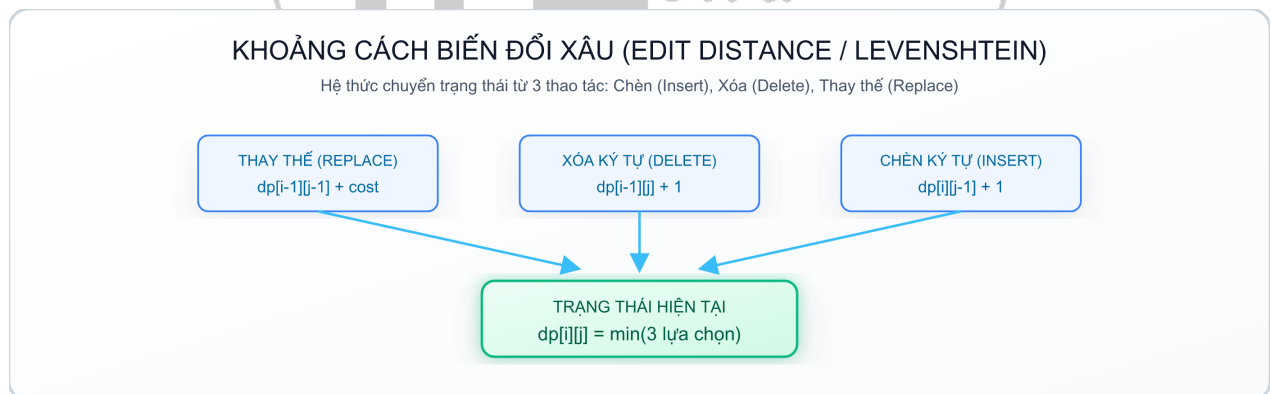
- **State Definition:** $dp[i][j]$ là số thao tác ít nhất biến $A[1 \dots i]$ thành $B[1 \dots j]$.
- **Base Cases:**
 - $dp[i][0] = i$ (Biến xâu độ dài i thành xâu rỗng cần i phép xóa).
 - $dp[0][j] = j$ (Biến xâu rỗng thành xâu độ dài j cần j phép chèn).
- **State Transition:**
 - Nếu $A[i] == B[j] \Rightarrow dp[i][j] = dp[i-1][j-1]$ (Không tốn chi phí).
 - Nếu $A[i] \neq B[j]$:

$$dp[i][j] = 1 + \min(dp[i-1][j-1], dp[i-1][j], dp[i][j-1])$$

Thay thế

Xóa

Chèn



Khoảng cách biến đổi xâu Edit Distance

4. Phân biệt rạch ròi: Xâu con đối xứng (substring) vs dãy con đối xứng (subsequence)

Đây là từ huyệt thuật ngữ cực kỳ quan trọng trong lập trình thi đấu:

PHÂN BIỆT RẠCH RỜI: XÂU CON LIÊN TIẾP VS DÃY CON ĐỐI XỨNG

Hai bài toán hoàn toàn khác nhau về bản chất Trạng Thái (State Identity) và Hệ thức truy hồi

1. XÂU CON LIÊN TIẾP (SUBSTRING)

$dp[i][j]$ = boolean (Đoạn $S[i..j]$ có đối xứng?)

- Yêu cầu các ký tự phải **LIÊN TIẾP**
- $dp[i][j] = \text{true}$ nếu $S[i] == S[j]$ && $dp[i+1][j-1]$
- Duyệt theo độ dài xâu con $len = 1 \rightarrow N$

Ví dụ: "babad" $\rightarrow$ "bab" hoặc "aba" ($len = 3$)

2. DÃY CON KHÔNG LIÊN TIẾP (SUBSEQUENCE)

$dp[i][j]$ = int (Độ dài dãy con đối xứng max)

- Ký tự **KHÔNG CẦN** liên tiếp
- $S[i] == S[j] \rightarrow 2 + dp[i+1][j-1]$
- $S[i] \neq S[j] \rightarrow \max(dp[i+1][j], dp[i][j-1])$

Ví dụ: "bbbab" $\rightarrow$ "bbbb" ($len = 4$)

Phân biệt Xâu con liên tiếp vs Dãy con đối xứng

4.1. Xâu con liên tiếp đối xứng dài nhất (longest palindromic substring)

- **Đặc tính:** Các ký tự phải **liên tiếp liên kề** nhau.
- **State Definition:** $dp[i][j]$ kiểu boolean, nhận giá trị true nếu đoạn con liên tiếp $S[i \dots j]$ là một xâu đối xứng.
- **Transition:** $dp[i][j] = (S[i] == S[j]) \wedge dp[i+1][j-1]$.
- **Duyệt:** Theo độ dài xâu con $len = 1 \rightarrow N$.

4.2. Dãy con không liên tiếp đối xứng dài nhất (longest palindromic subsequence)

- **Đặc tính:** Các ký tự **không cần liên tiếp**.
- **State Definition:** $dp[i][j]$ là độ dài lớn nhất của dãy con đối xứng trích xuất từ đoạn $S[i \dots j]$.
- **Transition:**
 - Nếu $S[i] == S[j] \Rightarrow dp[i][j] = 2 + dp[i+1][j-1]$.
 - Nếu $S[i] \neq S[j] \Rightarrow dp[i][j] = \max(dp[i+1][j], dp[i][j-1])$.
- **Cách giải tương đương:** Tính LCS giữa xâu S và xâu đảo ngược S^R !

5. Các bẫy lỗi lập trình kinh điển (bug traps)

1. **Bẫy lệch chỉ số 0-based của Xâu ký tự với 1-based của Bảng DP:**
 - Trong C++, string có chỉ số từ 0 đến $N - 1$. Khi truy cập ký tự thứ i trong bảng DP 1-based, phải viết $A[i - 1]$ thay vì $A[i]$.
2. **Bẫy thứ tự duyệt trong Quy hoạch động trên Đoạn con Palindrome:**
 - Trạng thái $dp[i][j]$ phụ thuộc vào $dp[i+1][j-1]$ (đoạn ngắn hơn). Nếu duyệt i từ $1 \rightarrow N$ xuôi thì ô $dp[i+1][j-1]$ chưa được tính $\Rightarrow$ Kết quả sai!
 - **Quy tắc đúng:** Luôn duyệt theo độ dài $len = 1 \rightarrow N$, sau đó duyệt điểm đầu $i = 1 \rightarrow N - len + 1$ và $j = i + len - 1$.
3. **Bẫy khởi tạo Base Case của Edit Distance:**
 - Quên khởi tạo cột 0 ($dp[i][0] = i$) và hàng 0 ($dp[0][j] = j$) sẽ dẫn đến toàn bộ bảng nhận giá trị rác.

6. Mẫu cài đặt chuẩn thi đấu (competitive templates)

Mẫu 1: Dãy con chung dài nhất (LCS) kèm khôi phục chuỗi

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    string a, b;
    if (!(cin >> a >> b)) return 0;

    int n = a.size();
    int m = b.size();
    if (n == 0 || m == 0) {
        cout << 0 << "\n\n";
        return 0;
    }

    vector<vector<int>> dp(n + 1, vector<int>(m + 1, 0));

    for (int i = 1; i <= n; ++i) {
        for (int j = 1; j <= m; ++j) {
            if (a[i - 1] == b[j - 1]) {
                dp[i][j] = 1 + dp[i - 1][j - 1];
            } else {
                dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]);
            }
        }
    }

    cout << dp[n][m] << "\n";

    // Khôi phục chuỗi LCS
    string lcs_str = "";
    int i = n, j = m;
    while (i > 0 && j > 0) {
        if (a[i - 1] == b[j - 1]) {
            lcs_str.push_back(a[i - 1]);
            --i;
            --j;
        } else if (dp[i - 1][j] >= dp[i][j - 1]) {
            --i;
        } else {
            --j;
        }
    }
    reverse(lcs_str.begin(), lcs_str.end());
    cout << lcs_str << "\n";

    return 0;
}
```

Mẫu 2: Khoảng cách biến đổi chuỗi (Edit Distance)

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
```

```

cin.tie(nullptr);

string a, b;
if (!(cin >> a >> b)) return 0;

int n = a.size();
int m = b.size();

vector<vector<int>> dp(n + 1, vector<int>(m + 1, 0));

for (int i = 0; i <= n; ++i) dp[i][0] = i;
for (int j = 0; j <= m; ++j) dp[0][j] = j;

for (int i = 1; i <= n; ++i) {
    for (int j = 1; j <= m; ++j) {
        if (a[i - 1] == b[j - 1]) {
            dp[i][j] = dp[i - 1][j - 1];
        } else {
            dp[i][j] = 1 + min({dp[i - 1][j - 1], // Replace
                                dp[i - 1][j],      // Delete
                                dp[i][j - 1]});      // Insert
        }
    }
}

cout << dp[n][m] << "\n";
return 0;
}

```

Bài tập thực hành

Bài 31 [CPPB-DPS-01]: Xâu Con Chung Dài Nhất Cơ Bản (LCS)

Bối cảnh: Trong lĩnh vực xử lý ngôn ngữ tự nhiên và kiểm tra đạo văn số học, hai văn bản số được chuyển hóa thành hai chuỗi ký tự S và T . Một chuỗi con chung là một chuỗi các ký tự cùng xuất hiện trong cả hai văn bản theo đúng thứ tự tương đối ban đầu nhưng không nhất thiết phải đứng liền kề nhau. Các nhà nghiên cứu cần xác định độ dài chuỗi con chung dài nhất giữa hai văn bản để đo lường mức độ tương đồng nội dung.

Nhiệm vụ: Cho hai chuỗi ký tự S và T . Hãy lập trình tìm độ dài của xâu con chung dài nhất của hai chuỗi đó.

Đầu vào (Input):

- Dòng 1: Chuỗi ký tự S ($1 \leq |S| \leq 2000$).
- Dòng 2: Chuỗi ký tự T ($1 \leq |T| \leq 2000$).

Đầu ra (Output):

- Độ dài LCS.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
AGGTAB GXTXAYB	4

Giải thích:

Với hai chuỗi $S = \text{"ABCB DAB"}$ và $T = \text{"BDCAB"}$:

Chuỗi con chung dài nhất có thể tìm được là "BCAB" (hoặc "BDAB") có độ dài đúng bằng 4.

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq |S|, |T| \leq 2000$.

Bài 32 [CPPB-DPS-02]: Truy Vết Chuỗi Con Chung Dài Nhất

Bối cảnh: Sau khi tính toán được độ dài tương đồng giữa hai chuỗi văn bản, hệ thống kiểm tra đạo văn cần trích xuất chính xác chuỗi ký tự chung dài nhất đó để làm bằng chứng đánh dấu nổi bật (highlight) trên giao diện phần mềm cho người thẩm định theo dõi.

Nhiệm vụ: Cho hai chuỗi ký tự S và T . Hãy lập trình tìm và in ra chuỗi con chung dài nhất của hai chuỗi. Nếu có nhiều chuỗi con chung cùng đạt độ dài lớn nhất, in ra một chuỗi bất kỳ thỏa mãn.

Đầu vào (Input):

- Dòng 1: Chuỗi S ($1 \leq |S| \leq 2000$).
- Dòng 2: Chuỗi T ($1 \leq |T| \leq 2000$).

Đầu ra (Output):

- Dòng 1: Độ dài LCS.
- Dòng 2: Chuỗi ký tự LCS.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
ABCB DAB	4
BDCAB	BDAB

Giải thích:

Với hai chuỗi $S = \text{"ABCB DAB"}$ và $T = \text{"BDCAB"}$:

Chuỗi con chung dài nhất đồng thời xuất hiện trong cả hai chuỗi là "BCAB" với độ dài là 4. Kết quả in ra chuỗi BCAB.

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq |S|, |T| \leq 2000$.

Bài 33 [CPPB-DPS-03]: Khoảng Cách Chỉnh Sửa (Edit Distance / Levenshtein)

Bối cảnh: Trong tính năng gợi ý tự động sửa lỗi chính tả của bàn phím thông minh, khi người dùng gõ nhầm một từ, máy tính cần tìm khoảng cách biến đổi tối thiểu từ chuỗi người dùng gõ thành chuỗi từ điển chuẩn. Có 3 thao tác chỉnh sửa ký tự cơ bản được hỗ trợ: chèn thêm 1 ký tự, xóa bớt 1 ký tự, hoặc thay thế 1 ký tự bằng 1 ký tự khác. Mỗi thao tác đều có chi phí là 1 đơn vị.

Nhiệm vụ: Cho hai chuỗi ký tự S và T . Hãy lập trình tìm số lượng thao tác chỉnh sửa ít nhất để biến đổi chuỗi S thành chuỗi T .

Đầu vào (Input):

- Dòng 1: Chuỗi S ($1 \leq |S| \leq 2000$).
- Dòng 2: Chuỗi T ($1 \leq |T| \leq 2000$).

Đầu ra (Output):

- Số phép biến đổi ít nhất.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
horse ros	3

Giải thích:

Để biến đổi chuỗi $S = \text{"kitten"}$ thành $T = \text{"sitting"}$:

1. Thay thế ký tự 'k' thành 's' \to "sitten".
2. Thay thế ký tự 'e' thành 'i' \to "sittin".
3. Chèn thêm ký tự 'g' vào cuối \to "sitting".

Tổng cộng cần đúng 3 thao tác chỉnh sửa, kết quả là 3.

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq |S|, |T| \leq 2000$.

Bài 34 [CPPB-DPS-04]: Chỉ Dùng Phép Xóa Biến Đổi Hai Xâu

Bối cảnh: Tại một máy nén dữ liệu truyền thông, hệ thống chỉ hỗ trợ duy nhất một thao tác cơ học là xóa bỏ ký tự. Cho hai chuỗi văn bản S và T . Kỹ sư cần đưa cả hai chuỗi về cùng một chuỗi hoàn toàn giống nhau bằng cách chỉ xóa bỏ các ký tự trên mỗi chuỗi.

Nhiệm vụ: Cho hai chuỗi ký tự S và T . Hãy lập trình tìm tổng số lượng ký tự ít nhất cần phải xóa bỏ trên cả hai chuỗi để phần còn lại của chúng trở nên đồng nhất.

Đầu vào (Input):

- Dòng 1: Chuỗi S ($1 \leq |S| \leq 2000$).
- Dòng 2: Chuỗi T ($1 \leq |T| \leq 2000$).

Đầu ra (Output):

- Tổng số ký tự xóa ít nhất.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
sea eat	2

Giải thích:

Với hai chuỗi $S = \text{"sea"}$ và $T = \text{"eat"}$:

Để hai chuỗi trở nên giống nhau, ta đưa cả hai về chuỗi chung là "ea" :

- Trên chuỗi S : Xóa ký tự 's' (1 ký tự).

- Trên chuỗi T : Xóa ký tự 't' (1 ký tự).

Tổng số ký tự cần xóa là $1 + 1 = 2$.

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq |S|, |T| \leq 2000$.

Bài 35 [CPPB-DPS-05]: Xâu Con Đối Xứng Dài Nhất (Longest Palindromic Subsequence)

Bối cảnh: Một chuỗi ký tự được gọi là đối xứng (Palindrome) nếu đọc từ trái sang phải hay từ phải sang trái đều thu được chuỗi hoàn toàn giống nhau (ví dụ: 'RADAR', 'MADAM'). Trong phân tích mã vạch bảo mật, bạn kỹ thuật muốn trích xuất từ chuỗi gốc một chuỗi con (không nhất thiết liên tiếp) có tính đối xứng sao cho độ dài của nó là lớn nhất.

Nhiệm vụ: Cho chuỗi ký tự S . Hãy lập trình tìm độ dài của chuỗi con đối xứng dài nhất trích xuất được từ S .

Đầu vào (Input):

- Một dòng chứa chuỗi S ($1 \leq |S| \leq 2000$).

Đầu ra (Output):

- Độ dài lớn nhất của xâu con đối xứng.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
bbbab	4

Giải thích:

Với chuỗi $S = \text{"BBABCBCAB"}$:

Một chuỗi con đối xứng dài nhất có thể chọn là "BABCBAB" (hoặc "BACBCAB") có độ dài bằng 7.

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq |S| \leq 2000$.

Bài 36 [CPPB-DPS-06]: Đoạn Con Đối Xứng Liên Tiếp Dài Nhất

Bối cảnh: Khác với chuỗi con rời rạc, một đoạn con liên tiếp (Substring) đối xứng đòi hỏi các ký tự phải đứng kề sát nhau và tạo thành một cụm đối xứng hoàn chỉnh. Đây là tác vụ trọng tâm trong việc nhận diện các đoạn lặp đảo ngược trong cấu trúc chuỗi gen sinh học.

Nhiệm vụ: Cho chuỗi ký tự S . Hãy lập trình tìm độ dài của đoạn con liên tiếp đối xứng dài nhất trong S .

Đầu vào (Input):

- Một dòng chứa chuỗi S ($1 \leq |S| \leq 2000$).

Đầu ra (Output):

- Dòng 1: Độ dài.
- Dòng 2: Nội dung đoạn con đối xứng.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
babad	3 bab

Giải thích:

Với chuỗi $S = \text{"babad"}$:

Đoạn con liên tiếp đối xứng dài nhất là "bab" (hoặc "aba") có độ dài bằng 3.

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq |S| \leq 2000$.

Bài 37 [CPPB-DPS-07]: Đếm Số Đoạn Con Đối Xứng Liên Tiếp

Bối cảnh: Trong một kỳ thi mật mã học sinh viên, ban giám khảo đưa ra một chuỗi văn bản và yêu cầu các thí sinh thống kê tất cả các phân đoạn liên tiếp có tính đối xứng xuất hiện trong chuỗi (kể cả các đoạn con có độ dài 1 ký tự).

Nhiệm vụ: Cho chuỗi ký tự S . Hãy lập trình đếm tổng số lượng đoạn con liên tiếp đối xứng có trong chuỗi.

Đầu vào (Input):

- Một dòng chứa chuỗi S ($1 \leq |S| \leq 2000$).

Đầu ra (Output):

- Số đoạn con đối xứng.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
aaa	6

Giải thích:

Với chuỗi $S = \text{"aaa"}$:

Có tất cả 6 đoạn con liên tiếp đối xứng gồm:

- 3 đoạn độ dài 1: "a" (vị trí 0), "a" (vị trí 1), "a" (vị trí 2).
- 2 đoạn độ dài 2: "aa" (vị trí 0-1), "aa" (vị trí 1-2).
- 1 đoạn độ dài 3: "aaa" (toàn bộ chuỗi).

Tổng số đoạn đối xứng là 6.

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq |S| \leq 2000$.

Bài 38 [CPPB-DPS-08]: Chèn Ít Ký Tự Nhất Tạo Chuỗi Đối Xứng

Bối cảnh: Một máy in nhãn hàng hóa bị lỗi chỉ in được một chuỗi ký tự khuyết thiếu. Để nhãn hàng có tính thẩm mỹ cân đối, kỹ thuật viên cần chèn thêm vào các vị trí bất kỳ trong chuỗi một số lượng ký tự ít nhất sao cho chuỗi thu được trở thành một chuỗi đối xứng hoàn hảo.

Nhiệm vụ: Cho chuỗi ký tự S . Hãy lập trình tìm số lượng ký tự ít nhất cần phải chèn thêm vào chuỗi S để biến nó thành một chuỗi đối xứng.

Đầu vào (Input):

- Một dòng chứa chuỗi S ($1 \leq |S| \leq 2000$).

Đầu ra (Output):

- Số ký tự chèn ít nhất.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
zzazz	0

Giải thích:

Với chuỗi $S = \text{"mbadm"}$:

Để chuỗi trở thành đối xứng, ta có thể chèn thêm 2 ký tự 'd' và 'b' vào các vị trí thích hợp để tạo thành chuỗi "mbdadbm". Số ký tự chèn thêm ít nhất là 2.

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq |S| \leq 2000$.

Bài 39 [CPPB-DPS-09]: Xâu Mẹ Chung Ngắn Nhất (Shortest Common Supersequence)

Bối cảnh: Một máy phát tín hiệu cần gửi một chuỗi mã nguồn sao cho máy thu khi nhận được có thể trích xuất ra đồng thời cả hai bản tin S và T dưới dạng các chuỗi con. Để tiết kiệm băng thông truyền dẫn số, độ dài của chuỗi mã phát đi phải là ngắn nhất có thể.

Nhiệm vụ: Cho hai chuỗi ký tự S và T . Hãy lập trình tìm độ dài ngắn nhất của một chuỗi chứa cả S và T dưới dạng các chuỗi con.

Đầu vào (Input):

- Dòng 1: Chuỗi S ($1 \leq |S| \leq 2000$).
- Dòng 2: Chuỗi T ($1 \leq |T| \leq 2000$).

Đầu ra (Output):

- Độ dài ngắn nhất của Supersequence.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
abac cab	5

Giải thích:

Với hai chuỗi $S = \text{"abac"}$ và $T = \text{"cab"}$:

Xâu mẹ chung ngắn nhất chứa cả hai chuỗi là "cabac" có độ dài đúng bằng 5 (chứa "cab" ở tiền tố và "abac" ở hậu tố).

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq |S|, |T| \leq 2000$.

Bài 40 [CPPB-DPS-10]: Đếm Số Lần Xuất Hiện Xâu Con Rời Rạc (Distinct Subsequences)

Bối cảnh: Trong bộ lọc từ khóa độc hại của mạng xã hội, hệ thống cần kiểm tra tần suất mà một từ khóa nhạy cảm T xuất hiện dưới dạng chuỗi con rời rạc bên trong một bài viết S . Hai cách xuất hiện được tính là khác nhau nếu tập hợp các vị trí chỉ số ký tự được chọn trong bài viết S là khác nhau.

Nhiệm vụ: Cho hai chuỗi ký tự S và T . Hãy lập trình đếm số cách khác nhau để chọn ra chuỗi con trong S bằng đúng chuỗi T , lấy dư cho $10^9 + 7$.

Đầu vào (Input):

- Dòng 1: Chuỗi S ($1 \leq |S| \leq 2000$).
- Dòng 2: Chuỗi T ($1 \leq |T| \leq 500$).

Đầu ra (Output):

- Số cách chọn theo modulo $10^9 + 7$.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
rabbbit rabbit	3

Giải thích:

Với $S = \text{"rabbbit"}$ và $T = \text{"rabbit"}$:

Trong chuỗi S có 3 ký tự 'b' liên tiếp. Để tạo thành từ "rabbit", ta có thể loại bỏ 1 trong 3 ký tự 'b' đó. Do đó có đúng 3 cách chọn khác nhau để thu được từ "rabbit". Kết quả in ra là 3.

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq |S| \leq 2000, 1 \leq |T| \leq 500$.

Bài 41 [CPPB-DPS-11]: Khớp Chuỗi Ký Tự Đại Diện (Wildcard Matching)

Bối cảnh: Một công cụ tìm kiếm tệp tin hỗ trợ tìm kiếm theo mẫu (pattern) chứa các ký tự đại diện thông dụng: Ký tự `?` có thể khớp với đúng một ký tự bất kỳ, và ký tự `\*` có thể khớp với bất kỳ chuỗi ký tự nào (kể cả chuỗi rỗng). Quản trị viên cần kiểm tra xem tên tệp văn bản S có khớp hoàn toàn với mẫu P hay không.

Nhiệm vụ: Cho chuỗi văn bản S và chuỗi mẫu P. Hãy lập trình kiểm tra xem chuỗi S có khớp toàn bộ với mẫu P hay không. Nếu khớp in ra `YES`, ngược lại in ra `NO`.

Đầu vào (Input):

- Dòng 1: Chuỗi S ($1 \leq |S| \leq 1000$).
- Dòng 2: Mẫu P ($1 \leq |P| \leq 1000$).

Đầu ra (Output):

- YES hoặc NO.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
adceb ab	YES

Giải thích:

Với văn bản S = "adceb" và mẫu P = "*a*b":

Ký tự '*' đầu tiên khớp với chuỗi rỗng, sau đó ký tự 'a' khớp 'a', ký tự '*' thứ hai khớp với đoạn "dce", và cuối cùng ký tự 'b' khớp 'b'. Chuỗi khớp hoàn toàn với mẫu, kết quả in ra là YES.

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq |S|, |P| \leq 1000$.

Bài 42 [CPPB-DPS-12]: Xâu Con Chung Của Ba Chuỗi (LCS 3 Strings)

Bối cảnh: Ba viện nghiên cứu sinh học độc lập cùng giải mã trình tự một đoạn gen kháng bệnh ở ba giống cây trồng khác nhau, thu được 3 chuỗi ADN ký hiệu là S₁, S₂ và S₃. Để xác định đoạn gen chung cốt lõi quy định tính trạng kháng bệnh, các nhà khoa học cần tìm độ dài của chuỗi con chung dài nhất xuất hiện đồng thời trong cả ba chuỗi ADN.

Nhiệm vụ: Cho ba chuỗi ký tự S₁, S₂ và S₃. Hãy lập trình tính độ dài chuỗi con chung dài nhất của cả ba chuỗi.

Đầu vào (Input):

- 3 dòng, mỗi dòng chứa một chuỗi ký tự ($1 \leq |S_i| \leq 100$).

Đầu ra (Output):

- Độ dài xâu con chung dài nhất.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
geeks geek geekfor	4

Giải thích:

Với 3 chuỗi ký tự:

$S_1 = \text{"geeks"}, S_2 = \text{"geeksfor"}, S_3 = \text{"geeksforgeeks"}:$

Chuỗi con chung dài nhất xuất hiện trong cả 3 chuỗi là "geeks" với độ dài bằng 5.

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq |S_i| \leq 100$.

Bài 43 [CPPB-DPS-13]: Xóa Ít Ký Tự Nhất Để Chuỗi Đối Xứng

Bối cảnh: Một thiết bị viễn thông thu nhận một chuỗi tín hiệu bị nhiễu chứa một số ký tự rác. Để khôi phục lại tính toàn vẹn của tín hiệu đối xứng nguyên bản, kỹ sư cần loại bỏ bớt một số lượng ký tự ít nhất từ chuỗi thu được sao cho chuỗi còn lại trở thành một chuỗi đối xứng.

Nhiệm vụ: Cho chuỗi ký tự S . Hãy lập trình tìm số lượng ký tự ít nhất cần xóa bỏ để chuỗi còn lại là một chuỗi đối xứng.

Đầu vào (Input):

- Một dòng chứa chuỗi S ($1 \leq |S| \leq 2000$).

Đầu ra (Output):

- Số ký tự xóa ít nhất.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
aebcbda	2

Giải thích:

Với chuỗi $S = \text{"aebcbda"}:$

Chuỗi con đối xứng dài nhất có thể giữ lại là "abcba" (độ dài 5). Để thu được chuỗi này, ta chỉ cần xóa đi đúng 2 ký tự là 'e' và 'd'. Số ký tự cần xóa ít nhất là 2.

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq |S| \leq 2000$.

Bài 44 [CPPB-DPS-14]: Độ Dài LCS Giữa Hai Chuỗi Gen

Bối cảnh: Trong đề án nghiên cứu tiến hóa học phân tử, các nhà di truyền học so sánh hai chuỗi gen dài được biểu diễn dưới dạng hai chuỗi ký tự chỉ gồm 4 loại nucleotide cơ bản: 'A', 'C', 'G',

`T`. Việc tìm độ dài xâu con chung dài nhất giữa hai chuỗi gen phản ánh mức độ gần gũi về mặt quan hệ tiến hóa giữa hai loài sinh vật.

Nhiệm vụ: Cho hai chuỗi gen S và T. Hãy lập trình tính độ dài chuỗi con chung dài nhất giữa hai chuỗi đó.

Đầu vào (Input):

- Dòng 1: Chuỗi S ($1 \leq |S| \leq 500$).
- Dòng 2: Chuỗi T ($1 \leq |T| \leq 500$).

Đầu ra (Output):

- Độ dài LCS.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
ACCGGTCGAGTGC GCGGAAGCCGGCCGAA GTCGTTCCGAATGCCGTTGCTCTGTAAA	20

Giải thích:

Với hai chuỗi gen S = "AGGTAB" và T = "GXTXAYB":

Xâu con chung dài nhất giữa hai mẫu gen là "GTAB" có độ dài bằng 4.

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq |S|, |T| \leq 500$.

Bài 45 [CPPB-DPS-15]: Tách Từ Trong Chuỗi (Word Break)

Bối cảnh: Một hệ thống nhận dạng quang học chữ viết (OCR) quét văn bản tiếng Anh cổ nhưng do lỗi khoảng cách, tất cả các từ trong câu bị dính liền thành một chuỗi ký tự duy nhất S. Cho trước một cuốn từ điển gồm N từ vựng chuẩn. Kỹ sư ngôn ngữ cần xác định xem chuỗi ký tự dính liền S có thể được phân tách thành một chuỗi các từ hợp lệ nằm trong cuốn từ điển hay không.

Nhiệm vụ: Cho chuỗi ký tự S và từ điển gồm N từ. Hãy lập trình kiểm tra xem chuỗi S có thể được phân tách hoàn toàn thành các từ trong từ điển hay không. Nếu có in ra `YES`, ngược lại in ra `NO`.

Đầu vào (Input):

- Dòng 1: Chuỗi S ($1 \leq |S| \leq 300$).
- Dòng 2: Số nguyên K ($1 \leq K \leq 100$).
- K dòng tiếp theo chứa các từ trong từ điển.

Đầu ra (Output):

- YES hoặc NO.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
leetcode 2 leet code	YES

Giải thích:

Với chuỗi $S = \text{"leetcode"}$ và từ điển gồm các từ $\{\text{"leet"}, \text{"code"}\}$:

Chuỗi S có thể phân tách thành hai từ hợp lệ là "leet" và "code" . Cả hai từ đều có mặt trong từ điển, do đó kết quả in ra là YES.

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq |S| \leq 300, 1 \leq K \leq 100$.



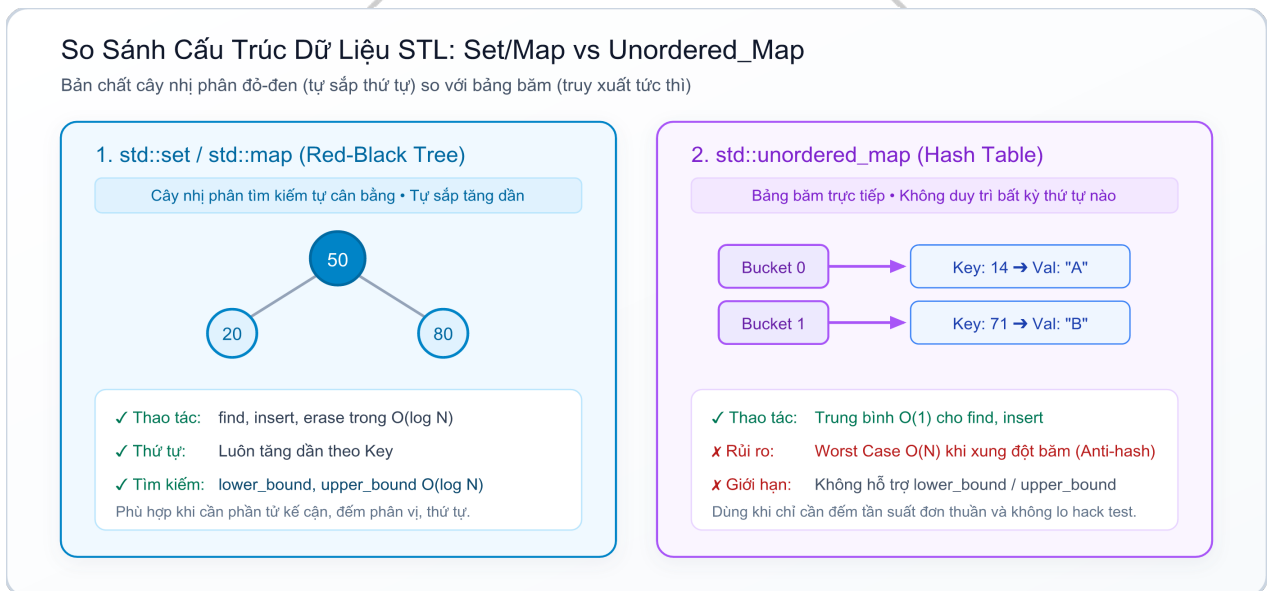
CHƯƠNG 06: CẤU TRÚC DỮ LIỆU NÂNG CAO

Bài 16: Cấu trúc dữ liệu STL: Set, map & heap

1. Bản chất các cấu trúc dữ liệu nâng cao trong thư viện chuẩn STL

Trong lập trình thi đấu hiện đại, việc tự cài đặt lại cây nhị phân cân bằng hay bảng băm từ đầu cho mọi bài toán là không khả thi. C++ Standard Template Library (STL) cung cấp các cấu trúc dữ liệu tối ưu hóa cực mạnh:

- **std::set / std::map:** Cây đỏ-đen (Red-Black Tree) tự cân bằng, luôn duy trì các phần tử theo thứ tự tăng dần. Các thao tác tìm kiếm, chèn, xóa đều có độ phức tạp đảm bảo $\mathcal{O}(\log N)$.
- **std::unordered_map / std::unordered_set:** Bảng băm trực tiếp (Hash Table), đạt thời gian trung bình $\mathcal{O}(1)$ cho các truy vấn.
- **std::priority_queue:** Cấu trúc Heap nhị phân hoàn chỉnh, cho phép truy xuất phần tử lớn nhất (hoặc nhỏ nhất) trong $\mathcal{O}(1)$ và thêm/bớt trong $\mathcal{O}(\log N)$.



So sánh Set Map vs Unordered Map

2. Kỹ thuật nén tọa độ (Coordinate Compression)

2.1. Bản chất bài toán & khi nào cần nén tọa độ?

- **Vấn đề:** Các giá trị trong mảng A có thể rất lớn ($A[i] \leq 10^9$ hoặc 10^{18}), ta không thể dùng giá trị này làm chỉ số mảng đếm tần suất hoặc dựng cây Segment Tree / Fenwick Tree. Tuy nhiên, số lượng phần tử N lại rất nhỏ ($N \leq 10^5$).
- **Nguyên lý Nén Tọa Độ:** Ánh xạ tập giá trị rời rạc ban đầu về tập số nguyên liên tiếp $\{0, 1, 2, \dots, K-1\}$ ($K \leq N$) sao cho **giữ nguyên thứ tự tương quan lớn bé** giữa các phần tử:

$$A[i] < A[j] \Leftrightarrow \text{rank}(A[i]) < \text{rank}(A[j])$$

KỸ THUẬT NÉN TỌA ĐỘ (COORDINATE COMPRESSION): $[10^9] \rightarrow [0..N-1]$

Mảng Gốc A (Giá trị cực lớn $\leq 10^9$):

1000000000	50000	1000000000	7	50000
------------	-------	------------	---	-------

Quy trình: Copy $\rightarrow$ sort() $\rightarrow$ unique() $\rightarrow$ erase() $\rightarrow$ lower_bound() ánh xạ

Mảng Nén Sau Ánh Xạ (Chỉ số 0..K-1):

2	1	2	0	1
---	---	---	---	---

Bảng tra cứu: vals = [7, 50000, 1000000000] (Kích thước mảng nén K = 3)

Mô hình Nén Tọa Độ

2.2. Quy trình 4 bước chuẩn mực trong C++

1. Sao chép mảng: `vector<long long> vals = a;`
2. Sắp xếp tăng dần: `sort(vals.begin(), vals.end());`
3. Loại bỏ trùng lặp: `vals.erase(unique(vals.begin(), vals.end()), vals.end());`
4. Ánh xạ bằng Tìm kiếm nhị phân:

```
int compressed_val = lower_bound(vals.begin(), vals.end(), a[i]) - vals.begin();
```

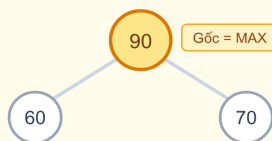
3. Hàng đợi ưu tiên (Priority Queue / heap)

Hàng Đợi Ưu Tiên `std::priority_queue` (Binary Heap)

Cấu trúc dữ liệu cây nhị phân hoàn chỉnh tự duy trì phần tử cực trị ở đỉnh trong $O(\log N)$

1. Max-Heap (Mặc Định)

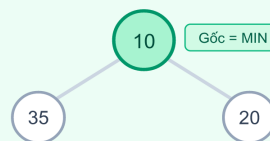
`priority_queue<int> pq;`



- `pq.top()  $\rightarrow$  90` (Lấy MAX trong $O(1)$)
- `pq.push(x), pq.pop()` mất thời gian $O(\log N)$

2. Min-Heap (Đảo Thứ Tự So Sánh)

`priority_queue<int, vector<int>, greater<int>> pq;`



- `pq.top()  $\rightarrow$  10` (Lấy MIN trong $O(1)$)
- `pq.push(x), pq.pop()` mất thời gian $O(\log N)$

Hàng đợi ưu tiên Max-Heap vs Min-Heap

- **Max-Heap (Mặc định):** `priority_queue<long long> max_pq;` $\Rightarrow$ `top()` trả về giá trị lớn nhất.
- **Min-Heap (Đảo thứ tự):** `priority_queue<long long, vector<long long>, greater<long long>> min_pq;` $\Rightarrow$ `top()` trả về giá trị nhỏ nhất.
- **Ứng dụng kinh điển:** Tìm K phần tử lớn nhất/nhỏ nhất trong luồng dữ liệu online, thuật toán Dijkstra, thuật toán Prim, duy trì Trung vị động (Median) bằng 2 Heap.

4. Các bẫy lỗi lập trình kinh điển (bug traps)

1. Bẫy Worst-case $\mathcal{O}(N)$ của unordered_map do Anti-Hash Test:

- Trong các kỳ thi competitive, hàm băm mặc định std::hash của unordered_map rất dễ bị các test đối kháng (Anti-hash tests) làm tràn bucket $\Rightarrow$ Độ phức tạp tụt xuống $\mathcal{O}(N^2)$ và bị TLE.
- **Quy tắc an toàn:** Dùng std::map khi $N \leq 2 \cdot 10^5$ (đảm bảo $\mathcal{O}(N \log N)$), hoặc dùng Custom Hash an toàn với hằng số thời gian ngẫu nhiên chrono.

2. Bẫy xóa phần tử trong std::multiset:

- Lệnh ms.erase(x) sẽ xóa TOÀN BỘ các phần tử có giá trị bằng x trong multiset!
- **Cú pháp chuẩn khi chỉ muốn xóa 1 bản sao:** ms.erase(ms.find(x));.

3. Bẫy truy cập map[key] tự động chèn phần tử mới:

- Khi gọi if (mp[x] > 0), nếu x chưa tồn tại trong map, C++ sẽ tự động chèn cặp (x, 0) vào map làm tăng kích thước bộ nhớ.
- **Cú pháp an toàn:** Dùng if (mp.count(x)) hoặc if (mp.find(x) != mp.end()).

5. Mẫu cài đặt chuẩn thi đấu (competitive templates)

Mẫu 1: Kỹ thuật nén tọa độ chuẩn mực

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;

    if (n <= 0) return 0;

    vector<long long> a(n);

    for (int i = 0; i < n; ++i) {
        cin >> a[i];
    }

    // 1. Tạo mảng nén
    vector<long long> vals = a;

    sort(vals.begin(), vals.end());
    vals.erase(unique(vals.begin(), vals.end()), vals.end());

    // 2. Ánh xạ từng phần tử
    vector<int> compressed(n);

    for (int i = 0; i < n; ++i) {
        compressed[i] = lower_bound(vals.begin(), vals.end(), a[i]) - vals.begin();
    }

    for (int i = 0; i < n; ++i) {
        cout << compressed[i] << (i + 1 == n ? "" : " ");
    }
    cout << "\n";
}
```



```

    return 0;
}

```

Mẫu 2: Duy trì trung vị động bằng 2 heap (median of stream)

```

#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;

    if (n <= 0) return 0;

    priority_queue<long long> left_max; // Nửa nhỏ hơn (Max-Heap)
    priority_queue<long long, vector<long long>, greater<long long>> right_min; // Nửa Lớn hơn (Min-Heap)

    for (int i = 0; i < n; ++i) {
        long long x;
        cin >> x;

        if (left_max.empty() || x <= left_max.top()) {
            left_max.push(x);
        } else {
            right_min.push(x);
        }

        // Tự cân bằng kích thước: left_max luôn có size == right_min hoặc size == right_min + 1
        if (left_max.size() > right_min.size() + 1) {
            right_min.push(left_max.top());
            left_max.pop();
        } else if (right_min.size() > left_max.size()) {
            left_max.push(right_min.top());
            right_min.pop();
        }

        // In trung vị hiện tại
        cout << left_max.top() << (i + 1 == n ? "" : " ");
    }
    cout << "\n";

    return 0;
}

```

Bài tập thực hành

Bài 46 [CPPB-STL-01]: Đếm Số Phần Tử Phân Biệt

Bối cảnh: Tại một hội nghị thượng đỉnh về chuyển đổi số, ban lễ tân quét mã QR định danh của N lượt đại biểu tham dự. Do một số đại biểu di chuyển nhiều lần qua cổng kiểm soát, danh sách các mã định danh thu thập được bị trùng lặp. Ban tổ chức cần thống kê chính xác số lượng đại biểu thực tế (tức số lượng mã định danh độc nhất, phân biệt) đã có mặt.

Nhiệm vụ: Cho danh sách gồm N số nguyên đại diện cho mã định danh. Hãy lập trình đếm và in ra số lượng giá trị phân biệt xuất hiện trong dãy số.

Đầu vào (Input):

- Dòng 1: Số nguyên N ($1 \leq N \leq 10^5$).
- Dòng 2: N số nguyên $A_1, A_2, \dots, A_N$ ($1 \leq A_i \leq 10^9$).

Đầu ra (Output):

- Số phần tử phân biệt.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
5 2 3 2 2 3	2

Giải thích:

Với dãy gồm 5 số nguyên [2, 3, 2, 1, 3]:

Các giá trị phân biệt xuất hiện trong dãy là {1, 2, 3}. Có tất cả 3 giá trị phân biệt, kết quả in ra là 3.

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq N \leq 10^5, 1 \leq A_i \leq 10^9$.

Bài 47 [CPPB-STL-02]: Bảng Tra Cứu Tần Suất Từ Khóa

Bối cảnh: Một công cụ tìm kiếm dữ liệu lớn ghi nhận N từ khóa được người dùng nhập vào thanh tìm kiếm trong ngày. Để phục vụ việc tối ưu hóa máy chủ và đề xuất xu hướng tìm kiếm hàng đầu, kỹ sư hệ thống cần thiết lập một bảng thống kê tần suất xuất hiện của từng từ khóa theo thứ tự bảng chữ cái (thứ tự từ điển tăng dần).

Nhiệm vụ: Cho danh sách gồm N từ khóa. Hãy lập trình đếm tần suất xuất hiện của mỗi từ khóa và in ra kết quả theo thứ tự từ điển tăng dần của các từ khóa.

Đầu vào (Input):

- Dòng 1: Số nguyên N ($1 \leq N \leq 50000$).
- N dòng tiếp theo: Mỗi dòng chứa 1 từ khóa.

Đầu ra (Output):

- Bảng tần suất theo thứ tự từ điển.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
4 apple banana apple orange	apple 2 banana 1 orange 1

Giải thích:

Với danh sách 4 từ khóa: ["apple", "banana", "apple", "cherry"]:

- Từ khóa "apple" xuất hiện 2 lần.
- Từ khóa "banana" xuất hiện 1 lần.
- Từ khóa "cherry" xuất hiện 1 lần.

Các từ khóa được sắp xếp đúng thứ tự từ điển a -> b -> c.

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq N \leq 50000$.

Bài 48 [CPPB-STL-03]: Nén Tọa Độ Mảng Số Lớn

Bối cảnh: Trong xử lý đồ họa máy tính, các điểm ảnh có tọa độ rất lớn lên tới 10^9 , nhưng tổng số lượng điểm ảnh thực tế xuất hiện trong khung hình chỉ tối đa là $N = 10^5$. Để tiết kiệm bộ nhớ khi đánh chỉ số mảng mà vẫn bảo toàn tuyệt đối thứ tự tương đối về độ lớn giữa các điểm ảnh, kỹ sư đồ họa áp dụng kỹ thuật nén tọa độ: thay thế mỗi giá trị bằng thứ hạng của nó từ 1 đến K (với K là số giá trị phân biệt).

Nhiệm vụ: Cho mảng gồm N số nguyên. Hãy lập trình thay thế mỗi phần tử trong mảng bằng thứ hạng nén của nó (bắt đầu từ 1 cho giá trị nhỏ nhất).

Đầu vào (Input):

- Dòng 1: Số nguyên N ($1 \leq N \leq 10^5$).
- Dòng 2: N số nguyên $A_1, A_2, \dots, A_N$ ($1 \leq A_i \leq 10^9$).

Đầu ra (Output):

- Dãy số sau khi nén tọa độ.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
5 100 20000 50 20000 100	1 2 0 2 1

Giải thích:

Với mảng số ban đầu là [100, 5, 100, 20, 5]:

Các giá trị phân biệt sau khi sắp xếp tăng dần là: $5 < 20 < 100$.

- Giá trị 5 nhỏ nhất nhận thứ hạng 1.
- Giá trị 20 nhận thứ hạng 2.
- Giá trị 100 nhận thứ hạng 3.

Mảng sau khi nén tọa độ tương ứng là: [3, 1, 3, 2, 1].

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq N \leq 10^5, 1 \leq A_i \leq 10^9$.

Bài 49 [CPPB-STL-04]: Tìm Phần Tử Nhỏ Nhất Lớn Hơn Hoặc Bằng X

Bối cảnh: Một sàn giao dịch hàng hóa trực tuyến duy trì một kho các lệnh bán với mức giá niêm yết $A_1, A_2, \dots, A_N$. Khi một nhà đầu tư gửi lệnh mua với mức trần giá tối đa là X , hệ thống giao dịch tự động cần nhanh chóng tìm ra lệnh bán có mức giá nhỏ nhất thỏa mãn điều kiện lớn hơn hoặc bằng X để khớp lệnh.

Nhiệm vụ: Cho mảng A gồm N số nguyên và Q truy vấn, mỗi truy vấn gồm một số nguyên X . Hãy lập trình tìm phần tử nhỏ nhất trong mảng lớn hơn hoặc bằng X . Nếu không có phần tử nào thỏa mãn, in ra -1 .

Đầu vào (Input):

- Dòng 1: Hai số N và Q ($1 \leq N, Q \leq 10^5$).
- Dòng 2: N số nguyên trong tập hợp ($1 \leq A_i \leq 10^9$).
- Q dòng tiếp theo: Mỗi dòng chứa 1 số X .

Đầu ra (Output):

- Kết quả Q truy vấn.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
5 3	
10 20 30 40 50	30
25	50
50	-1
60	

Giải thích:

Với mảng [1, 4, 6, 8, 10] và các truy vấn X :

- Truy vấn $X = 5$: Phần tử nhỏ nhất trong mảng ≥ 5 là 6.
- Truy vấn $X = 11$: Không có phần tử nào trong mảng ≥ 11 , in ra -1.

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq N, Q \leq 10^5, 1 \leq A_i, X \leq 10^9$.

Bài 50 [CPPB-STL-05]: Hàng Đợt Ưu Tiên K Phần Tử Lớn Nhất

Bối cảnh: Một nền tảng phát video trực tuyến theo dõi luồng tương tác của khán giả với hàng triệu lượt đánh giá. Trong luồng dữ liệu liên tục gồm N lượt đánh giá điểm số, ban biên tập cần liên tục lọc ra danh sách K lượt đánh giá có điểm số cao nhất để hiển thị lên bảng tin trang chủ.

Nhiệm vụ: Cho danh sách gồm N số nguyên và số nguyên dương K . Hãy lập trình tìm và in ra K phần tử lớn nhất trong dãy theo thứ tự giảm dần.

Đầu vào (Input):

- Dòng 1: Hai số N và K ($1 \leq K \leq N \leq 10^5$).
- Dòng 2: N số nguyên $A_1, A_2, \dots, A_N$ ($1 \leq A_i \leq 10^9$).

Đầu ra (Output):

- K số nguyên lớn nhất theo thứ tự giảm dần.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
6 3 10 50 30 20 60 40	60 50 40

Giải thích:

Với mảng gồm 6 phần tử $[3, 2, 1, 5, 6, 4]$ và cần lấy $K = 2$ phần tử lớn nhất: Hai phần tử lớn nhất trong dãy là 6 và 5. In ra theo thứ tự giảm dần: 6 5.

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq K \leq N \leq 10^5, 1 \leq A_i \leq 10^9$.

Bài 51 [CPPB-STL-06]: Quản Lý Tập Hợp Đa Trùng Lặp (Multiset)

Bối cảnh: Một kho hàng thông minh hỗ trợ 3 loại thao tác quản lý sản phẩm: thêm một sản phẩm có mã giá x vào kho, xóa bỏ đúng một sản phẩm có mã giá x khỏi kho (nếu có), và truy vấn sản phẩm có mã giá rẻ nhất hiện đang có trong kho.

Nhiệm vụ: Cho Q thao tác thuộc một trong 3 loại: '1 x' (thêm x), '2 x' (xóa một phần tử x), '3' (in ra giá trị nhỏ nhất hiện tại). Hãy lập trình mô phỏng lại hệ thống và in ra kết quả cho các thao tác loại 3.

Đầu vào (Input):

- Dòng 1: Số nguyên Q ($1 \leq Q \leq 50000$).
- Q dòng tiếp theo: type và x ($1 \leq x \leq 10^9$).

Đầu ra (Output):

- Kết quả các thao tác loại 3.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
5	
1 5	
1 5	2
3 5	1
2 5	
3 5	

Giải thích:

Với chuỗi thao tác: thêm 5, thêm 2, thêm 5, truy vấn min -> in ra 2; xóa 2, truy vấn min -> in ra 5.

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq Q \leq 50000, 1 \leq x \leq 10^9$.

Bài 52 [CPPB-STL-07]: Hợp Nhất Các Đoạn Số (Merge Intervals)

Bối cảnh: Một máy chủ lập lịch tiếp nhận N khoảng thời gian đặt lịch phòng họp, khoảng thời gian thứ i bắt đầu từ thời điểm L_i và kết thúc tại R_i . Một số khoảng thời gian bị chồng lấn (giao nhau) hoặc tiếp xúc liền kề. Ban quản trị tòa nhà cần hợp nhất tất cả các khoảng thời gian bị giao nhau lại thành các khối thời gian liền mạch độc lập.

Nhiệm vụ: Cho danh sách N khoảng thời gian $[L_i, R_i]$. Hãy lập trình hợp nhất các khoảng giao nhau và in ra danh sách các khoảng thời gian sau khi đã hợp nhất theo thứ tự thời điểm bắt đầu tăng dần.

Đầu vào (Input):

- Dòng 1: Số nguyên N ($1 \leq N \leq 50000$).
- N dòng tiếp theo: L_i, R_i ($1 \leq L_i \leq R_i \leq 10^9$).

Đầu ra (Output):

- Danh sách các đoạn sau khi hợp nhất.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
4	
1 3	3
2 6	1 6
8 10	8 10
15 18	15 18

Giải thích:

Với các khoảng thời gian $[1, 3]$, $[2, 6]$, $[8, 10]$, $[15, 18]$:

- Khoảng $[1, 3]$ và $[2, 6]$ giao nhau vì $2 \leq 3$, hợp nhất thành khoảng $[1, 6]$.
- Các khoảng $[8, 10]$ và $[15, 18]$ độc lập không giao nhau.

Kết quả thu được 3 khoảng: [1, 6], [8, 10], [15, 18].

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq N \leq 50000, 1 \leq L_i \leq R_i \leq 10^9$.

Bài 53 [CPPB-STL-08]: Tìm Trung Vị Động Trong Luồng Dữ Liệu

Bối cảnh: Trong hệ thống giám sát tải mạng máy tính, các gói tin liên tục gửi về các thông số độ trễ (ping). Để đánh giá độ trễ trung bình chuẩn xác mà không bị ảnh hưởng bởi các giá trị ngoại lai cá biệt, hệ thống cần tính toán giá trị trung vị (median) của luồng dữ liệu ngay sau mỗi khi tiếp nhận thêm một con số mới.

Nhiệm vụ: Cho một luồng dữ liệu gồm N số nguyên đến lần lượt từng số một. Với mỗi số được thêm vào, hãy in ra giá trị trung vị của toàn bộ dãy số đã nhận được từ đầu đến thời điểm đó (lấy phần nguyên dưới nếu số lượng phần tử chẵn).

Đầu vào (Input):

- Dòng 1: Số nguyên N ($1 \leq N \leq 50000$).
- Dòng 2: N số nguyên $A_1, A_2, \dots, A_N$ ($1 \leq A_i \leq 10^9$).

Đầu ra (Output):

- N giá trị trung vị cách nhau bởi dấu cách.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
4 5 15 1 3	5 5 5 3

Giải thích:

Với luồng dữ liệu đến lần lượt: 5, 15, 1, 3:

- Nhận 5: dãy [5] \to trung vị là 5.
- Nhận 15: dãy [5, 15] \to trung vị là 5 (hoặc trung bình lấy nguyên).
- Nhận 1: dãy [1, 5, 15] \to trung vị là 5.
- Nhận 3: dãy [1, 3, 5, 15] \to trung vị là 3 (hoặc phần nguyên).

Kết quả in ra dãy trung vị động tương ứng.

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq N \leq 50000, 1 \leq A_i \leq 10^9$.

Bài 54 [CPPB-STL-09]: Đếm Số Phần Tử Phân Biệt Trong Cửa Sổ K

Bối cảnh: Một máy quét giám sát an ninh quét qua một chuỗi N mã số xe lưu thông trên đường cao tốc. Máy quét sử dụng một khung quan sát (cửa sổ trượt) có kích thước cố định là K xe liên tiếp. Khi cửa sổ trượt dịch chuyển từng vị trí từ đầu đến cuối hàng xe, hệ thống cần đếm xem trong mỗi cửa sổ hiện tại có bao nhiêu mã số xe phân biệt nhau.

Nhiệm vụ: Cho mảng A gồm N số nguyên và kích thước cửa sổ K. Hãy lập trình in ra số lượng phần tử phân biệt trong mỗi cửa sổ kích thước K khi trượt từ trái sang phải.

Đầu vào (Input):

- Dòng 1: Hai số N và K ($1 \leq K \leq N \leq 10^5$).
- Dòng 2: N số nguyên $A_1, A_2, \dots, A_N$ ($1 \leq A_i \leq 10^9$).

Đầu ra (Output):

- $N - K + 1$ số nguyên.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
7 4 1 2 1 3 4 2 3	3 4 4 3

Giải thích:

Với mảng gồm 7 phần tử $[1, 2, 1, 3, 4, 2, 3]$ và kích thước cửa sổ $K = 4$:

- Cửa sổ 1 $[1, 2, 1, 3]$: gồm các giá trị phân biệt $\{1, 2, 3\}$ to 3 phần tử.
- Cửa sổ 2 $[2, 1, 3, 4]$: gồm $\{1, 2, 3, 4\}$ to 4 phần tử.
- Cửa sổ 3 $[1, 3, 4, 2]$: gồm $\{1, 2, 3, 4\}$ to 4 phần tử.
- Cửa sổ 4 $[3, 4, 2, 3]$: gồm $\{2, 3, 4\}$ to 3 phần tử.

Kết quả in ra: 3 4 4 3.

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq K \leq N \leq 10^5, 1 \leq A_i \leq 10^9$.

Bài 55 [CPPB-STL-10]: Nối Dây Chi Phí Nhỏ Nhất (Huffman Greedy)

Bối cảnh: Trong một xưởng cơ khí viễn thông, có N đoạn cáp quang rời rạc với chiều dài lần lượt là $L_1, L_2, \dots, L_N$. Công nhân cần hàn nối tất cả các đoạn cáp này lại thành một sợi cáp duy nhất dài liên tục. Mỗi lần hàn nối hai sợi cáp bất kỳ có chiều dài x và y , chi phí điện năng tiêu hao đúng bằng tổng chiều dài của hai sợi cáp đó ($x + y$). Sợi cáp mới ghép có chiều dài $x + y$ lại được đưa vào tập hợp để tiếp tục nối tiếp.

Nhiệm vụ: Cho danh sách chiều dài N đoạn cáp. Hãy lập trình tìm thứ tự nối cáp sao cho tổng chi phí hàn nối là nhỏ nhất có thể.

Đầu vào (Input):

- Dòng 1: Số nguyên N ($1 \leq N \leq 50000$).
- Dòng 2: N số nguyên $A_1, A_2, \dots, A_N$ ($1 \leq A_i \leq 10^5$).

Đầu ra (Output):

- Tổng chi phí nhỏ nhất.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
4 4 3 2 6	29

Giải thích:

Với 4 đoạn dây có độ dài [4, 3, 2, 6]:

1. Nối hai dây ngắn nhất 2 và 3 thành dây độ dài 5, chi phí tốn $2 + 3 = 5$. Danh sách dây còn: [4, 5, 6].
 2. Nối tiếp hai dây ngắn nhất 4 và 5 thành dây độ dài 9, chi phí tốn $4 + 5 = 9$. Danh sách dây còn: [6, 9].
 3. Nối hai dây cuối 6 và 9 thành dây độ dài 15, chi phí tốn $6 + 9 = 15$.
- Tổng chi phí nhỏ nhất là $5 + 9 + 15 = 29$.

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq N \leq 50000, 1 \leq A_i \leq 10^5$.

Bài 56 [CPPB-STL-11]: Lập Lịch Công Việc Số Máy Chủ Ít Nhất

Bối cảnh: Một trung tâm điện toán đám mây tiếp nhận N tác vụ xử lý dữ liệu. Tác vụ thứ i bắt đầu tại thời điểm S_i và kết thúc tại thời điểm E_i . Một máy chủ chỉ có thể thực hiện một tác vụ tại một thời điểm. Nếu hai tác vụ có khoảng thời gian chạy bị chồng chéo nhau, chúng bắt buộc phải được giao cho hai máy chủ vật lý khác nhau.

Nhiệm vụ: Cho danh sách thời gian bắt đầu và kết thúc của N tác vụ. Hãy lập trình xác định số lượng máy chủ vật lý tối thiểu cần chuẩn bị để phục vụ toàn bộ các tác vụ.

Đầu vào (Input):

- Dòng 1: Số nguyên N ($1 \leq N \leq 50000$).
- N dòng tiếp theo: S_i, E_i ($1 \leq S_i < E_i \leq 10^9$).

Đầu ra (Output):

- Số máy chủ ít nhất.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
3 0 30 5 10 15 20	2

Giải thích:

Với 3 tác vụ: [0, 30], [5, 10], [15, 20]:

Tại thời điểm $t = 5$, tác vụ 1 [0, 30] đang chạy trên máy 1, nên tác vụ 2 [5, 10] bắt buộc phải mở thêm máy 2.

Tại thời điểm $t = 15$, tác vụ 2 đã xong nhưng tác vụ 1 vẫn đang chạy, nên tác vụ 3 có thể tái sử dụng máy 2.

Số máy chủ tối thiểu cần dùng là 2.

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq N \leq 50000, 1 \leq S_i < E_i \leq 10^9$.

Bài 57 [CPPB-STL-12]: Đếm Cặp Số Có Hiệu Bằng K

Bối cảnh: Trong một thuật toán mã hóa khóa công khai, một danh sách gồm N số nguyên bí mật được đưa vào xử lý. Kỹ sư an ninh cần đếm xem trong danh sách có bao nhiêu cặp chỉ số (i, j) thỏa mãn điều kiện phần tử đứng sau lớn hơn phần tử đứng trước đúng một khoảng cách cố định K , tức là $A_j - A_i = K$ (với $i < j$).

Nhiệm vụ: Cho mảng A gồm N số nguyên và số nguyên K . Hãy lập trình đếm số lượng cặp (i, j) có $i < j$ và $A_j - A_i = K$.

Đầu vào (Input):

- Dòng 1: Hai số N và K ($1 \leq N \leq 10^5, 0 \leq K \leq 10^9$).
- Dòng 2: N số nguyên $A_1, A_2, \dots, A_N$ ($1 \leq A_i \leq 10^9$).

Đầu ra (Output):

- Số cặp thỏa mãn.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
5 2 1 5 3 4 2	3

Giải thích:

Với mảng $[1, 5, 3, 4, 2]$ và $K = 2$:

Các cặp có hiệu bằng 2 là:

- $A_2 - A_3 = 5 - 3 = 2$.

- $A_3 - A_1 = 3 - 1 = 2$.

- $A_4 - A_5 = 4 - 2 = 2$.

Có tổng cộng 3 cặp số thỏa mãn.

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq N \leq 10^5, 0 \leq K, A_i \leq 10^9$.

Bài 58 [CPPB-STL-13]: Phần Tử Xuất Hiện Nhiều Nhất (Mode)

Bối cảnh: Một hệ thống kiểm phiếu biểu quyết tiếp nhận N lá phiếu được đánh mã số nguyên. Ban kiểm phiếu cần xác định giá trị mã số nào xuất hiện với tần suất nhiều nhất trong hòm phiếu (giá trị

mốt - Mode). Nếu có nhiều mã số cùng đạt số phiếu cao nhất bằng nhau, hệ thống quy định sẽ ưu tiên chọn mã số có giá trị số học nhỏ nhất.

Nhiệm vụ: Cho danh sách N lá phiếu nguyên. Hãy lập trình tìm mã số xuất hiện nhiều lần nhất. Nếu có nhiều mã số cùng xuất hiện nhiều nhất, in ra mã số có giá trị nhỏ nhất trong số đó.

Đầu vào (Input):

- Dòng 1: Số nguyên N ($1 \leq N \leq 50000$).
- Dòng 2: N số nguyên $A_1, A_2, \dots, A_N$ ($1 \leq A_i \leq 10^9$).

Đầu ra (Output):

- Giá trị và tần suất.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
6 1 3 2 1 4 1	1 3

Giải thích:

Với dãy số $[1, 2, 2, 3, 1]$:

Cả hai số 1 và 2 đều xuất hiện đúng 2 lần (nhiều nhất). Theo quy định ưu tiên giá trị số học nhỏ hơn, ta chọn số 1. Kết quả in ra là 1.

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq N \leq 50000, 1 \leq A_i \leq 10^9$.

Bài 59 [CPPB-STL-14]: Cặp Điểm Gần Nhất (Closest Pair Of Points)

Bối cảnh: Trên màn hình hiển thị radar hàng hải, có N tàu biển đang hoạt động trên mặt biển. Tàu thứ i có tọa độ vị trí là (X_i, Y_i) . Để cảnh báo nguy cơ va chạm sớm cho đài chỉ huy, hệ thống an toàn hàng hải cần tính toán khoảng cách hình học nhỏ nhất (khoảng cách Euclid) giữa hai tàu biển bất kỳ trong toàn bộ khu vực.

Nhiệm vụ: Cho tọa độ của N điểm trên mặt phẳng. Hãy lập trình tìm bình phương khoảng cách Euclid nhỏ nhất giữa hai điểm bất kỳ.

Đầu vào (Input):

- Dòng 1: Số nguyên N ($2 \leq N \leq 10000$).
- N dòng tiếp theo: Mỗi dòng gồm tọa độ X_i, Y_i ($0 \leq X_i, Y_i \leq 10^5$).

Đầu ra (Output):

- Bình phương khoảng cách nhỏ nhất.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
4 0 0 1 2 3 1 4 0	5

Giải thích:

Với 4 điểm tọa độ: (0, 0), (1, 1), (2, 2), (2, 0):

Khoảng cách giữa điểm (1, 1) và (2, 2) có bình phương là $(2-1)^2 + (2-1)^2 = 1 + 1 = 2$.

Khoảng cách giữa điểm (1, 1) và (2, 0) có bình phương là $(2-1)^2 + (0-1)^2 = 1 + 1 = 2$.

Bình phương khoảng cách nhỏ nhất giữa hai điểm bất kỳ là 2.

Ràng buộc & Giới hạn:

- 100% số test có $2 \leq N \leq 10000, 0 \leq X_i, Y_i \leq 10^5$.

Bài 60 [CPPB-STL-15]: Hệ Thống Xếp Hạng Thi Đấu Dynamic

Bối cảnh: Tại kỳ thi lập trình thuật toán trực tuyến DKOJ của iKHEDU, hệ thống chấm thi thời gian thực cần liên tục ghi nhận điểm số và cập nhật kết quả thi đấu của các thí sinh. Mỗi khi thí sinh nộp bài hoặc cần tra cứu điểm, hệ thống sẽ thực thi một trong hai loại thao tác: cộng điểm cho một thí sinh hoặc truy vấn điểm tích lũy hiện tại của thí sinh đó.

Nhiệm vụ: Cho danh sách Q thao tác của hệ thống. Bạn hãy lập trình mô phỏng lại hệ thống và in ra kết quả cho mỗi thao tác tra cứu điểm.

Đầu vào (Input):

- Dòng 1: Số nguyên Q ($1 \leq Q \leq 20000$).
- Q dòng tiếp theo chứa các truy vấn.

Đầu ra (Output):

- Kết quả các truy vấn loại 2.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
4 1 Alice 100 1 Bob 150 2 Alice 1 Alice 60	100

Giải thích:

Diễn biến 4 thao tác của hệ thống thi đấu:

1. Thao tác 1 ('1 Alice 100'): Alice được cộng 100 điểm. Điểm hiện tại của Alice là 100.

2. Thao tác 2 (`1 Bob 150`): Bob được cộng 150 điểm. Điểm hiện tại của Bob là 150.
3. Thao tác 3 (`2 Alice`): Truy vấn điểm của Alice. Hệ thống in ra điểm hiện tại là 100.
4. Thao tác 4 (`1 Alice 60`): Alice được cộng thêm 60 điểm nữa. Điểm tích lũy mới của Alice trở thành $100 + 60 = 160$.

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq Q \leq 20000$.

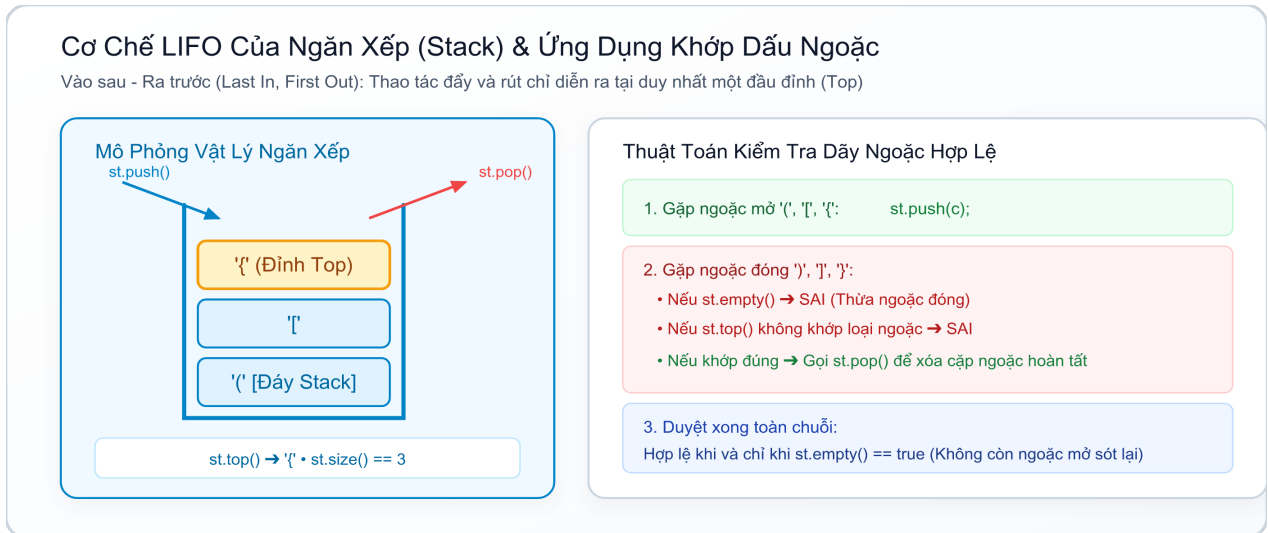


Bài 17: Cấu trúc dữ liệu ngăn xếp (stack) & Monotonic Stack

1. Bản chất cấu trúc dữ liệu ngăn xếp (stack)

Ngăn xếp (Stack) là cấu trúc dữ liệu hoạt động theo nguyên lý **LIFO (Last In, First Out — Vào sau, Ra trước)**:

- Phần tử được thêm vào cuối cùng sẽ là phần tử đầu tiên được lấy ra.
- Các thao tác cơ bản trong C++ `std::stack`: `push(x)` (thêm vào đỉnh), `pop()` (xóa đỉnh), `top()` (truy cập đỉnh), `empty()`, `size()`. Toàn bộ thao tác đều đạt thời gian tối ưu tuyệt đối $\mathcal{O}(1)$.



Cơ chế LIFO của Stack và Khớp Dấu Ngoặc

2. Kỹ thuật ngăn xếp đơn điệu (Monotonic Stack)

2.1. Bản chất bài toán & khi nào cần Monotonic Stack?

- **Vấn đề:** Cho mảng A gồm N phần tử. Với mỗi vị trí i , cần tìm vị trí phần tử **đầu tiên bên phải (hoặc bên trái)** có giá trị lớn hơn (hoặc nhỏ hơn) $A[i]$.
- **Cách ngây thơ:** Duyệt 2 vòng lặp lồng nhau $\Rightarrow \mathcal{O}(N^2)$ (bị TLE khi $N = 10^5$).
- **Nguyên lý Monotonic Stack:** Duy trì một ngăn xếp chứa các chỉ số mà giá trị tương ứng trong mảng luôn tuân theo tính chất **đơn điệu** (tăng dần hoặc giảm dần). Khi gặp phần tử mới vi phạm tính đơn điệu, ta liên tục `pop()` các phần tử ở đỉnh ngăn xếp và ghi nhận đáp án cho chúng.

Monotonic Stack: Tìm Phần Tử Lớn Hơn Tiếp Theo (NGE)

Bất biến duy trì các chỉ số i có giá trị $A[i]$ giảm dần nghiêm ngặt trên ngăn xếp

MẢNG ĐẦU VÀO $A = [4, 2, 1, 5, 3]$



Trạng thái Stack trước khi gặp 5:

st = [đáy: 4 ($i=0$), 2 ($i=1$), đỉnh: 1 ($i=2$)]

Phần tử 5 lớn hơn tất cả phần tử hiện có trong Stack!

Diễn Biến Khi Gặp $A[3] = 5$ (Vòng Lặp While st.top() < $A[i]$):

① So sánh $5 > 1 \rightarrow$ Pop đỉnh 1 ra khỏi Stack, ghi nhận $NGE[2] = 5$

② So sánh tiếp $5 > 2 \rightarrow$ Pop đỉnh 2 ra khỏi Stack, ghi nhận $NGE[1] = 5$

③ So sánh tiếp $5 > 4 \rightarrow$ Pop đỉnh 4 ra khỏi Stack, ghi nhận $NGE[0] = 5$. Cuối cùng push chỉ số 3 vào Stack rỗng.

$\rightarrow$ Kết quả toàn mảng: $NGE = [5, 5, 5, -1, -1]$

Mỗi phần tử chỉ Push và Pop đúng 1 lần $\rightarrow$ Độ phức tạp $O(N)$

Mô hình Monotonic Stack NGE

2.2. Phân tích độ phức tạp khấu hao (amortized analysis $O(N)$)

Mỗi phần tử của mảng được push() vào ngăn xếp đúng 1 lần và bị pop() ra khỏi ngăn xếp tối đa 1 lần trong toàn bộ quá trình chạy.

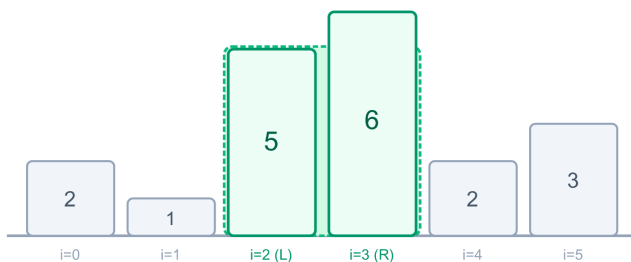
Tổng số thao tác trên Stack $\leq 2N \Rightarrow$ Thời gian trung bình $O(N)$!

3. Bài toán kinh điển: Hình chữ nhật lớn nhất trên biểu đồ cột (largest rectangle in histogram)

Hình Chữ Nhật Lớn Nhất Trên Biểu Đồ Cột (Largest Rectangle in Histogram)

Mỗi cột $h[i]$ mở rộng sang trái ($L[i]$) và sang phải ($R[i]$) chừng nào các cột lân cận vẫn có chiều cao $\geq h[i]$

BIỂU ĐỒ CỘT VÀ VÙNG DIỆN TÍCH TỐI ƯU



Diện Tích Tối Ưu

- Cột xét: $h[2] = 5$
- Biên trái $L[2]$: index 2
- Biên phải $R[2]$: index 3
- Chiều rộng $w = 3 - 2 + 1 = 2$

Max Area = $5 \times 2 = 10$

★ Chiến lược Monotonic Stack:
Tìm $L[i]$ và $R[i]$ cho toàn bộ N cột trong $O(N)$

Hình chữ nhật lớn nhất trên Histogram

- **Bản chất:** Với mỗi cột i có chiều cao $H[i]$, ta cần tìm:
 1. $L[i]$: Vị trí cột đầu tiên bên trái có chiều cao $< H[i]$.
 2. $R[i]$: Vị trí cột đầu tiên bên phải có chiều cao $< H[i]$.
- Khi đó, hình chữ nhật lớn nhất nhận $H[i]$ làm chiều cao tối đa sẽ có chiều rộng $W = R[i] - L[i] - 1$, diện tích là $S[i] = H[i] \times (R[i] - L[i] - 1)$.

- Sử dụng 2 lượt Monotonic Stack (hoặc 1 lượt thông minh), ta tính toàn bộ mảng L và R trong $\mathcal{O}(N)$.

4. Các bẫy lỗi lập trình kinh điển (bug traps)

1. Bẫy gọi `st.top()` hoặc `st.pop()` khi Stack rỗng:

- Truy cập đỉnh ngăn xếp khi `st.empty() == true` sẽ dẫn đến lỗi bộ nhớ nghiêm trọng (Segmentation Fault / Runtime Error).
- Quy tắc an toàn:** Luôn kiểm tra `while (!st.empty() && ...)` trước khi gọi `st.top()` hay `st.pop()`.

2. Bẫy quên kiểm tra `st.empty()` ở cuối bài toán Dãy ngoặc đúng:

- Sau khi duyệt hết chuỗi, nếu không còn ngoặc đóng nào nhưng trong stack vẫn còn ngoặc mở dư thừa (ví dụ chuỗi `"(((")`), dãy ngoặc vẫn là **KHÔNG HỢP LỆ**.
- Điều kiện đủ:** Dãy hợp lệ khi và chỉ khi không bị lỗi giữa chừng VÀ `st.empty() == true` ở cuối.

3. Bẫy tràn số khi tính diện tích hình chữ nhật lớn nhất:

- Chiều cao $H[i] \leq 10^9$ và chiều rộng $W \leq 10^5 \Rightarrow$ Diện tích có thể lên tới 10^{14} , vượt quá giới hạn 32-bit int. Bắt buộc phải ép kiểu sang long long.

5. Mẫu cài đặt chuẩn thi đấu (competitive templates)

Mẫu 1: Tìm phần tử lớn hơn tiếp theo (next greater element)

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;

    if (n <= 0) return 0;

    vector<long long> a(n);

    for (int i = 0; i < n; ++i) {
        cin >> a[i];
    }

    vector<long long> nge(n, -1);

    stack<int> st; // Lưu chỉ số

    for (int i = 0; i < n; ++i) {
        while (!st.empty() && a[i] > a[st.top()]) {
            nge[st.top()] = a[i];
            st.pop();
        }
        st.push(i);
    }
}
```



```

    for (int i = 0; i < n; ++i) {
        cout << nge[i] << (i + 1 == n ? "" : " ");
    }
    cout << "\n";

    return 0;
}

```

Mẫu 2: Hình chữ nhật lớn nhất trên biểu đồ cột (histogram)

```

#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;

    if (n <= 0) return 0;

    vector<long long> h(n);

    for (int i = 0; i < n; ++i) {
        cin >> h[i];
    }

    // Thêm phần tử Lính canh 0 ở cuối để đẩy toàn bộ stack ra
    h.push_back(0);
    stack<int> st;

    long long max_area = 0;

    for (int i = 0; i <= n; ++i) {
        while (!st.empty() && h[i] < h[st.top()]) {
            long long height = h[st.top()];
            st.pop();
            long long width = st.empty() ? i : (i - st.top() - 1);
            max_area = max(max_area, height * width);
        }
        st.push(i);
    }

    cout << max_area << "\n";
    return 0;
}

```

Bài tập thực hành

Bài 61 [CPPB-STK-01]: Kiểm Tra Dãy Ngoặc Đúng Đơn Loại

Bối cảnh: Trong một trình biên dịch ngôn ngữ lập trình, việc đóng mở các cặp dấu ngoặc tròn '(' và ')' phải tuân thủ nghiêm ngặt quy tắc ngữ pháp: Mỗi dấu mở ngoặc '(' phải có một dấu đóng ngoặc ')' tương ứng xuất hiện sau nó, và tại mọi thời điểm tính từ đầu chuỗi, số lượng dấu mở ngoặc không bao giờ được ít hơn số lượng dấu đóng ngoặc.

Nhiệm vụ: Cho một chuỗi chỉ gồm các ký tự '(' và ')'. Hãy lập trình kiểm tra xem chuỗi đó có phải là một dãy ngoặc đúng hay không. Nếu đúng in ra 'YES', ngược lại in ra 'NO'.

Đầu vào (Input):

- Một dòng duy nhất chứa chuỗi S ($1 \leq |S| \leq 10^5$).

Đầu ra (Output):

- YES hoặc NO.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
((()))	YES

Giải thích:

Với chuỗi $S = "((()))"$:

Mỗi dấu mở ngoặc đều có đúng một dấu đóng ngoặc tương ứng ghép đôi hợp lệ và không có dấu đóng ngoặc nào bị thừa. Kết quả in ra là YES.

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq |S| \leq 10^5$.

Bài 62 [CPPB-STK-02]: Dây Ngoặc Hỗn Hợp Nhiều Loại

Bối cảnh: Trình biên dịch mã nguồn mở rộng hỗ trợ đồng thời 3 loại cặp dấu ngoặc khác nhau: ngoặc tròn '()', ngoặc vuông '[]' và ngoặc nhọn '{}'. Một chuỗi ngoặc hỗn hợp được coi là hợp lệ nếu các cặp ngoặc cùng loại được đóng mở đúng quy tắc lồng nhau, không bị đóng chéo (ví dụ: '[()]' là sai quy tắc vì ngoặc tròn mở trước nhưng lại bị ngoặc vuông xen vào đóng trước).

Nhiệm vụ: Cho chuỗi S chỉ gồm các ký tự '(', ')', '[', ']', '{', '}'. Hãy lập trình kiểm tra tính hợp lệ của chuỗi. Nếu hợp lệ in ra 'YES', ngược lại in ra 'NO'.

Đầu vào (Input):

- Một dòng chứa chuỗi S ($1 \leq |S| \leq 10^5$).

Đầu ra (Output):

- YES hoặc NO.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
{[()]}	YES

Giải thích:

Với chuỗi $S = "{[()]}"$:

Cặp ngoặc tròn nằm hoàn toàn bên trong ngoặc vuông, và ngoặc vuông nằm bên trong ngoặc nhọn. Quy tắc lồng nhau được bảo đảm trọn vẹn, kết quả in ra là YES.

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq |S| \leq 10^5$.

Bài 63 [CPPB-STK-03]: Đánh Giá Biểu Thức Hậu Tố (Reverse Polish Notation)

Bối cảnh: Trong thiết kế vi xử lý và máy tính cầm tay, biểu thức toán học thường được chuyển đổi sang dạng ký pháp nghịch đảo Ba Lan (hậu tố - Postfix) để máy tính dễ dàng tính toán mà không cần quan tâm đến độ ưu tiên của dấu ngoặc. Trong biểu thức hậu tố, toán tử luôn đứng sau hai toán hạng của nó (ví dụ: ``2 3 +`` tương đương ``2 + 3 = 5``).

Nhiệm vụ: Cho một biểu thức hậu tố gồm các số nguyên và 4 phép toán cơ bản ``+``, ``-``, ``*``, ``/`` (chia lấy phần nguyên). Hãy lập trình tính và in ra giá trị cuối cùng của biểu thức.

Đầu vào (Input):

- Dòng 1: Số nguyên N ($1 \leq N \leq 10^5$).
- Dòng 2: N token cách nhau bởi khoảng trắng.

Đầu ra (Output):

- Giá trị số nguyên của biểu thức.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
5 2 1 + 3 *	9

Giải thích:

Với biểu thức hậu tố `"2 1 + 3 *"`:

1. Gặp toán tử ``+``: Thực hiện $2 + 1 = 3$.
2. Biểu thức trở thành `"3 3 *"`.
3. Gặp toán tử ``*``: Thực hiện $3 \times 3 = 9$.

Giá trị cuối cùng thu được là 9.

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq N \leq 10^5$.

Bài 64 [CPPB-STK-04]: Xóa Ký Tự Trùng Lặp Liên Kề

Bối cảnh: Một trò chơi xếp chữ tương tác có quy tắc: Khi hai ký tự giống hệt nhau đứng sát cạnh nhau trong chuỗi, chúng sẽ tự động va chạm và cùng biến mất. Sau khi hai ký tự đó biến mất, hai phần còn lại của chuỗi sẽ dồn sát vào nhau và nếu lại tạo ra hai ký tự giống nhau kề nhau thì quá trình triệt tiêu lại tiếp tục diễn ra cho đến khi không còn cặp ký tự kề nhau nào giống nhau.

Nhiệm vụ: Cho chuỗi ký tự S . Hãy lập trình xác định chuỗi ký tự cuối cùng thu được sau khi tất cả các cặp trùng lặp liên kề đã bị triệt tiêu hoàn toàn.

Đầu vào (Input):

- Một dòng chứa chuỗi S ($1 \leq |S| \leq 10^5$).

Đầu ra (Output):

- Chuỗi kết quả hoặc EMPTY.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
abbaca	ca

Giải thích:

Với chuỗi $S = \text{"abbaca"}$:

1. Cặp "bb" ở giữa triệt tiêu \to chuỗi còn lại là "aaca".
2. Cặp "aa" mới tạo thành kề nhau lại tiếp tục triệt tiêu \to chuỗi còn lại là "ca".

Không còn cặp nào trùng nhau kề nhau, chuỗi kết quả in ra là ca.

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq |S| \leq 10^5$.

Bài 65 [CPPB-STK-05]: Phần Tử Lớn Hơn Tiếp Theo (Next Greater Element)

Bối cảnh: Một đồ thị phân tích thị trường chứng khoán gồm N mốc thời gian liên tiếp với mức giá cổ phiếu lần lượt là $A_1, A_2, \dots, A_N$. Với mỗi phiên giao dịch tại mốc i , nhà phân tích tài chính muốn biết mức giá của phiên giao dịch đầu tiên xuất hiện sau phiên i (bên phải i) có giá trị lớn hơn hẳn A_i . Nếu từ mốc i về sau không có phiên nào có giá cao hơn, ghi nhận giá trị -1 .

Nhiệm vụ: Cho dãy số nguyên A gồm N phần tử. Với mỗi phần tử trong mảng, hãy tìm phần tử đầu tiên nằm bên phải nó có giá trị lớn hơn nó. Nếu không có, gán giá trị -1 .

Đầu vào (Input):

- Dòng 1: Số nguyên N ($1 \leq N \leq 10^5$).
- Dòng 2: N số nguyên $A_1, A_2, \dots, A_N$ ($1 \leq A_i \leq 10^9$).

Đầu ra (Output):

- N số nguyên cách nhau bởi khoảng trắng.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
4 4 5 2 25	5 25 25 -1

Giải thích:

Với mảng gồm 4 phần tử $[4, 5, 2, 25]$:

- Phần tử 4: Bên phải phần tử đầu tiên lớn hơn 4 là 5.
- Phần tử 5: Bên phải phần tử đầu tiên lớn hơn 5 là 25.
- Phần tử 2: Bên phải phần tử đầu tiên lớn hơn 2 là 25.
- Phần tử 25: Bên phải không còn số nào lớn hơn 25 \to in ra -1.

Kết quả in ra: 5 25 25 -1.

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq N \leq 10^5, 1 \leq A_i \leq 10^9$.

Bài 66 [CPPB-STK-06]: Phần Tử Nhỏ Hơn Gần Nhất Bên Trái (Previous Smaller Element)

Bối cảnh: Một hệ thống băng tải tự động gồm N vị trí cảm biến. Với mỗi vị trí cảm biến i , hệ thống cần xác định vị trí của cảm biến đầu tiên nằm về phía bên trái có chỉ số đọc nhỏ hơn cảm biến i để tính toán độ dốc địa hình ngược chiều dòng sản phẩm.

Nhiệm vụ: Cho mảng A gồm N số nguyên. Với mỗi phần tử A_i , hãy tìm phần tử đầu tiên nằm bên trái nó có giá trị nhỏ hơn nó. Nếu không có phần tử nào thỏa mãn, gán -1 .

Đầu vào (Input):

- Dòng 1: Số nguyên N ($1 \leq N \leq 10^5$).
- Dòng 2: N số nguyên $A_1, A_2, \dots, A_N$ ($1 \leq A_i \leq 10^9$).

Đầu ra (Output):

- N số nguyên.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
5 4 5 2 10 8	-1 4 -1 2 2

Giải thích:

Với mảng $[4, 5, 2, 10, 8]$:

- Số 4: Bên trái không có số nào $\rightarrow -1$.
- Số 5: Số đầu tiên bên trái nhỏ hơn 5 là 4.
- Số 2: Bên trái không có số nào nhỏ hơn 2 $\rightarrow -1$.
- Số 10: Số đầu tiên bên trái nhỏ hơn 10 là 2.
- Số 8: Số đầu tiên bên trái nhỏ hơn 8 là 2.

Kết quả in ra: -1 4 -1 2 2.

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq N \leq 10^5, 1 \leq A_i \leq 10^9$.

Bài 67 [CPPB-STK-07]: Độ Dài Đoạn Ngoặc Đúng Liên Tiếp Dài Nhất

Bối cảnh: Trong một chuỗi ký tự ngoặc hỗn độn bị lỗi truyền dẫn, kỹ sư phần mềm muốn khôi phục lại phần thông điệp hợp lệ dài nhất. Hãy tìm độ dài của một đoạn con liên tiếp dài nhất trong chuỗi sao cho đoạn con đó tạo thành một dãy ngoặc đúng hoàn chỉnh.

Nhiệm vụ: Cho chuỗi S chỉ gồm các ký tự '(' và ')'. Hãy lập trình tìm độ dài lớn nhất của một đoạn con liên tiếp là dãy ngoặc đúng.

Đầu vào (Input):

- Một dòng chứa chuỗi S ($1 \leq |S| \leq 10^5$).

Đầu ra (Output):

- Độ dài lớn nhất.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
)()())	4

Giải thích:

Với chuỗi $S = ")()())"$:

Đoạn con liên tiếp bắt đầu từ vị trí 1 đến vị trí 4 là $)()()$ tạo thành dãy ngoặc đúng hoàn chỉnh có độ dài bằng 4. Đây là đoạn ngoặc đúng dài nhất.

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq |S| \leq 10^5$.

Bài 68 [CPPB-STK-08]: Xóa K Chữ Số Để Được Số Nhỏ Nhất

Bối cảnh: Một mã định danh số lớn gồm N chữ số dạng chuỗi ký tự. Do yêu cầu rút gọn mã trong hệ thống lưu trữ, quản trị viên cần xóa bỏ đúng K chữ số bất kỳ khỏi chuỗi sao cho các chữ số còn lại giữ nguyên thứ tự ban đầu và tạo thành một số nguyên có giá trị nhỏ nhất có thể (không để số 0 vô nghĩa đứng ở đầu nếu kết quả lớn hơn 0).

Nhiệm vụ: Cho chuỗi số S và số nguyên K . Hãy lập trình tìm số nguyên nhỏ nhất thu được sau khi xóa đúng K chữ số.

Đầu vào (Input):

- Một dòng gồm chuỗi số S ($1 \leq |S| \leq 10^5$) và số K ($0 \leq K < |S|$).

Đầu ra (Output):

- Số nhỏ nhất tìm được.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
1432219 3	1219

Giải thích:

Với số ban đầu là "1432219" và cần xóa đi $K = 3$ chữ số:

Ta xóa các chữ số 4, 3, 2 tại các vị trí đầu để giữ lại số "1219". Đây là số nguyên nhỏ nhất có thể tạo thành.

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq |S| \leq 10^5$.

Bài 69 [CPPB-STK-09]: Hình Chữ Nhật Lớn Nhất Trên Histogram

Bối cảnh: Một biểu đồ cột (histogram) gồm N cột hình chữ nhật đứng xếp kề sát nhau trên cùng một đường nằm ngang. Mỗi cột có chiều rộng cố định là 1 đơn vị, cột thứ i có chiều cao là H_i . Bạn thiết kế đồ họa cần tìm diện tích của hình chữ nhật lớn nhất có thể vẽ lọt hoàn toàn vào bên trong biểu đồ cột này.

Nhiệm vụ: Cho danh sách chiều cao của N cột. Hãy lập trình tìm diện tích lớn nhất của một hình chữ nhật nằm gọn bên trong biểu đồ cột.

Đầu vào (Input):

- Dòng 1: Số nguyên N ($1 \leq N \leq 10^5$).
- Dòng 2: N số nguyên $H_1, H_2, \dots, H_N$ ($0 \leq H_i \leq 10^9$).

Đầu ra (Output):

- Diện tích lớn nhất.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
6 2 1 5 6 2 3	10

Giải thích:

Với biểu đồ có chiều cao các cột là $[2, 1, 5, 6, 2, 3]$:

Hình chữ nhật lớn nhất được tạo thành bởi hai cột có chiều cao 5 và 6. Chiều cao của hình chữ nhật này là $\min(5, 6) = 5$, chiều rộng là 2 cột. Diện tích lớn nhất đạt được là $5 \times 2 = 10$.

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq N \leq 10^5, 0 \leq H_i \leq 10^9$.

Bài 70 [CPPB-STK-10]: Hứng Nước Mưa (Trapping Rain Water)

Bối cảnh: Một mô hình địa hình gồm N khối bê tông thẳng đứng có chiều rộng 1 đơn vị được xếp thẳng hàng kề sát nhau, khối thứ i có độ cao H_i . Sau một cơn mưa rào lớn, nước mưa sẽ đọng lại ở các thung lũng giữa các khối bê tông cao hơn hai bên. Hãy tính toán tổng lượng nước mưa tối đa có thể đọng lại sau cơn mưa.

Nhiệm vụ: Cho danh sách chiều cao của N khối bê tông. Hãy lập trình tính tổng đơn vị thể tích nước mưa có thể đọng lại.

Đầu vào (Input):

- Dòng 1: Số nguyên N ($1 \leq N \leq 10^5$).
- Dòng 2: N số nguyên không âm $H_1, H_2, \dots, H_N$ ($0 \leq H_i \leq 10^5$).

Đầu ra (Output):

- Tổng lượng nước mưa đọng lại.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
12 0 1 0 2 1 0 1 3 2 1 2 1	6

Giải thích:

Với độ cao địa hình là [0, 1, 0, 2, 1, 0, 1, 3, 2, 1, 2, 1]:

Nước mưa sẽ bị giữ lại ở các vùng trũng giữa các cột cao:

- Tại vị trí 2: nước đọng 1 đơn vị.
- Tại vị trí 4: nước đọng 1 đơn vị.
- Tại vị trí 5: nước đọng 2 đơn vị.
- Tại vị trí 6: nước đọng 1 đơn vị.
- Tại vị trí 9: nước đọng 1 đơn vị.

Tổng lượng nước đọng lại là $1 + 1 + 2 + 1 + 1 = 6$ đơn vị.

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq N \leq 10^5, 0 \leq H_i \leq 10^5$.

Bài 71 [CPPB-STK-11]: Hình Chữ Nhật Toàn 1 Lớn Nhất Trong Ma Trận

Bối cảnh: Một tấm tôn kích thước $N \times M$ ô vuông bị đục thủng một số vị trí (ô bị đục ký hiệu bằng số '0', ô nguyên vẹn ký hiệu bằng số '1'). Người thợ cơ khí cần cắt ra từ tấm tôn một miếng hình chữ nhật nguyên vẹn (chỉ chứa toàn số '1') sao cho diện tích của miếng tôn cắt ra là lớn nhất có thể.

Nhiệm vụ: Cho ma trận nhị phân $N \times M$. Hãy lập trình tìm diện tích lớn nhất của hình chữ nhật con chỉ chứa toàn số '1'.

Đầu vào (Input):

- Dòng 1: Hai số N và M ($1 \leq N, M \leq 1000$).
- N dòng tiếp theo biểu diễn ma trận.

Đầu ra (Output):

- Diện tích lớn nhất.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
4 5 10100 10111 11111 10010	6

Giải thích:

Với ma trận kích thước 4×5 :

Hình chữ nhật con toàn số 1 lớn nhất có kích thước 2×3 (chiều cao 2, chiều rộng 3) gồm 6 ô số 1. Diện tích lớn nhất đạt được là 6.

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq N, M \leq 1000$.

Bài 72 [CPPB-STK-12]: Tổng Giá Trị Nhỏ Nhất Của Mọi Đoạn Con

Bối cảnh: Cho mảng A gồm N số nguyên. Một đoạn con của mảng là một chuỗi các phần tử liên tiếp $A[L..R]$ ($1 \leq L \leq R \leq N$). Với mỗi đoạn con, ta xác định giá trị nhỏ nhất của đoạn con đó là $\min(A[L..R])$. Ban toán học cần tính tổng của tất cả các giá trị nhỏ nhất này trên mọi đoạn con có thể tạo thành từ mảng.

Nhiệm vụ: Cho mảng A gồm N số nguyên. Hãy lập trình tính tổng giá trị nhỏ nhất của tất cả các đoạn con, lấy dư cho $10^9 + 7$.

Đầu vào (Input):

- Dòng 1: Số nguyên N ($1 \leq N \leq 10^5$).
- Dòng 2: N số nguyên dương $A_1, A_2, \dots, A_N$ ($1 \leq A_i \leq 10^9$).

Đầu ra (Output):

- Tổng theo modulo $10^9 + 7$.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
4 3 1 2 4	17

Giải thích:

Với mảng gồm 4 phần tử [3, 1, 2, 4]:

Các đoạn con có giá trị nhỏ nhất tương ứng:

- Độ dài 1: [3] -> 3, [1] -> 1, [2] -> 2, [4] -> 4 (tổng = 10).
- Độ dài 2: [3,1] -> 1, [1,2] -> 1, [2,4] -> 2 (tổng = 4).
- Độ dài 3: [3,1,2] -> 1, [1,2,4] -> 1 (tổng = 2).
- Độ dài 4: [3,1,2,4] -> 1 (tổng = 1).

Tổng cộng toàn bộ là $10 + 4 + 2 + 1 = 17$.

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq N \leq 10^5, 1 \leq A_i \leq 10^9$.

Bài 73 [CPPB-STK-13]: Next Greater Element Trên Mảng Vòng Tròn

Bối cảnh: Một vòng đu quay gồm N cabin được đánh số theo vòng tròn từ 1 đến N, cabin thứ N lại kết nối liền kề với cabin số 1. Cabin thứ i có chiều cao A_i . Khi đứng tại cabin i và nhìn theo chiều

kim đồng hồ quanh vòng tròn, hãy tìm giá trị của cabin đầu tiên có chiều cao lớn hơn cabin i . Nếu đi hết một vòng tròn mà không có cabin nào cao hơn, ghi nhận -1 .

Nhiệm vụ: Cho mảng tròn A gồm N phần tử. Với mỗi phần tử, hãy tìm phần tử lớn hơn đầu tiên tiếp theo trên mảng vòng tròn. Nếu không có, in ra -1 .

Đầu vào (Input):

- Dòng 1: Số nguyên N ($1 \leq N \leq 10^5$).
- Dòng 2: N số nguyên $A_1, A_2, \dots, A_N$ ($1 \leq A_i \leq 10^9$).

Đầu ra (Output):

- N số nguyên.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
3 1 2 1	2 -1 2

Giải thích:

Với mảng tròn gồm 3 phần tử $[1, 2, 1]$:

- Phần tử 1 (đầu): Nhìn tiếp theo gặp số 2 lớn hơn 1 \to in ra 2.
- Phần tử 2 (giữa): Nhìn tiếp theo gặp 1, rồi vòng lại đầu gặp 1, không có số nào lớn hơn 2 \to in ra -1.
- Phần tử 1 (cuối): Vòng lại đầu mảng gặp 1, tiếp tục gặp 2 lớn hơn 1 \to in ra 2.

Kết quả in ra: 2 -1 2.

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq N \leq 10^5, 1 \leq A_i \leq 10^9$.

Bài 74 [CPPB-STK-14]: Tầm Nhìn Tòa Tháp (Stock Span)

Bối cảnh: Trên một tuyến phố ven biển có N tòa tháp cao tầng xếp thẳng hàng từ trái sang phải, tòa tháp thứ i có chiều cao H_i . Tầm nhìn về phía sau của một tòa tháp i được định nghĩa là số lượng tòa tháp liên tiếp nằm ngay bên trái nó (tính cả chính nó) mà có chiều cao nhỏ hơn hoặc bằng H_i .

Nhiệm vụ: Cho danh sách chiều cao của N tòa tháp. Hãy lập trình tính tầm nhìn cho từng tòa tháp trong dãy.

Đầu vào (Input):

- Dòng 1: Số nguyên N ($1 \leq N \leq 10^5$).
- Dòng 2: N số nguyên $H_1, H_2, \dots, H_N$ ($1 \leq H_i \leq 10^9$).

Đầu ra (Output):

- N số nguyên.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
7 100 80 60 70 60 75 85	1 1 1 2 1 4 6

Giải thích:

Với chiều cao các tháp [100, 80, 60, 70, 60, 75, 85]:

- Tháp 1 (100): tầm nhìn 1 (chính nó).
- Tháp 2 (80): tầm nhìn 1.
- Tháp 3 (60): tầm nhìn 1.
- Tháp 4 (70): bao trùm tháp 60 và chính nó \to tầm nhìn 2.
- Tháp 5 (60): tầm nhìn 1.
- Tháp 6 (75): bao trùm tháp 60, 70, 60 và chính nó \to tầm nhìn 4.
- Tháp 7 (85): bao trùm tất cả các tháp trừ tháp 100 \to tầm nhìn 6.

Kết quả in ra: 1 1 1 2 1 4 6.

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq N \leq 10^5, 1 \leq H_i \leq 10^9$.

Bài 75 [CPPB-STK-15]: Đánh Giá Biểu Thức Trung Tổ (Infix Expression)

Bối cảnh: Một máy tính bỏ túi thông minh tiếp nhận biểu thức số học viết ở dạng trung tổ tự nhiên (dạng toán học thông thường có chứa các dấu ngoặc tròn `()`, số nguyên và 4 phép toán cơ bản `+`, `-`, `\*`, `/`). Biểu thức tuân thủ độ ưu tiên toán học: nhân chia trước, cộng trừ sau, trong ngoặc trước, ngoài ngoặc sau.

Nhiệm vụ: Cho một chuỗi biểu thức số học trung tổ hợp lệ. Hãy lập trình tính toán và in ra giá trị số học cuối cùng của biểu thức.

Đầu vào (Input):

- Một dòng chứa chuỗi biểu thức S ($1 \leq |S| \leq 10^5$).

Đầu ra (Output):

- Giá trị số nguyên của biểu thức.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
3+2*2	7

Giải thích:

Với biểu thức số học "1 + (2 * 3) - 4 / 2":

1. Tính trong ngoặc: $2 \times 3 = 6$.
2. Biểu thức trở thành: $1 + 6 - 4 / 2$.

3. Thực hiện phép chia: $4 / 2 = 2$.

4. Biểu thức thành: $1 + 6 - 2 = 5$.

Kết quả in ra là 5.

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq |S| \leq 10^5$.



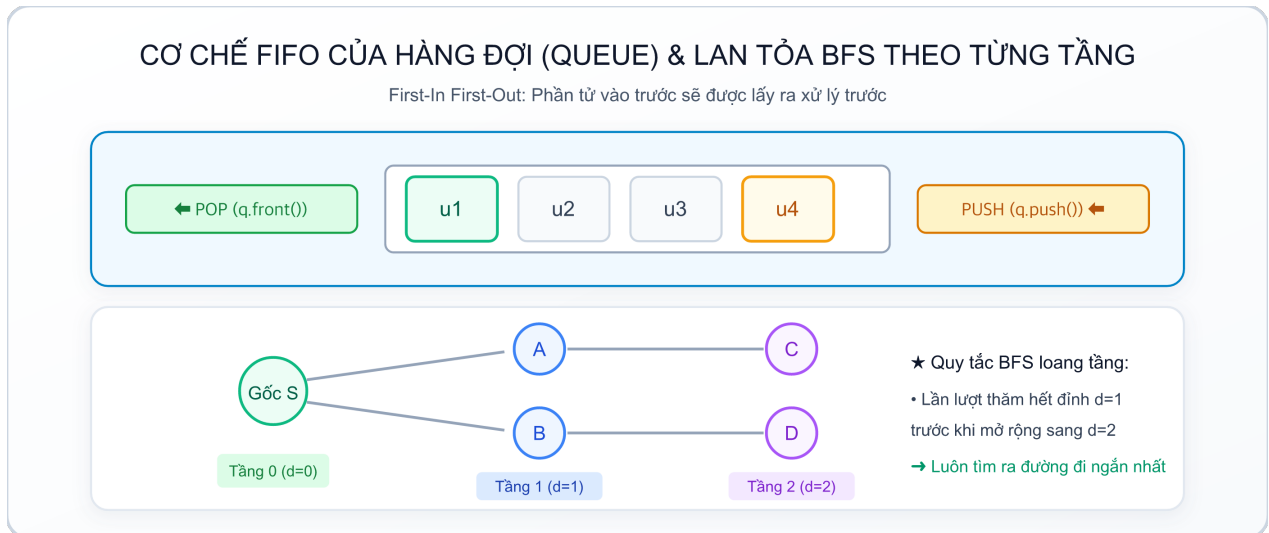
Bài 18: Cấu trúc dữ liệu hàng đợi (queue, deque) & Monotonic Deque

1. Bản chất cấu trúc dữ liệu hàng đợi (queue & deque)

1.1. Hàng đợi chuẩn (queue — FIFO)

Hàng đợi hoạt động theo nguyên lý **FIFO (First In, First Out — Vào trước, Ra trước)**:

- Phần tử được thêm vào ở đuôi (push), và được lấy ra ở đầu (pop).
- Đây là cấu trúc dữ liệu nền tảng của thuật toán Tìm kiếm theo chiều rộng (BFS).



1.2. Hàng đợi hai đầu (double-ended queue — Deque)

`std::deque` cho phép thực hiện thêm và xóa phần tử ở **CẢ HAI ĐẦU** với độ phức tạp tối ưu $\mathcal{O}(1)$:

- `push_front()`, `pop_front()`: Thao tác ở đầu hàng đợi.
- `push_back()`, `pop_back()`: Thao tác ở đuôi hàng đợi.

2. Kỹ thuật deque cửa sổ trượt min/max $\mathcal{O}(N)$ (Sliding Window Monotonic Deque)

2.1. Bản chất bài toán

- Cho mảng A gồm N phần tử và số K . Cần tìm giá trị nhỏ nhất (hoặc lớn nhất) trong mọi cửa sổ trượt độ dài K : $[i - K + 1 \dots i]$ ($K \leq i \leq N$).
- **Cách dùng Multiset / Priority Queue**: Mất $\mathcal{O}(N \log K)$.
- **Cách dùng Monotonic Deque**: Đạt thời gian tối ưu tuyệt đối $\mathcal{O}(N)$ tuyến tính!

HÀNG ĐỢI HAI ĐẦU (DEQUE) TRONG BÀI TOÁN SLIDING WINDOW MIN/MAX $O(N)$

`std::deque` (Thao Tác Cả Hai Đầu Trong $O(1)$)

`pop_front()` / `push_front()`

`pop_back()` / `push_back()`

Bất Biến Duy Trì Min Cửa Sổ Độ Dài K:

1. Loại bỏ phần tử lỗi thời hết hạn cửa sổ: $\text{index} < i - k + 1 \rightarrow \text{dq.pop_front}()$
2. Duy trì tính đơn điệu tăng: Loại bỏ mọi $x \geq A[i]$ ở đuôi $\rightarrow \text{dq.pop_back}()$
3. Phần tử nhỏ nhất hiện tại luôn nằm tại $\text{dq.front}() \rightarrow O(1)$

Monotonic Deque Của Số Trượt

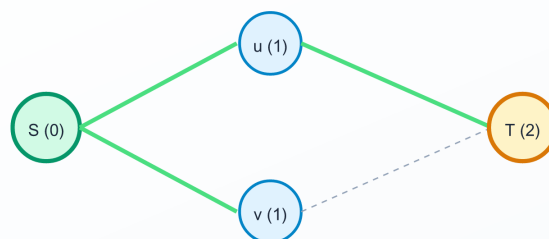
2.2. Bất biến 3 bước duy trì min cửa sổ

Tại mỗi vị trí i khi phần tử $A[i]$ bước vào:

1. **Loại bỏ phần tử hết hạn (Out of Window):** Nếu phần tử ở đầu $\text{dq.front}() < i - K + 1 \Rightarrow \text{dq.pop_front}()$.
2. **Duy trì tính đơn điệu tăng:** Trong khi $!\text{dq.empty}()$ và $A[\text{dq.back}()] \geq A[i] \Rightarrow \text{dq.pop_back}()$ (vì $A[i]$ vừa nhỏ hơn vừa tồn tại lâu hơn các phần tử ở đuôi).
3. **Thêm phần tử mới và lấy đáp án:** $\text{dq.push_back}(i)$. Khi $i \geq K - 1$, giá trị nhỏ nhất của cửa sổ hiện tại chính là $A[\text{dq.front}()]$.

3. Ứng dụng nền tảng: Tìm đường đi ngắn nhất bằng queue (BFS nhập môn)

TÌM ĐƯỜNG ĐI NGẮN NHẤT TRÊN ĐỒ THỊ KHÔNG TRỌNG SỐ BẰNG BFS



Định Lý BFS Cơ Bản

- Mỗi đỉnh thăm lần đầu tiên đảm bảo có khoảng cách min $\text{dist}[v]$
- $\text{dist}[v] = \text{dist}[u] + 1$
- $\text{parent}[v] = u$ (truy vết)
- Độ phức tạp: $O(V + E)$

Đường đi ngắn nhất bằng BFS

- Trên đồ thị không có trọng số (hoặc đồ thị lưới di chuyển 4 hướng có chi phí mỗi bước bằng 1), thuật toán BFS sử dụng Queue luôn đảm bảo:

Lần đầu tiên một đỉnh v được lấy ra khỏi Queue, khoảng cách $dist[v]$ chắc chắn là khoảng cách ngắn nhất từ đỉnh nguồn S .

4. Các bẫy lỗi lập trình kinh điển (bug traps)

1. Bẫy gọi `q.front()` khi Queue rỗng:

- Tương tự Stack, gọi `q.front()` hoặc `q.pop()` khi `q.empty() == true` gây Segmentation Fault.

2. Bẫy lưu giá trị thay vì lưu chỉ số trong Monotonic Deque:

- Nếu chỉ lưu giá trị $A[i]$, ta không thể kiểm tra xem phần tử ở đầu `dq.front()` đã vượt ra khỏi phạm vi cửa sổ $i - K + 1$ hay chưa.

- **Quy tắc bắt buộc:** Luôn lưu chỉ số i vào trong Deque!

3. Bẫy quên đánh dấu visited ngay khi push vào Queue trong BFS:

- Nếu chờ đến khi pop mới đánh dấu `visited[u] = true`, một đỉnh có thể bị đẩy vào Queue hàng chục lần từ các đỉnh lân cận $\Rightarrow$ Bùng nổ bộ nhớ và thời gian (TLE/MLE).
- **Quy tắc sống còn:** Bắt buộc gán `visited[v] = true` ngay tại thời điểm `q.push(v)`.

5. Mẫu cài đặt chuẩn thi đấu (competitive templates)

Mẫu 1: Min trên mọi cửa sổ trượt độ dài k bằng Monotonic Deque $\mathcal{O}(N)$

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, k;
    if (!(cin >> n >> k)) return 0;

    if (n <= 0 || k <= 0 || k > n) return 0;

    vector<long long> a(n);

    for (int i = 0; i < n; ++i) {
        cin >> a[i];
    }

    deque<int> dq; // Lưu chỉ số, duy trì  $A[dq[i]]$  tăng dần

    vector<long long> result;

    for (int i = 0; i < n; ++i) {
        // 1. Xóa phần tử quá hạn cửa sổ
        while (!dq.empty() && dq.front() < i - k + 1) {
            dq.pop_front();
        }

        // 2. Duy trì tính đơn điệu tăng
        while (!dq.empty() && a[dq.back()] >= a[i]) {
            dq.pop_back();
        }

        // 3. Thêm phần tử hiện tại
        dq.push_back(i);
        result.push_back(a[dq.front()]);
    }

    for (int i = 0; i < result.size(); ++i) {
        cout << result[i] << " ";
        if (i % 10 == 9) cout << "\n";
    }
    return 0;
}
```

```

        dq.push_back(i);

        // 4. Ghi nhận kết quả khi cửa sổ đủ kích thước k
        if (i >= k - 1) {
            result.push_back(a[dq.front()]);
        }
    }

    for (int i = 0; i < (int)result.size(); ++i) {
        cout << result[i] << (i + 1 == (int)result.size() ? "" : " ");
    }
    cout << "\n";

    return 0;
}

```

Mẫu 2: BFS tìm bước đi ngắn nhất từ 1 đến n

```

#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;
    if (!(cin >> n >> m)) return 0;

    if (n <= 0) return 0;

    vector<vector<int>> adj(n + 1);

    for (int i = 0; i < m; ++i) {
        int u, v;
        cin >> u >> v;

        adj[u].push_back(v);
        adj[v].push_back(u);
    }

    vector<int> dist(n + 1, -1);

    queue<int> q;

    // Khởi tạo gốc 1
    dist[1] = 0;
    q.push(1);

    while (!q.empty()) {
        int u = q.front();
        q.pop();

        for (int v : adj[u]) {
            if (dist[v] == -1) { // Chưa thăm
                dist[v] = dist[u] + 1;
                q.push(v); // Đánh dấu ngay khi push
            }
        }
    }

    cout << dist[n] << "\n";
    return 0;
}

```


Bài tập thực hành

Bài 76 [CPPB-QUE-01]: Cài Đặt Hàng Đợi Cơ Bản

Bối cảnh: Một hệ thống xếp hàng tự động tại quầy giao dịch ngân hàng hoạt động theo nguyên tắc vào trước ra trước (FIFO - First In First Out). Hệ thống tiếp nhận Q yêu cầu thuộc các dạng: thêm một khách hàng có số định danh x vào cuối hàng đợi, phục vụ khách hàng đang đứng ở đầu hàng (loại bỏ khỏi hàng), và kiểm tra số định danh của khách hàng đang đứng đầu hàng hiện tại.

Nhiệm vụ: Cho danh sách Q thao tác thuộc 3 loại: `1 x` (thêm x vào cuối hàng), `2` (loại bỏ phần tử đầu hàng nếu hàng không rỗng), `3` (in ra phần tử đầu hàng, nếu rỗng in ra `-1`). Hãy lập trình mô phỏng lại hàng đợi và in ra kết quả cho các thao tác loại 3.

Đầu vào (Input):

- Dòng 1: Số nguyên Q ($1 \leq Q \leq 50000$).
- Q dòng tiếp theo chứa các thao tác.

Đầu ra (Output):

- Kết quả các thao tác loại 3.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
5	
1 10	
1 20	10
3	20
2	
3	

Giải thích:

Với chuỗi thao tác: thêm 10, thêm 20, truy vấn đầu hàng $\rightarrow$ in ra 10; phục vụ đầu hàng (loại 10), truy vấn đầu hàng $\rightarrow$ in ra 20.

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq Q \leq 50000, 1 \leq x \leq 10^9$.

Bài 77 [CPPB-QUE-02]: Sinh Chuỗi Số Nhị Phân Bằng Queue

Bối cảnh: Một chip tạo mã nhị phân cần xuất ra liên tục danh sách biểu diễn nhị phân của các số tự nhiên từ 1 đến N theo thứ tự tăng dần (ví dụ: với $N = 3$, danh sách là `1`, `10`, `11`). Nhờ tính chất hàng đợi, mỗi khi lấy ra một chuỗi nhị phân S , ta có thể dễ dàng tạo ra hai chuỗi nhị phân kế tiếp bằng cách ghép thêm ký tự `0` và `1` vào cuối.

Nhiệm vụ: Cho số nguyên dương N . Hãy lập trình sinh và in ra chuỗi biểu diễn nhị phân của tất cả các số từ 1 đến N , các chuỗi cách nhau bởi khoảng trắng.

Đầu vào (Input):

- Một dòng chứa số nguyên N ($1 \leq N \leq 10^5$).

Đầu ra (Output):

- N chuỗi nhị phân.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
4	1 10 11 100

Giải thích:

Với $N = 5$, danh sách biểu diễn nhị phân của các số từ 1 đến 5 là: 1 (số 1), 10 (số 2), 11 (số 3), 100 (số 4), 101 (số 5).

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq N \leq 10^5$.

Bài 78 [CPPB-QUE-03]: BFS Tìm Bước Đi Ngắn Nhất Đồ Thị

Bối cảnh: Một mạng máy tính gồm N máy chủ được đánh số từ 1 đến N và M kênh kết nối hai chiều không trọng số. Một gói tin xuất phát từ máy chủ nguồn S muốn truyền đến máy chủ đích D . Mỗi lần truyền qua một kênh kết nối tốn đúng 1 bước nhảy (hop). Quản trị viên cần xác định số bước nhảy ít nhất để gói tin đến được đích.

Nhiệm vụ: Cho đồ thị N đỉnh M cạnh và hai đỉnh S, D . Hãy lập trình tìm số bước đi ngắn nhất từ S tới D . Nếu không thể đến được, in ra `-1`.

Đầu vào (Input):

- Dòng 1: Hai số N và M ($1 \leq N \leq 10^5, 0 \leq M \leq 2 \cdot 10^5$).
- M dòng tiếp theo: Mỗi dòng gồm 2 đỉnh u, v .

Đầu ra (Output):

- Số cạnh ít nhất hoặc `-1`.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
4 4 1 2 2 3 3 4 1 3	2

Giải thích:

Với mạng gồm 4 máy chủ kết nối theo chuỗi 1 - 2 - 3 - 4 và một kênh tắt nối trực tiếp 1 - 3:

Để đi từ 1 tới 4, gói tin đi qua kênh tắt $1 \times 0 3$ tốn 1 bước, sau đó từ $3 \times 0 4$ tốn 1 bước nữa. Tổng số bước đi ít nhất là 2.

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq N \leq 10^5, 0 \leq M \leq 2 \cdot 10^5$.

Bài 79 [CPPB-QUE-04]: Truy Vết Lộ Trình Ngắn Nhất Bằng BFS

Bối cảnh: Sau khi tính toán được số bước nhảy tối thiểu để truyền gói tin giữa hai máy chủ, trung tâm điều hành cần in ra chính xác danh sách thứ tự các máy chủ mà gói tin đã đi qua để cấu hình bảng định tuyến mạng.

Nhiệm vụ: Cho đồ thị vô hướng N đỉnh M cạnh và hai đỉnh S, D . Hãy lập trình tìm và in ra một đường đi ngắn nhất từ S tới D . Nếu không có đường đi, in ra `-1``.

Đầu vào (Input):

- Dòng 1: N, M ($1 \leq N \leq 10^5, 0 \leq M \leq 2 \cdot 10^5$).
- M dòng tiếp theo: Các cạnh vô hướng.

Đầu ra (Output):

- Dòng 1: Số đỉnh.
- Dòng 2: Lộ trình các đỉnh.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
5 5	
1 2	
2 3	3
3 5	1 4 5
1 4	
4 5	

Giải thích:

Với đồ thị có đường đi ngắn nhất từ 1 tới 4 là $1 \times 0 3 \times 0 4$:

Dòng 1 in ra 3 (số đỉnh trên đường đi).

Dòng 2 in ra 1 3 4.

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq N \leq 10^5, 0 \leq M \leq 2 \cdot 10^5$.

Bài 80 [CPPB-QUE-05]: Min Mọi Cửa Sổ Trượt Bằng Monotonic Deque $O(N)$

Bối cảnh: Một máy đo nhiệt độ lò phản ứng hạt nhân ghi nhận chuỗi N giá trị đo liên tiếp. Một cửa sổ quan sát kích thước cố định gồm K phép đo trượt dần từ đầu đến cuối dãy đo. Tại mỗi vị trí của

cửa sổ trượt, hệ thống cảnh báo an toàn cần xác định giá trị nhiệt độ thấp nhất bên trong cửa sổ đó với tốc độ xử lý tức thời.

Nhiệm vụ: Cho mảng A gồm N số nguyên và kích thước cửa sổ K. Hãy lập trình tìm giá trị nhỏ nhất trong mỗi cửa sổ trượt kích thước K khi di chuyển từ trái sang phải.

Đầu vào (Input):

- Dòng 1: Hai số N và K ($1 \leq K \leq N \leq 10^5$).
- Dòng 2: N số nguyên $A_1, A_2, \dots, A_N$ ($1 \leq A_i \leq 10^9$).

Đầu ra (Output):

- $N - K + 1$ số nguyên.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
8 3 1 3 -1 -3 5 3 6 7	-1 -3 -3 -3 3 3

Giải thích:

Với mảng $[1, 3, -1, -3, 5, 3, 6, 7]$ và cửa sổ $K = 3$:

- Cửa sổ 1 $[1, 3, -1] \rightarrow \min = -1$.
- Cửa sổ 2 $[3, -1, -3] \rightarrow \min = -3$.
- Cửa sổ 3 $[-1, -3, 5] \rightarrow \min = -3$.
- Cửa sổ 4 $[-3, 5, 3] \rightarrow \min = -3$.
- Cửa sổ 5 $[5, 3, 6] \rightarrow \min = 3$.
- Cửa sổ 6 $[3, 6, 7] \rightarrow \min = 3$.

Kết quả in ra: -1 -3 -3 -3 3 3.

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq K \leq N \leq 10^5, -10^9 \leq A_i \leq 10^9$.

Bài 81 [CPPB-QUE-06]: Max Mọi Cửa Sổ Trượt Bằng Monotonic Deque $O(N)$

Bối cảnh: Tương tự hệ thống giám sát lò phản ứng, để kịp thời phát hiện các đỉnh áp suất bất thường, hệ thống cần đồng thời theo dõi giá trị áp suất lớn nhất bên trong mỗi cửa sổ trượt gồm K mốc đo liên tiếp.

Nhiệm vụ: Cho mảng A gồm N số nguyên và kích thước cửa sổ K. Hãy lập trình tìm giá trị lớn nhất trong mỗi cửa sổ trượt kích thước K.

Đầu vào (Input):

- Dòng 1: Hai số N và K ($1 \leq K \leq N \leq 10^5$).
- Dòng 2: N số nguyên $A_1, A_2, \dots, A_N$ ($1 \leq A_i \leq 10^9$).

Đầu ra (Output):

- $N - K + 1$ số nguyên.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
8 3 1 3 -1 -3 5 3 6 7	3 3 5 5 6 7

Giải thích:

Với mảng $[1, 3, -1, -3, 5, 3, 6, 7]$ và $K = 3$:

- Cửa sổ $[1, 3, -1] \rightarrow \max = 3$.
- Cửa sổ $[3, -1, -3] \rightarrow \max = 3$.
- Cửa sổ $[-1, -3, 5] \rightarrow \max = 5$.
- Cửa sổ $[-3, 5, 3] \rightarrow \max = 5$.
- Cửa sổ $[5, 3, 6] \rightarrow \max = 6$.
- Cửa sổ $[3, 6, 7] \rightarrow \max = 7$.

Kết quả in ra: 3 3 5 5 6 7.

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq K \leq N \leq 10^5, -10^9 \leq A_i \leq 10^9$.

Bài 82 [CPPB-QUE-07]: Kiểm Tra Đồ Thị Hai Phía (Bipartite Graph)

Bối cảnh: Trong một hội thảo giao lưu quốc tế, có N đại biểu và M mối quan hệ quen biết lẫn nhau. Ban tổ chức muốn chia toàn bộ N đại biểu vào đúng 2 phòng thảo luận khác nhau sao cho không có hai người nào quen nhau lại ngồi chung trong cùng một phòng (tương đương bài toán tô màu đồ thị bằng 2 màu sao cho hai đỉnh kề nhau luôn có màu khác nhau).

Nhiệm vụ: Cho đồ thị vô hướng N đỉnh M cạnh. Hãy lập trình kiểm tra xem đồ thị có phải là đồ thị hai phía hay không. Nếu có in ra 'YES', ngược lại in ra 'NO'.

Đầu vào (Input):

- Dòng 1: N, M ($1 \leq N \leq 10^5, 0 \leq M \leq 2 \cdot 10^5$).
- M dòng tiếp theo: Các cạnh vô hướng.

Đầu ra (Output):

- YES hoặc NO.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
4 4 1 2 2 3 3 4 4 1	YES

Giải thích:

Với đồ thị là hình vuông gồm 4 đỉnh có các cạnh: 1-2, 2-3, 3-4, 4-1:

Ta có thể chia thành hai tập đỉnh độc lập: tập 1 gồm {1, 3} và tập 2 gồm {2, 4}. Không có hai đỉnh nào cùng tập có cạnh nối. Đồ thị là hai phía, kết quả in ra YES.

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq N \leq 10^5, 0 \leq M \leq 2 \cdot 10^5$.

Bài 83 [CPPB-QUE-08]: Biến Đổi Số Bước Nhỏ Nhất Từ A Sang B

Bối cảnh: Một trò chơi giải đố toán học bắt đầu với số nguyên dương A. Tại mỗi bước, người chơi có thể thực hiện một trong hai thao tác: nhân đôi số hiện tại ($x \rightarrow 2x$) hoặc trừ số hiện tại đi 1 đơn vị ($x \rightarrow x - 1$). Hãy tìm số thao tác ít nhất để biến đổi số A ban đầu thành đúng số mục tiêu B.

Nhiệm vụ: Cho hai số nguyên dương A và B. Hãy lập trình tìm số bước biến đổi ít nhất từ A thành B.

Đầu vào (Input):

- Một dòng chứa hai số nguyên A và B ($1 \leq A, B \leq 10^4$).

Đầu ra (Output):

- Số thao tác ít nhất.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
4 6	2

Giải thích:

Để biến đổi từ A = 4 sang B = 6:

- Bước 1: Trừ 1 đơn vị: $4 - 1 = 3$.

- Bước 2: Nhân đôi: $3 \times 2 = 6$.

Chỉ cần đúng 2 bước biến đổi, kết quả in ra là 2.

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq A, B \leq 10^4$.

Bài 84 [CPPB-QUE-09]: 0-1 BFS Tìm Đường Đi Ngắn Nhất Trọng Số 0/1

Bối cảnh: Một mạng lưới giao thông gồm N nút giao và M tuyến đường một chiều. Điểm đặc biệt là mỗi tuyến đường chỉ có trọng số chi phí hoặc là 0 đồng (đường công cộng miễn phí) hoặc là 1 đồng (đường cao tốc có thu phí tượng trưng). Cần tìm tổng chi phí ít nhất để di chuyển từ nút giao xuất phát S tới nút giao đích D .

Nhiệm vụ: Cho đồ thị có hướng N đỉnh M cạnh với trọng số mỗi cạnh thuộc $\{0, 1\}$. Hãy lập trình tìm khoảng cách ngắn nhất từ S tới D . Nếu không đến được, in ra `-1`.

Đầu vào (Input):

- Dòng 1: N, M ($1 \leq N \leq 10^5, 0 \leq M \leq 2 \cdot 10^5$).
- M dòng tiếp theo: Mỗi dòng gồm u, v, w ($w \in \{0, 1\}$).

Đầu ra (Output):

- Chi phí ngắn nhất.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
<pre> 4 4 1 2 1 2 3 0 3 4 1 1 4 1 </pre>	<pre> 1 </pre>

Giải thích:

Với đồ thị có đường đi từ 1 tới 3 qua cạnh trọng số 1, và từ 3 tới 4 qua cạnh trọng số 0: Tổng chi phí từ 1 tới 4 là $1 + 0 = 1$ đồng. Đây là chi phí tối thiểu.

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq N \leq 10^5, 0 \leq M \leq 2 \cdot 10^5$.

Bài 85 [CPPB-QUE-10]: Đoạn Con Tổng Lớn Nhất Độ Dài Tối Đa K

Bối cảnh: Một công ty dầu khí phân tích chỉ số lưu lượng khai thác của một giếng dầu qua N ngày liên tiếp tạo thành dãy số $A_1, A_2, \dots, A_N$. Ban giám đốc muốn chọn một đợt vận hành thử nghiệm gồm một chuỗi các ngày liên tiếp sao cho số ngày vận hành không vượt quá K ngày và tổng lưu lượng khai thác thu được là lớn nhất có thể.

Nhiệm vụ: Cho mảng A gồm N số nguyên và số nguyên K . Hãy lập trình tìm tổng lớn nhất của một đoạn con liên tiếp có độ dài tối đa là K .

Đầu vào (Input):

- Dòng 1: Hai số N và K ($1 \leq K \leq N \leq 10^5$).
- Dòng 2: N số nguyên $A_1, A_2, \dots, A_N$ ($-10^9 \leq A_i \leq 10^9$).

Đầu ra (Output):

- Tổng lớn nhất.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
5 3 -1 2 4 -3 5	6

Giải thích:

Với mảng $[-1, 2, 3, -2, 4]$ và độ dài tối đa $K = 2$:

Đoạn con liên tiếp có độ dài không quá 2 có tổng lớn nhất là đoạn $[2, 3]$ (độ dài 2) cho tổng là $2 + 3 = 5$ (hoặc đoạn $[4]$ có tổng 4).

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq K \leq N \leq 10^5, -10^9 \leq A_i \leq 10^9$.

Bài 86 [CPPB-QUE-11]: Trò Chơi Vòng Tròn Josephus

Bối cảnh: Trong một trò chơi dân gian, N bạn học sinh đứng thành một vòng tròn và được đánh số từ 1 đến N theo chiều kim đồng hồ. Bắt đầu đếm từ bạn số 1, cứ mỗi khi đếm đến người thứ K thì người đó sẽ phải rời khỏi vòng tròn. Quá trình đếm tiếp tục với người đứng kế tiếp cho đến khi chỉ còn lại đúng một người cuối cùng trụ lại.

Nhiệm vụ: Cho số lượng người N và bước đếm K . Hãy lập trình xác định số thứ tự của người cuối cùng còn lại trong vòng tròn.

Đầu vào (Input):

- Một dòng chứa hai số nguyên N và K ($1 \leq N, K \leq 10^4$).

Đầu ra (Output):

- Vị trí người sống sót.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
7 3	4

Giải thích:

Với $N = 7$ người và bước đếm $K = 3$:

Thứ tự các người bị loại lần lượt là: 3, 6, 2, 7, 5, 1. Người cuối cùng còn lại là người số 4.

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq N, K \leq 10^4$.

Bài 87 [CPPB-QUE-12]: Khoảng Cách Đến Trạm Cứu Hỏa Gần Nhất (Multi-Source BFS)

Bối cảnh: Một bản đồ đô thị gồm N khu dân cư và M tuyến đường hai chiều. Trong thành phố có K trạm cứu hỏa được đặt tại một số khu dân cư nhất định. Để đảm bảo an toàn phòng cháy chữa cháy, ban điều hành cứu hộ cần xác định khoảng cách ngắn nhất (số cạnh) từ mỗi khu dân cư tới trạm cứu hỏa gần nhất.

Nhiệm vụ: Cho bản đồ thành phố và danh sách vị trí các trạm cứu hỏa. Hãy lập trình tính khoảng cách ngắn nhất từ từng khu dân cư đến trạm cứu hỏa gần nhất.

Đầu vào (Input):

- Dòng 1: N, M, K ($1 \leq N \leq 10^5, 0 \leq M \leq 2 \cdot 10^5, 1 \leq K \leq N$).
- Dòng 2: Danh sách K trạm cứu hỏa.
- M dòng tiếp theo: Các con đường nối giữa hai nhà.

Đầu ra (Output):

- N số nguyên.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
<pre> 4 3 2 1 4 1 2 2 3 3 4 </pre>	<pre> 0 1 1 0 </pre>

Giải thích:

Với 4 khu dân cư nối liên tiếp 1 - 2 - 3 - 4 và trạm cứu hỏa đặt tại khu 2:

- Khu 1: cách trạm cứu hỏa 1 bước.
- Khu 2: có sẵn trạm cứu hỏa \to khoảng cách 0.
- Khu 3: cách trạm cứu hỏa 1 bước.
- Khu 4: cách trạm cứu hỏa 2 bước.

Kết quả in ra: 1 0 1 2.

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq N \leq 10^5, 0 \leq M \leq 2 \cdot 10^5$.

Bài 88 [CPPB-QUE-13]: Đoạn Con Dài Nhất Có Độ Chênh Lệch Max-Min $\leq C$

Bối cảnh: Một dây chuyền kiểm soát chất lượng linh kiện điện tử quét qua một dây gồm N sản phẩm với chỉ số dung sai $A_1, A_2, \dots, A_N$. Một lô sản phẩm đạt chuẩn ổn định là một đoạn con các sản phẩm liên tiếp sao cho độ chênh lệch giữa sản phẩm có dung sai lớn nhất và sản phẩm có dung sai nhỏ nhất trong lô không vượt quá một ngưỡng cho phép C .

Nhiệm vụ: Cho mảng A gồm N số nguyên và ngưỡng chênh lệch C. Hãy lập trình tìm độ dài lớn nhất của một đoạn con liên tiếp có $\max - \min \leq C$.

Đầu vào (Input):

- Dòng 1: Hai số N và C ($1 \leq N \leq 10^5, 0 \leq C \leq 10^9$).
- Dòng 2: N số nguyên $A_1, A_2, \dots, A_N$ ($1 \leq A_i \leq 10^9$).

Đầu ra (Output):

- Độ dài lớn nhất.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
6 2 4 2 2 2 4 4	6

Giải thích:

Với mảng [8, 2, 4, 7] và ngưỡng C = 4:

Đoạn con [2, 4] có $\max = 4$, $\min = 2$, độ chênh lệch là $4 - 2 = 2 \leq 4$ có độ dài 2.

Đoạn con [4, 7] có $\max = 7$, $\min = 4$, độ chênh lệch là $7 - 4 = 3 \leq 4$ có độ dài 2.

Độ dài lớn nhất đạt được là 2.

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq N \leq 10^5, 0 \leq C, A_i \leq 10^9$.

Bài 89 [CPPB-QUE-14]: Chọn Đoạn Tối Đa Không Quá K Phần Tử Liên Kề

Bối cảnh: Một tuyến phố đi bộ có N vị trí treo biển quảng cáo, vị trí thứ i đem lại doanh thu A_i . Để đảm bảo không gian cảnh quan xanh cho tuyến phố, quy định đô thị yêu cầu không được phép treo biển quảng cáo tại quá K vị trí đứng liền sát nhau.

Nhiệm vụ: Cho danh sách doanh thu của N vị trí và giới hạn K. Hãy lập trình chọn các vị trí treo biển sao cho không có quá K vị trí liền kề được chọn và tổng doanh thu thu về là lớn nhất.

Đầu vào (Input):

- Dòng 1: Hai số N và K ($1 \leq K \leq N \leq 10^5$).
- Dòng 2: N số nguyên dương $A_1, A_2, \dots, A_N$ ($1 \leq A_i \leq 10^9$).

Đầu ra (Output):

- Tổng lớn nhất.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
5 2 1 2 3 4 5	12

Giải thích:

Với 5 vị trí có doanh thu [1, 2, 3, 4, 5] và giới hạn $K = 2$ (không được chọn quá 2 vị trí liên tiếp): Phương án tối ưu là bỏ chọn vị trí thứ 3 (doanh thu 3), giữ lại các vị trí 1, 2, 4, 5. Các cụm chọn gồm [1, 2] (độ dài 2) và [4, 5] (độ dài 2), mang lại tổng doanh thu tối đa là $1 + 2 + 4 + 5 = 12$.

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq K \leq N \leq 10^5, 1 \leq A_i \leq 10^9$.

Bài 90 [CPPB-QUE-15]: Đua Xe Mê Cung Đổi Hướng Ít Nhất (0-1 BFS State)

Bối cảnh: Trong một giải đua xe robot trong mê cung lưới ô vuông $N \times M$, robot cần di chuyển từ ô xuất phát S tới ô đích D. Mỗi lần robot đi thẳng theo hướng đang di chuyển thì hoàn toàn miễn phí (chi phí 0), nhưng mỗi khi robot phải rẽ lái đổi sang một trong các hướng vuông góc thì bánh lái sẽ tiêu hao 1 đơn vị năng lượng.

Nhiệm vụ: Cho bản đồ mê cung $N \times M$ và vị trí S, D. Hãy lập trình tìm số lần đổi hướng ít nhất để robot đi từ S tới D. Nếu không có đường đi, in ra -1.

Đầu vào (Input):

- Dòng 1: Hai số N và M ($1 \leq N, M \leq 500$).
- N dòng tiếp theo biểu diễn mê cung.

Đầu ra (Output):

- Số lần đổi hướng ít nhất hoặc -1.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
<pre> 3 3 S.. .#. ..E </pre>	<pre> 1 </pre>

Giải thích:

Với mê cung 3×3 không vật cản từ góc (1,1) tới (3,3):

Robot chỉ cần đi thẳng hết hàng 1 sang phải, sau đó rẽ lái 1 lần duy nhất để đi thẳng xuống dưới tới đích. Số lần đổi hướng ít nhất là 1.

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq N, M \leq 500$.

CHƯƠNG 07: ĐỒ THỊ & CÂY TRUY VẤN ĐOẠN

Bài 19: Lý thuyết đồ thị cơ bản: Duyệt BFS & DFS

1. Bản chất đồ thị & các phương pháp biểu diễn

Đồ thị $G = (V, E)$ là cấu trúc toán học biểu diễn tập hợp các đỉnh (Vertices — V) và các cạnh nối giữa chúng (Edges — E). Đồ thị có thể là vô hướng (Undirected) hoặc có hướng (Directed), có trọng số hoặc không có trọng số.

BIỂU DIỄN ĐỒ THỊ: MA TRẬN KỀ VS DANH SÁCH KỀ (ADJACENCY LIST)

Mã Trận Kề: $\text{adj}[N][N]$

```
int adj[1005][1005];
```

- Kiểm tra cạnh (u, v) : $O(1)$
- Bộ nhớ: $O(N^2) \rightarrow$ Sập bộ nhớ khi $N=10^5$
- Duyệt đỉnh kề: $O(N)$

$\rightarrow$ Chỉ dùng khi $N \leq 1000$ hoặc đồ thị dày

Danh Sách Kề: $\text{vector}<\text{int}> \text{adj}[N]$

```
vector<int> adj[100005];
```

- Bộ nhớ: $O(N + M)$ tối ưu tuyệt đối
- Duyệt đỉnh kề: $O(\deg(u))$ cực nhanh
- Thêm cạnh: $\text{adj}[u].\text{push_back}(v)$ trong $O(1)$

$\rightarrow$ Chuẩn mực thi đấu Olympic cho $N, M \leq 10^5$

Biểu diễn Đồ thị: Ma trận kề vs Danh sách kề

1.1. Ma trận kề (adjacency matrix)

- Mảng 2 chiều $\text{int adj}[N][N]$: $\text{adj}[u][v] = 1$ nếu có cạnh nối giữa u và v .
- **Ưu điểm:** Kiểm tra cạnh (u, v) trong $O(1)$.
- **Nhược điểm:** Tốn $O(N^2)$ bộ nhớ. Khi $N = 10^5$, ma trận cần 40GB RAM $\Rightarrow$ Sập bộ nhớ ngay lập tức (MLE). Chỉ dùng khi $N \leq 1000$.

1.2. Danh sách kề (adjacency list — Chuẩn mực thi đấu)

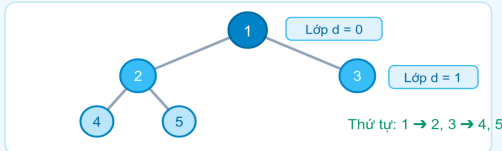
- Sử dụng mảng các vector $\text{vector}<\text{int}> \text{adj}[N + 1]$: $\text{adj}[u]$ chứa toàn bộ các đỉnh kề trực tiếp với u .
- **Bộ nhớ:** Đúng $O(V + E)$, cực kỳ tiết kiệm và tối ưu cho đồ thị thưa trong lập trình thi đấu ($N, M \leq 2 \cdot 10^5$).
- **Duyệt đỉnh kề:** for ($\text{int } v : \text{adj}[u]$) tốn thời gian tỷ lệ thuận với bậc của đỉnh $O(\deg(u))$.

2. Hai chiến lược duyệt đồ thị kinh điển: BFS vs DFS

So Sánh Chiến Lược Duyệt Đồ Thị: BFS vs DFS

Duyệt theo chiều rộng (theo từng lớp bán kính sóng nước) so với duyệt theo chiều sâu (đi kích đường một nhánh)

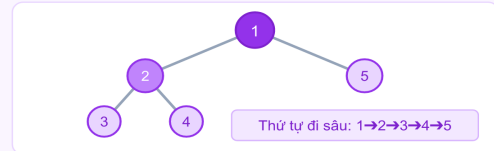
1. BFS: Duyệt Chiều Rộng (Hàng Đợi - Queue)



- Cấu trúc: `std::queue<int> q;`
- Điểm mạnh: Tìm đường đi ngắn nhất (trọng số bằng nhau)
- Độ phức tạp: $O(V + E)$ thời gian và $O(V)$ bộ nhớ

Ứng dụng: Loang lưới, tìm số bước ít nhất, cây BFS.

2. DFS: Duyệt Chiều Sâu (Đệ Quy / Ngăn Xếp)



- Cấu trúc: Hàm đệ quy `dfs(u)` hoặc `std::stack`
- Điểm mạnh: Tìm thành phần liên thông, chu trình, khớp/cầu
- Độ phức tạp: $O(V + E)$ thời gian và $O(V)$ stack gọi hàm

Ứng dụng: Đếm miền, Topo sort, Euler, Quay lui đồ thị.

So sánh BFS vs DFS

2.1. Tìm kiếm theo chiều rộng (breadth-first search — BFS)

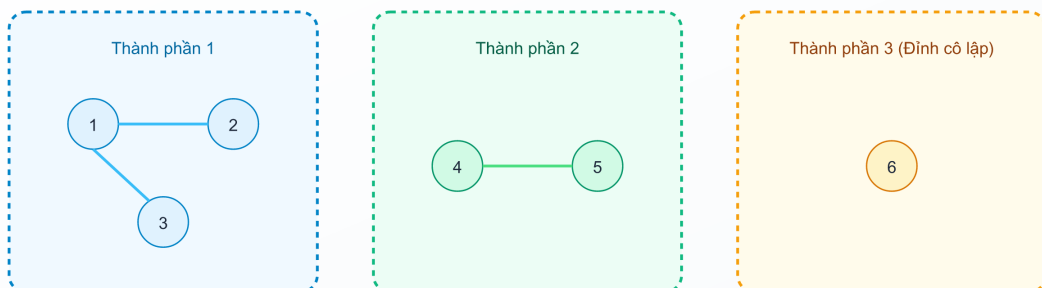
- Sử dụng **Hàng đợi (Queue)**, lan tỏa theo từng tầng bán kính $d = 0, 1, 2, \dots$ tính từ đỉnh nguồn S .
- **Đặc tính vàng:** Tìm đường đi có ít cạnh nhất (ngắn nhất) trên đồ thị không trọng số.

2.2. Tìm kiếm theo chiều sâu (depth-first search — DFS)

- Sử dụng **Đệ quy (hoặc Stack)**, đi sâu hết mức có thể trên một nhánh cho đến khi gặp ngõ cụt thì quay lui (Backtracking).
- **Đặc tính vàng:** Cực kỳ hiệu quả để đếm thành phần liên thông, phát hiện chu trình, kiểm tra tính liên thông, định hướng cây DFS.

3. Ứng dụng: Đếm số thành phần liên thông & kiểm tra chu trình

ĐẾM SỐ THÀNH PHẦN LIÊN THÔNG TRÊN ĐỒ THỊ VÔ HƯỚNG



Duyệt mọi đỉnh $i = 1..N$: Nếu `!visited[i]` → `count++`, gọi `DFS(i)` hoặc `BFS(i)` loang hết thành phần.

Đếm số thành phần liên thông

- **Thuật toán đếm thành phần liên thông:** Duyệt qua mọi đỉnh $i \in [1, N]$. Nếu đỉnh i chưa được thăm (!visited[i]), tăng biến đếm số thành phần liên thông components++ và gọi DFS(i) để loang thăm toàn bộ các đỉnh thuộc cùng thành phần.
- **Phát hiện chu trình trên đồ thị vô hướng bằng DFS:** Khi duyệt từ u sang đỉnh kề v , nếu v đã được thăm (visited[v] == true) và $v \neq \text{parent}[u]$, ta khẳng định đồ thị **CÓ CHU TRÌNH!**

4. Các bẫy lỗi lập trình kinh điển (bug traps)

1. **Bẫy thêm cạnh đồ thị vô hướng chỉ thêm 1 chiều:**
 - Với đồ thị vô hướng, cạnh giữa u và v phải thêm cả 2 chiều: adj[u].push_back(v); adj[v].push_back(u);. Quên thêm chiều thứ hai làm đồ thị biến thành đồ thị có hướng sai hoàn toàn.
2. **Bẫy tràn ngăn xếp đệ quy (Stack Overflow) khi DFS đồ thị sâu:**
 - Nếu đồ thị là một đường thẳng $N = 2 \cdot 10^5$ đỉnh, hàm đệ quy DFS sẽ gọi sâu $2 \cdot 10^5$ tầng, vượt quá giới hạn ngăn xếp (Call Stack) của một số hệ điều hành và gây Runtime Error.
3. **Bẫy cạnh lặp và khuyên (Multiple Edges & Self-loops):**
 - Đề bài có thể cho nhiều cạnh nối giữa cùng một cặp đỉnh (u, v) hoặc cạnh tự nối $u \rightarrow u$. Cần kiểm tra hoặc xử lý cẩn thận nếu thuật toán yêu cầu tính toán bậc hoặc trọng số tối thiểu.

5. Mẫu cài đặt chuẩn thi đấu (competitive templates)

Mẫu 1: DFS đếm số thành phần liên thông và tìm kích thước từng thành phần

⚠ Lưu ý về Stack Overflow: Hàm DFS đệ quy dưới đây có thể gây tràn ngăn xếp hệ thống (Segmentation Fault) khi đồ thị có dạng đường thẳng $N = 2 \times 10^5$ đỉnh (độ sâu đệ quy lên tới N tầng). Trong thi đấu thực tế, nên dùng **DFS bằng std::stack tường minh** hoặc thiết lập ulimit -s unlimited (Linux) trước khi chạy. Mẫu đệ quy được giữ lại ở đây vì tính trực quan su phạm.

```
#include <bits/stdc++.h>
using namespace std;

int n, m;
vector<vector<int>>> adj;

vector<bool> visited;

int dfs(int u) {
    visited[u] = true;
    int comp_size = 1;
    for (int v : adj[u]) {
        if (!visited[v]) {
            comp_size += dfs(v);
        }
    }
    return comp_size;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    if (!(cin >> n >> m)) return 0;
```

```

    if (n <= 0) return 0;

    adj.assign(n + 1, vector<int>());
    visited.assign(n + 1, false);

    for (int i = 0; i < m; ++i) {
        int u, v;
        cin >> u >> v;

        adj[u].push_back(v);
        adj[v].push_back(u);
    }

    int component_count = 0;
    vector<int> component_sizes;

    for (int i = 1; i <= n; ++i) {
        if (!visited[i]) {
            component_count++;
            int sz = dfs(i);
            component_sizes.push_back(sz);
        }
    }

    cout << component_count << "\n";
    for (int i = 0; i < (int)component_sizes.size(); ++i) {
        cout << component_sizes[i] << (i + 1 == (int)component_sizes.size() ? "" : " ");
    }
    cout << "\n";

    return 0;
}

```

Mẫu 2: Phát hiện chu trình trên đồ thị vô hướng bằng DFS

```

#include <bits/stdc++.h>
using namespace std;

int n, m;
vector<vector<int>> adj;

vector<bool> visited;

bool has_cycle = false;

void dfs_cycle(int u, int p) {
    visited[u] = true;
    for (int v : adj[u]) {
        if (!visited[v]) {
            dfs_cycle(v, u);
        } else if (v != p) {
            has_cycle = true; // Gặp lại đỉnh đã thăm khác cha -> Chu trình!
        }
    }
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    if (!(cin >> n >> m)) return 0;
}

```

```

if (n <= 0) return 0;

adj.assign(n + 1, vector<int>());
visited.assign(n + 1, false);

for (int i = 0; i < m; ++i) {
    int u, v;
    cin >> u >> v;

    adj[u].push_back(v);
    adj[v].push_back(u);
}

for (int i = 1; i <= n; ++i) {
    if (!visited[i]) {
        dfs_cycle(i, 0);
    }
}

if (has_cycle) {
    cout << "YES\n";
} else {
    cout << "NO\n";
}

return 0;
}

```

Bài tập thực hành

Bài 91 [CPPB-GRA-01]: Chuyển Danh Sách Cạnh Sang Danh Sách Kề

Bối cảnh: Một đồ thị mạng lưới giao thông gồm N nút giao và M con đường hai chiều ban đầu được lưu trữ dưới dạng danh sách các cặp cạnh (u, v) . Để thuận tiện cho việc lập trình các thuật toán tìm kiếm và duyệt đồ thị, kỹ sư cầu đường cần chuyển đổi dữ liệu mạng lưới sang cấu trúc danh sách kề: Với mỗi nút giao, liệt kê tất cả các nút giao có đường nối trực tiếp với nó theo thứ tự số hiệu tăng dần.

Nhiệm vụ: Cho đồ thị vô hướng N đỉnh M cạnh. Hãy lập trình chuyển đổi sang biểu diễn danh sách kề và in ra các đỉnh kề của từng đỉnh theo thứ tự tăng dần.

Đầu vào (Input):

- Dòng 1: N, M ($1 \leq N \leq 10000, 0 \leq M \leq 20000$).
- M dòng tiếp theo: Mỗi dòng gồm 2 đỉnh u, v .

Đầu ra (Output):

- N dòng biểu diễn danh sách kề.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
3 2	2 2 3
1 2	1 1
1 3	1 1

Giải thích:

Với đồ thị 4 đỉnh và các cạnh (1, 2), (1, 3), (2, 4):

- Đỉnh 1 kề với các đỉnh: 2, 3.
- Đỉnh 2 kề với các đỉnh: 1, 4.
- Đỉnh 3 kề với đỉnh: 1.
- Đỉnh 4 kề với đỉnh: 2.

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq N \leq 10000, 0 \leq M \leq 20000$.

Bài 92 [CPPB-GRA-02]: Duyệt Đồ Thị Theo Chiều Sâu (DFS Traversal)

Bối cảnh: Một robot thám hiểm cần khám phá toàn bộ các gian phòng trong một hang động ngầm gồm N gian phòng và M lối đi hai chiều. Robot áp dụng chiến lược tìm kiếm theo chiều sâu: Từ gian phòng hiện tại, luôn ưu tiên tiến sâu vào một gian phòng chưa từng được khám phá kề cạnh có số hiệu nhỏ nhất; khi không thể đi tiếp thì quay lui lại gian phòng trước đó.

Nhiệm vụ: Cho đồ thị vô hướng N đỉnh M cạnh và đỉnh xuất phát S. Hãy lập trình in ra thứ tự các đỉnh được robot ghé thăm trong quá trình duyệt theo chiều sâu (khi có nhiều lựa chọn, luôn ưu tiên đỉnh có số hiệu nhỏ hơn).

Đầu vào (Input):

- Dòng 1: N, M, S ($1 \leq N \leq 10^5, 0 \leq M \leq 2 \cdot 10^5, 1 \leq S \leq N$).
- M dòng tiếp theo: Các cạnh vô hướng.

Đầu ra (Output):

- Thứ tự các đỉnh được thăm.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
4 3 1	
1 2	
2 3	1 2 3 4
1 4	

Giải thích:

Với đồ thị 4 đỉnh gồm các cạnh (1, 2), (1, 3), (2, 4) xuất phát từ đỉnh 1:

- Từ đỉnh 1 thăm đỉnh 2 (nhỏ hơn 3).

- Từ đỉnh 2 tiếp tục thăm đỉnh 4.
 - Đỉnh 4 không còn đường mới, quay lui về 2, rồi về 1, từ 1 thăm tiếp đỉnh 3.
- Thứ tự ghé thăm là: 1 2 4 3.

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq N \leq 10^5, 0 \leq M \leq 2 \cdot 10^5$.

Bài 93 [CPPB-GRA-03]: Duyệt Đồ Thị Theo Chiều Rộng (BFS Traversal)

Bối cảnh: Một thông điệp cảnh báo khẩn cấp cần được lan truyền qua một mạng xã hội gồm N người dùng và M mối quan hệ bạn bè. Xuất phát từ người dùng S , thông điệp sẽ được gửi đồng thời tới tất cả những người bạn trực tiếp của S trước (tầng 1), sau đó mới tiếp tục lan truyền tới bạn của bạn (tầng 2) theo nguyên tắc loang theo chiều rộng (ưu tiên người có số hiệu nhỏ hơn).

Nhiệm vụ: Cho đồ thị vô hướng N đỉnh M cạnh và đỉnh xuất phát S . Hãy lập trình in ra thứ tự các đỉnh nhận được thông điệp theo chiến lược duyệt theo chiều rộng.

Đầu vào (Input):

- Dòng 1: N, M, S ($1 \leq N \leq 10^5, 0 \leq M \leq 2 \cdot 10^5$).
- M dòng tiếp theo: Các cạnh vô hướng.

Đầu ra (Output):

- Thứ tự các đỉnh được thăm.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
4 3 1 1 2 1 3 2 4	1 2 3 4

Giải thích:

Với đồ thị có các cạnh $(1, 2), (1, 3), (2, 4)$ xuất phát từ đỉnh 1:

- Tầng 1 (kề trực tiếp với 1): thăm đỉnh 2 và đỉnh 3 theo thứ tự tăng dần.
- Tầng 2: từ đỉnh 2 thăm tiếp đỉnh 4.

Thứ tự ghé thăm là: 1 2 3 4.

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq N \leq 10^5, 0 \leq M \leq 2 \cdot 10^5$.

Bài 94 [CPPB-GRA-04]: Đếm Số Thành Phần Liên Thông

Bối cảnh: Một quần đảo gồm N hòn đảo và M cây cầu hai chiều nối giữa một số cặp đảo. Hai hòn đảo thuộc cùng một cụm đảo liên thông nếu cư dân có thể đi lại giữa chúng qua các cây cầu (trực

tiếp hoặc gián tiếp). Ban quản lý du lịch cần xác định xem toàn bộ quần đảo đang bị chia cắt thành bao nhiêu cụm đảo biệt lập.

Nhiệm vụ: Cho đồ thị vô hướng N đỉnh M cạnh. Hãy lập trình đếm số lượng thành phần liên thông của đồ thị.

Đầu vào (Input):

- Dòng 1: N, M ($1 \leq N \leq 10^5, 0 \leq M \leq 2 \cdot 10^5$).
- M dòng tiếp theo: Các cạnh vô hướng.

Đầu ra (Output):

- Số lượng thành phần liên thông.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
5 3 1 2 2 3 4 5	2

Giải thích:

Với 5 đỉnh và các cạnh (1, 2), (3, 4):

- Cụm 1 gồm các đỉnh {1, 2}.
- Cụm 2 gồm các đỉnh {3, 4}.
- Cụm 3 gồm đỉnh cô lập {5}.

Có tất cả 3 thành phần liên thông độc lập, kết quả in ra là 3.

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq N \leq 10^5, 0 \leq M \leq 2 \cdot 10^5$.

Bài 95 [CPPB-GRA-05]: Kích Thước Thành Phần Liên Thông Lớn Nhất

Bối cảnh: Vẫn tại quần đảo gồm N hòn đảo và M cây cầu, ban quy hoạch kinh tế muốn tìm cụm đảo liên thông phát triển sầm uất nhất (chứa số lượng hòn đảo nhiều nhất) để đầu tư xây dựng trung tâm logistics tập trung.

Nhiệm vụ: Cho đồ thị vô hướng N đỉnh M cạnh. Hãy lập trình tìm số lượng đỉnh thuộc về thành phần liên thông có kích thước lớn nhất.

Đầu vào (Input):

- Dòng 1: N, M ($1 \leq N \leq 10^5, 0 \leq M \leq 2 \cdot 10^5$).
- M dòng tiếp theo: Các cạnh vô hướng.

Đầu ra (Output):

- Kích thước lớn nhất.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
5 3 1 2 2 3 4 5	3

Giải thích:

Với 5 đỉnh và các cạnh (1, 2), (2, 3), (4, 5):

- Cụm 1 gồm {1, 2, 3} có 3 hòn đảo.

- Cụm 2 gồm {4, 5} có 2 hòn đảo.

Kích thước của thành phần liên thông lớn nhất là 3.

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq N \leq 10^5, 0 \leq M \leq 2 \cdot 10^5$.

Bài 96 [CPPB-GRA-06]: Kiểm Tra Đường Đi Giữa Hai Đỉnh

Bối cảnh: Trong hệ thống mạng lưới điện quốc gia gồm N trạm biến áp và M đường dây tải điện hai chiều, trung tâm điều độ cần kiểm tra nhanh xem liệu có tồn tại tuyến đường dây kết nối (trực tiếp hoặc qua các trạm trung gian) giữa hai trạm biến áp S và D hay không.

Nhiệm vụ: Cho đồ thị vô hướng N đỉnh M cạnh và hai đỉnh S, D . Hãy lập trình kiểm tra xem có tồn tại đường đi giữa S và D hay không. Nếu có in ra 'YES', ngược lại in ra 'NO'.

Đầu vào (Input):

- Dòng 1: N, M, S, T ($1 \leq N \leq 10^5, 0 \leq M \leq 2 \cdot 10^5, 1 \leq S, T \leq N$).
- M dòng tiếp theo: Các cạnh vô hướng.

Đầu ra (Output):

- YES hoặc NO.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
4 2 1 4 1 2 2 3	NO

Giải thích:

Với đồ thị có các cạnh (1, 2), (2, 3) và đỉnh 4 cô lập:

- Kiểm tra giữa 1 và 3: Tồn tại đường đi $1 \times o 2 \times o 3$, in ra YES.

- Nếu kiểm tra giữa 1 và 4: Không có đường đi, in ra NO.

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq N \leq 10^5, 0 \leq M \leq 2 \cdot 10^5$.

Bài 97 [CPPB-GRA-07]: Phát Hiện Chu Trình Trên Đồ Thị Vô Hướng

Bối cảnh: Một mạng lưới đường ống cấp nước đô thị cần được kiểm tra thiết kế. Nếu trong mạng lưới xuất hiện một chu trình khép kín (một vòng tròn đường ống quay về điểm xuất phát mà không đi lặp lại cạnh nào), áp lực nước có thể bị nhiễu loạn dòng chảy. Kỹ sư thủy lợi cần kiểm tra xem bản vẽ mạng lưới có chứa chu trình hay không.

Nhiệm vụ: Cho đồ thị vô hướng N đỉnh M cạnh. Hãy lập trình kiểm tra xem đồ thị có chứa chu trình hay không. Nếu có in ra `YES`, ngược lại in ra `NO`.

Đầu vào (Input):

- Dòng 1: N, M ($1 \leq N \leq 10^5, 0 \leq M \leq 2 \cdot 10^5$).
- M dòng tiếp theo: Các cạnh vô hướng.

Đầu ra (Output):

- YES hoặc NO.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
3 3 1 2 2 3 3 1	YES

Giải thích:

Với đồ thị gồm 3 đỉnh có các cạnh (1, 2), (2, 3), (3, 1):

Ba đỉnh này tạo thành một tam giác khép kín 1 - 2 - 3 - 1, là một chu trình hoàn chỉnh. Kết quả in ra là YES.

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq N \leq 10^5, 0 \leq M \leq 2 \cdot 10^5$.

Bài 98 [CPPB-GRA-08]: Đường Đi Ngắn Nhất Trên Đồ Thị Không Trọng Số

Bối cảnh: Một đoàn tàu hỏa vận chuyển hàng hóa di chuyển trên mạng lưới đường ray gồm N nhà ga và M chặng ray hai chiều nối giữa các ga kề nhau. Đoàn tàu cần đi từ ga xuất phát S đến ga đích D . Mỗi chặng ray mất đúng 1 giờ chạy tàu. Hãy tìm thời gian ít nhất (tổng số chặng ray ít nhất) để tàu đến được ga đích.

Nhiệm vụ: Cho đồ thị vô hướng N đỉnh M cạnh và hai đỉnh S, D . Hãy lập trình tìm số cạnh trên đường đi ngắn nhất từ S tới D . Nếu không có đường đi, in ra `-1`.

Đầu vào (Input):

- Dòng 1: N, M ($1 \leq N \leq 10^5, 0 \leq M \leq 2 \cdot 10^5$).

- M dòng tiếp theo: Các cạnh vô hướng.

Đầu ra (Output):

- Khoảng cách ngắn nhất.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
4 4 1 2 2 3 3 4 1 3	2

Giải thích:

Với mạng đường ray có các chặng (1, 2), (2, 3), (1, 4), (4, 3) từ ga 1 tới ga 3:

Có hai lộ trình cùng đạt 2 chặng là $1 \times 0 2 \times 0 3$ hoặc $1 \times 0 4 \times 0 3$. Thời gian ít nhất là 2 chặng.

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq N \leq 10^5, 0 \leq M \leq 2 \cdot 10^5$.

Bài 99 [CPPB-GRA-09]: Kiểm Tra Đồ Thị Cây (Tree Verification)

Bối cảnh: Trong cấu trúc liên kết mạng máy tính, cấu trúc dạng cây (Tree) là mô hình mạng tối ưu nhất vì nó đảm bảo tất cả N máy chủ đều liên thông với nhau mà hoàn toàn không có chu trình dư thừa (số lượng kênh kết nối đúng bằng $N - 1$). Kỹ sư hệ thống cần xác thực xem một mạng lưới cho trước có thỏa mãn đầy đủ các điều kiện của một đồ thị cây hay không.

Nhiệm vụ: Cho đồ thị vô hướng N đỉnh M cạnh. Hãy lập trình kiểm tra xem đồ thị có phải là một cây hợp lệ hay không. Nếu đúng in ra 'YES', ngược lại in ra 'NO'.

Đầu vào (Input):

- Dòng 1: N, M ($1 \leq N \leq 10^5, 0 \leq M \leq 2 \cdot 10^5$).
- M dòng tiếp theo: Các cạnh vô hướng.

Đầu ra (Output):

- YES hoặc NO.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
4 3 1 2 2 3 3 4	YES

Giải thích:

Với đồ thị có $N = 4$ đỉnh và $M = 3$ cạnh: $(1, 2), (1, 3), (1, 4)$:

Đồ thị liên thông toàn bộ và không chứa bất kỳ chu trình nào, số cạnh đúng bằng $4 - 1 = 3$. Đồ thị là một cây, kết quả in ra là YES.

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq N \leq 10^5, 0 \leq M \leq 2 \cdot 10^5$.

Bài 100 [CPPB-GRA-10]: Sắp Xếp Tô-pô (Topological Sort)

Bối cảnh: Một chương trình đào tạo kỹ sư công nghệ thông tin gồm N môn học được đánh số từ 1 đến N . Giữa các môn học có M điều kiện môn tiên quyết dạng $u \times o v$ (nghĩa là sinh viên bắt buộc phải hoàn thành môn học u trước khi được phép đăng ký môn học v). Ban đào tạo cần lập ra một lộ trình học tập hợp lệ thỏa mãn tất cả các điều kiện tiên quyết.

Nhiệm vụ: Cho đồ thị có hướng N đỉnh M cạnh không có chu trình. Hãy lập trình tìm một thứ tự sắp xếp tô-pô của các môn học.

Đầu vào (Input):

- Dòng 1: N, M ($1 \leq N \leq 10^5, 0 \leq M \leq 2 \cdot 10^5$).
- M dòng tiếp theo: Mỗi dòng gồm cạnh có hướng $u \rightarrow v$.

Đầu ra (Output):

- Thứ tự tô-pô hoặc -1.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
4 3 1 2 2 3 1 3	1 4 2 3

Giải thích:

Với 3 môn học và các điều kiện: môn 1 trước môn 2 ($1 \times o 2$), môn 2 trước môn 3 ($2 \times o 3$):

Lộ trình học tập bắt buộc duy nhất là: 1 2 3.

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq N \leq 10^5, 0 \leq M \leq 2 \cdot 10^5$.

Bài 101 [CPPB-GRA-11]: Chu Trình Ngắn Nhất Trên Đồ Thị (Girth)

Bối cảnh: Trong nghiên cứu cấu trúc phân tử hóa học của các hợp chất vòng hữu cơ, các nhà hóa học cần xác định chu trình vòng có kích thước nhỏ nhất (chu vi nhỏ nhất - Girth) xuất hiện trong đồ thị liên kết nguyên tử.

Nhiệm vụ: Cho đồ thị vô hướng N đỉnh M cạnh không có khuyên và không có cạnh bội. Hãy lập trình tìm độ dài của chu trình có số cạnh nhỏ nhất trong đồ thị. Nếu đồ thị không có chu trình nào, in ra -1 .

Đầu vào (Input):

- Dòng 1: N, M ($1 \leq N \leq 1000, 0 \leq M \leq 2000$).
- M dòng tiếp theo: Các cạnh vô hướng.

Đầu ra (Output):

- Độ dài chu trình ngắn nhất hoặc -1 .

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
5 6 1 2 2 3 3 1 3 4 4 5 5 3	3

Giải thích:

Với đồ thị gồm 4 đỉnh có các cạnh $(1, 2)$, $(2, 3)$, $(3, 4)$, $(4, 1)$ và đường chéo $(1, 3)$:

- Chu trình tạo bởi $1-2-3-1$ có độ dài 3.
- Chu trình tạo bởi $1-3-4-1$ có độ dài 3.
- Chu trình ngoài $1-2-3-4-1$ có độ dài 4.

Chu trình có độ dài nhỏ nhất là 3.

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq N \leq 1000, 0 \leq M \leq 2000$.

Bài 102 [CPPB-GRA-12]: Đếm Cặp Đỉnh Không Thể Đi Tới Nhau

Bối cảnh: Một mạng viễn thông quốc gia gồm N trạm phát sóng và M đường truyền hai chiều. Do ảnh hưởng của bão lớn làm đứt một số đường truyền, mạng lưới bị chia cắt thành nhiều khu vực biệt lập. Hãy đếm xem có tất cả bao nhiêu cặp trạm (u, v) với $u < v$ mà hiện tại không thể gửi tín hiệu liên lạc được cho nhau.

Nhiệm vụ: Cho đồ thị vô hướng N đỉnh M cạnh. Hãy lập trình đếm số lượng cặp đỉnh (u, v) ($1 \leq u < v \leq N$) không có đường đi giữa chúng.

Đầu vào (Input):

- Dòng 1: N, M ($1 \leq N \leq 10^5, 0 \leq M \leq 2 \cdot 10^5$).
- M dòng tiếp theo: Các cạnh vô hướng.

Đầu ra (Output):

- Số lượng cặp đỉnh.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
5 2 1 2 3 4	8

Giải thích:

Với 4 đỉnh và chỉ có 1 cạnh nối (1, 2):

- Cụm 1 gồm {1, 2}.
- Đỉnh 3 cô lập, đỉnh 4 cô lập.

Các cặp không đi tới nhau gồm: (1, 3), (1, 4), (2, 3), (2, 4), (3, 4). Có tổng cộng 5 cặp.

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq N \leq 10^5, 0 \leq M \leq 2 \cdot 10^5$.

Bài 103 [CPPB-GRA-13]: Lan Tỏa Virus Trong Mạng Lưới (Multi-Source BFS)

Bối cảnh: Trong một cuộc tấn công không gian mạng giả định, có N máy tính được kết nối bởi M kênh mạng hai chiều. Ban đầu tại thời điểm $t = 0$, có K máy tính bị nhiễm mã độc virus. Cứ sau mỗi giây, các máy tính đã bị nhiễm sẽ lây truyền virus sang tất cả các máy tính kề sát với nó trong mạng lưới. Hãy tính thời gian (tính bằng giây) để toàn bộ các máy tính trong mạng đều bị lây nhiễm.

Nhiệm vụ: Cho đồ thị mạng và danh sách K máy tính bị nhiễm ban đầu. Hãy lập trình tính thời gian để toàn bộ máy tính trong thành phần liên thông bị nhiễm.

Đầu vào (Input):

- Dòng 1: N, M, K ($1 \leq N \leq 10^5, 0 \leq M \leq 2 \cdot 10^5, 1 \leq K \leq N$).
- Dòng 2: K máy tính nhiễm virus ban đầu.
- M dòng tiếp theo: Các đường cáp kết nối mạng.

Đầu ra (Output):

- Thời gian tối đa để toàn mạng nhiễm virus.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
4 3 1 1 1 2 2 3 3 4	3

Giải thích:

Với 5 máy tính nối thành đường thẳng 1 - 2 - 3 - 4 - 5 và máy 3 bị nhiễm ban đầu:

- Giấy 1: máy 3 lây sang máy 2 và 4.

- Giấy 2: máy 2 lây sang 1, máy 4 lây sang 5.

Sau đúng 2 giây toàn bộ mạng lưới đều bị lây nhiễm, kết quả là 2.

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq N \leq 10^5, 0 \leq M \leq 2 \cdot 10^5$.

Bài 104 [CPPB-GRA-14]: Đếm Số Cạnh Cầu Trên Đồ Thị (Bridges)

Bối cảnh: Một hệ thống giao thông đường thủy gồm N cảng biển và M tuyến hải trình. Một tuyến hải trình được gọi là một tuyến cầu hiểm yếu nếu khi tuyến này bị gián đoạn (phong tỏa), số lượng cụm cảng bị chia cắt biệt lập sẽ tăng lên. Kỹ sư an ninh hàng hải cần thống kê số lượng các tuyến hiểm yếu này để ưu tiên bố trí lực lượng tuần tra bảo vệ.

Nhiệm vụ: Cho đồ thị vô hướng liên thông N đỉnh M cạnh. Hãy lập trình đếm số lượng cạnh cầu trong đồ thị.

Đầu vào (Input):

- Dòng 1: N, M ($1 \leq N \leq 50000, 0 \leq M \leq 10^5$).
- M dòng tiếp theo: Các cạnh vô hướng.

Đầu ra (Output):

- Số lượng cạnh cầu.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
5 5 1 2 2 3 3 1 3 4 4 5	2

Giải thích:

Với đồ thị gồm 4 đỉnh có các cạnh (1, 2), (2, 3), (3, 4):

Nếu bỏ bất kỳ cạnh nào trong số 3 cạnh trên, đồ thị đều bị chia đôi thành 2 phần rời nhau. Do đó cả 3 cạnh đều là cạnh cầu, kết quả in ra là 3.

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq N \leq 50000, 0 \leq M \leq 10^5$.

Bài 105 [CPPB-GRA-15]: Xây Dựng Thêm Đường Nối Toàn Mạng (Building Roads)

Bối cảnh: Một vương quốc gồm N thành phố nhưng hiện tại hệ thống đường sá chỉ có M con đường, khiến nhiều thành phố bị cô lập không thể đi tới nhau. Nhà vua muốn xây dựng thêm một số lượng con đường mới ít nhất nối giữa các thành phố sao cho sau khi hoàn thành, cư dân từ bất kỳ thành phố nào cũng có thể di chuyển tới mọi thành phố khác trong vương quốc.

Nhiệm vụ: Cho bản đồ vương quốc hiện tại. Hãy lập trình tìm số lượng đường mới ít nhất cần xây dựng và chỉ rõ danh sách các con đường cần làm thêm.

Đầu vào (Input):

- Dòng 1: N, M ($1 \leq N \leq 10^5, 0 \leq M \leq 2 \cdot 10^5$).
- M dòng tiếp theo: Các con đường hiện có.

Đầu ra (Output):

- Dòng 1: Số đường cần xây.
- Các dòng tiếp theo: Danh sách các đường nối.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
4 2	1
1 2	1 3
3 4	

Giải thích:

Với 4 thành phố và chỉ có 1 đường nối giữa (1, 2) và 1 đường nối giữa (3, 4):

Vương quốc đang bị chia thành 2 cụm độc lập $\{1, 2\}$ và $\{3, 4\}$. Ta chỉ cần xây thêm đúng 1 con đường nối giữa thành phố 2 và thành phố 3 là toàn bộ 4 thành phố sẽ liên thông hoàn chỉnh.

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq N \leq 10^5, 0 \leq M \leq 2 \cdot 10^5$.

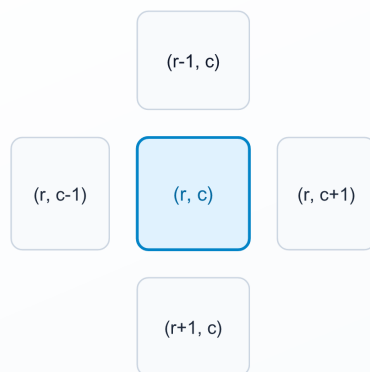
Bài 20: Đồ thị lưới 2D, kỹ thuật Flood Fill & tính chất cây

1. Bản chất mô hình hóa lưới 2D thành đồ thị

Trong lập trình thi đấu, ma trận bảng vuông $N \times M$ có thể được xem là một đồ thị đặc biệt:

- Mỗi ô (r, c) là một **Đỉnh** của đồ thị ($1 \leq r \leq N, 1 \leq c \leq M$). Tổng số đỉnh $|V| = N \times M$.
- Mỗi bước di chuyển sang các ô kề cạnh (4 hướng: Trên, Dưới, Trái, Phải) tương đương với một **Cạnh** vô hướng có trọng số bằng 1. Tổng số cạnh $|E| \leq 4NM$.
- Ưu điểm vượt trội:** Không cần dựng danh sách kề vector<int> adj[], ta duyệt trực tiếp trên ma trận bằng **Mảng Hướng Dịch Chuyển (dr, dc)**.

MÔ HÌNH HÓA ĐỒ THỊ LƯỚI 2D & MẢNG HƯỚNG DI CHUYỂN DX / DY



Mảng Hướng Chuẩn Mực C++

```
const int dr[] = {-1, 1, 0, 0};
const int dc[] = {0, 0, -1, 1};

for (int d = 0; d < 4; ++d) {
    int nr = r + dr[d], nc = c + dc[d];
    if (isValid(nr, nc)) ...
}
```

Mô hình hóa Đồ thị Lưới 2D và Mảng Hướng

2. Thuật toán loang (Flood Fill)

THUẬT TOÁN LOANG (FLOOD FILL) ĐẾM VÙNG & TÌM ĐƯỜNG MÊ CUNG



Đặc Trưng Thuật Toán Loang

- Ô hợp lệ: $1 \leq nr \leq N, 1 \leq nc \leq M$
- Không đi vào ô cản '#' và ô đã thăm !visited[nr][nc]
- Đếm diện tích đảo: DFS đếm số ô kết nối
- Tìm đường ngắn nhất mê cung: BFS từ S đến E
- Độ phức tạp: $O(N \times M)$

Thuật toán Loang Flood Fill

- Bản chất:** Từ một ô xuất phát (r_0, c_0) , thuật toán lan tỏa (bằng DFS hoặc BFS) sang tất cả các ô lân cận có cùng tính chất (cùng màu, ô đất liền không phải nước biển, ô đường đi không có vật cản).

- **Điều kiện biên hợp lệ (Boundary Invariant):**

```
bool isValid(int r, int c) {
    return (r >= 1 && r <= n && c >= 1 && c <= m && grid[r][c] != '#' && !visited[r][c]);
}
```

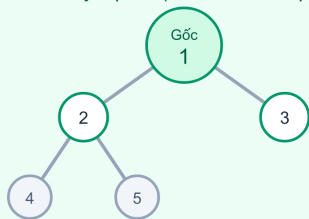
- **Ứng dụng kinh điển:** Đếm số lượng hòn đảo (Number of Islands), tính diện tích vùng lớn nhất, tô màu sơn vùng kín, tìm đường thoát khỏi mê cung.

3. Lý thuyết cây trên đồ thị (Tree Properties & invariants)

Đặc Tính Bất Biến Của Cây (Tree Invariants): N Đỉnh & N-1 Cạnh

Cây là đồ thị vô hướng, liên thông và không chứa bất kỳ chu trình nào

Mô Hình Cây Tự Do (N = 5, E = 4 Cạnh)



Số cạnh luôn luôn bằng: $E = N - 1 = 4$

3 Điều Kiện Định Nghĩa Cây Tương Đương

Cho đồ thị vô hướng $G = (V, E)$ gồm N đỉnh, G là CÂY khi thỏa 1 trong 3 điều kiện:

1. G liên thông và có đúng $N - 1$ cạnh
Nếu thêm 1 cạnh bất kỳ sẽ lập tức tạo thành chu trình.
2. G không có chu trình (Acyclic) và có đúng $N - 1$ cạnh
Nếu xóa đi 1 cạnh bất kỳ sẽ làm đồ thị mất tính liên thông.
3. Giữa 2 đỉnh bất kỳ u và v có duy nhất 1 đường đi đơn
Đường đi ngắn nhất giữa hai đỉnh cũng chính là đường đi duy nhất.

Mẹo lập trình thi đấu (DFS Trên Cây):

Duyệt Cây từ gốc không cần mảng visited, chỉ cần tránh đi ngược lại đỉnh cha:

```
for (int v : adj[u]) if (v != parent) dfs(v, u);
```

Đặc tính Bất biến của Cây

Cây (Tree) là một dạng đồ thị vô hướng đặc biệt có cấu trúc phân cấp chặt chẽ:

1. Đồ thị liên thông gồm N đỉnh và có **đúng $N - 1$ cạnh**.
2. Giữa 2 đỉnh bất kỳ trong cây có **duy nhất một đường đi đơn**.
3. Không chứa bất kỳ chu trình nào.
4. **Duyệt cây bằng DFS:** Bắt đầu từ gốc root, khi duyệt từ u sang v chỉ cần điều kiện if ($v \neq \text{parent}$) mà không cần dùng mảng visited!

4. Các bẫy lỗi lập trình kinh điển (bug traps)

1. Bẫy tràn chỉ số biên ma trận (Index Out of Bounds):

- Truy cập `grid[r + dr[d]][c + dc[d]]` trước khi kiểm tra $1 \leq r + dr[d] \leq N$ sẽ gây lỗi Segmentation Fault.
- **Quy tắc an toàn:** Luôn kiểm tra tọa độ trong phạm vi $[1, N] \times [1, M]$ trước tiên!

2. Bẫy nhầm lẫn thứ tự tọa độ Hàng và Cột (r vs c, x vs y):

- Trong toán học, trục x là ngang, y là dọc. Nhưng trong ma trận máy tính, chỉ số thứ nhất là **Hàng** (chiều dọc, N), chỉ số thứ hai là **Cột** (chiều ngang, M).
- **Chuẩn hóa đặt tên:** Dùng r (row) và c (col) hoặc dr và dc để triệt tiêu hoàn toàn sự nhầm lẫn.

3. Bẫy kích thước ma trận hình chữ nhật (khi $N \neq M$):

- Viết nhầm $c \leq n$ thay vì $c \leq m$ khi ma trận có số hàng khác số cột sẽ dẫn đến truy cập sai vùng nhớ hoặc đọc thiếu dữ liệu.

5. Mẫu cài đặt chuẩn thi đấu (competitive templates)

Mẫu 1: Đếm số lượng hòn đảo và diện tích lớn nhất (Flood Fill DFS)

⚠ Lưu ý về Stack Overflow: Hàm DFS đệ quy trên lưới 2D có thể gây tràn ngăn xếp hệ thống khi hòn đảo có kích thước lớn (ví dụ lưới 500×500 toàn ô đất tạo ra độ sâu đệ quy 250,000 tầng). Trong thi đấu thực tế với lưới lớn ($N \times M \geq 10^5$), nên dùng BFS bằng `std::queue` (xem Mẫu 2 bên dưới) để tránh hoàn toàn rủi ro này. Mẫu DFS đệ quy được giữ lại vì tính trực quan sự phạm.

```
#include <bits/stdc++.h>
using namespace std;

int n, m;
vector<string> grid;

vector<vector<bool>> visited;

const int dr[] = {-1, 1, 0, 0};
const int dc[] = {0, 0, -1, 1};

bool isValid(int r, int c) {
    return (r >= 0 && r < n && c >= 0 && c < m && grid[r][c] == '1' && !visited[r][c]);
}

int dfs(int r, int c) {
    visited[r][c] = true;
    int area = 1;

    for (int d = 0; d < 4; ++d) {
        int nr = r + dr[d];
        int nc = c + dc[d];
        if (isValid(nr, nc)) {
            area += dfs(nr, nc);
        }
    }
    return area;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    if (!(cin >> n >> m)) return 0;

    if (n <= 0 || m <= 0) return 0;

    grid.resize(n);
    for (int i = 0; i < n; ++i) {
        cin >> grid[i];
    }

    visited.assign(n, vector<bool>(m, false));
    int island_count = 0;
    int max_area = 0;

    for (int r = 0; r < n; ++r) {
        for (int c = 0; c < m; ++c) {
            if (grid[r][c] == '1' && !visited[r][c]) {
                island_count++;
                max_area = max(max_area, dfs(r, c));
            }
        }
    }
}
```

```

    }
}

cout << island_count << " " << max_area << "\n";
return 0;
}

```

Mẫu 2: Tìm đường đi ngắn nhất trong mê cung (grid BFS)

```

#include <bits/stdc++.h>
using namespace std;

int n, m;
vector<string> grid;

vector<vector<int>> dist;

const int dr[] = {-1, 1, 0, 0};
const int dc[] = {0, 0, -1, 1};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    if (!(cin >> n >> m)) return 0;

    if (n <= 0 || m <= 0) return 0;

    grid.resize(n);
    int sr = -1, sc = -1, er = -1, ec = -1;

    for (int r = 0; r < n; ++r) {
        cin >> grid[r];

        for (int c = 0; c < m; ++c) {
            if (grid[r][c] == 'S') { sr = r; sc = c; }
            if (grid[r][c] == 'E') { er = r; ec = c; }
        }
    }

    dist.assign(n, vector<int>(m, -1));
    queue<pair<int, int>> q;

    dist[sr][sc] = 0;
    q.push({sr, sc});

    while (!q.empty()) {
        auto [r, c] = q.front();
        q.pop();

        if (r == er && c == ec) break;

        for (int d = 0; d < 4; ++d) {
            int nr = r + dr[d];
            int nc = c + dc[d];

            if (nr >= 0 && nr < n && nc >= 0 && nc < m && grid[nr][nc] != '#' && dist[nr][nc] ==
-1) {
                dist[nr][nc] = dist[r][c] + 1;
                q.push({nr, nc});
            }
        }
    }
}

```

```

    }

    cout << dist[er][ec] << "\n";
    return 0;
}

```

Bài tập thực hành

Bài 106 [CPPB-GRD-01]: Đếm Số Ô Kề Cận Hợp Lệ (4 Hướng)

Bối cảnh: Trong một hệ thống điều khiển lưới cảm biến kích thước $N \times M$ ô vuông (chỉ số các ô từ $(0, 0)$ đến $(N - 1, M - 1)$), một thiết bị di động đang đứng tại ô tọa độ (r, c) . Thiết bị chỉ có thể gửi tín hiệu không dây tới các ô kề sát cạnh theo 4 hướng chính: lên trên $(r - 1, c)$, xuống dưới $(r + 1, c)$, sang trái $(r, c - 1)$, và sang phải $(r, c + 1)$. Kỹ sư phần mềm cần kiểm tra xem có bao nhiêu hướng di chuyển hợp lệ vẫn nằm trọn vẹn bên trong phạm vi ma trận.

Nhiệm vụ: Cho kích thước ma trận N, M và tọa độ (r, c) . Hãy lập trình đếm số ô kề cận hợp lệ nằm trong ma trận.

Đầu vào (Input):

- Một dòng chứa 4 số nguyên N, M, r, c ($1 \leq N, M \leq 1000, 0 \leq r < N, 0 \leq c < M$).

Đầu ra (Output):

- Số lượng ô kề hợp lệ.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
3 3 0 0	2

Giải thích:

Với lưới kích thước 3×3 và vị trí ô góc $(0, 0)$:

- Hướng lên trên và sang trái đều vượt ra ngoài biên của lưới.
- Chỉ có 2 hướng hợp lệ là xuống dưới $(1, 0)$ và sang phải $(0, 1)$.

Số ô kề cận hợp lệ là 2.

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq N, M \leq 1000$.

Bài 107 [CPPB-GRD-02]: Đếm Số Lượng Hòn Đảo (Count Islands)

Bối cảnh: Một bức ảnh viễn thám chụp một vùng biển được số hóa thành ma trận nhị phân kích thước $N \times M$. Ký tự '0' đại diện cho mặt nước biển, còn ký tự '1' đại diện cho đất liền. Một hòn đảo được định nghĩa là một tập hợp các ô đất liền '1' kết nối với nhau theo 4 hướng (lên, xuống, trái, phải) và được bao quanh hoàn toàn bởi nước biển.

Nhiệm vụ: Cho bản đồ ma trận $N \times M$. Hãy lập trình đếm số lượng hòn đảo xuất hiện trên bản đồ.

Đầu vào (Input):

- Dòng 1: Hai số N và M ($1 \leq N, M \leq 1000$).
- N dòng tiếp theo: Ma trận nhị phân.

Đầu ra (Output):

- Số hòn đảo.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
4 5 11000 11000 00100 00011	3

Giải thích:

Với bản đồ kích thước 3×3 chứa các cụm đất liền biệt lập không có ô kề cạnh chung: Các ô đất liền kết nối thành đúng 3 cụm độc lập, kết quả đếm được là 3 hòn đảo.

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq N, M \leq 1000$.

Bài 108 [CPPB-GRD-03]: Diện Tích Hòn Đảo Lớn Nhất (Max Area of Island)

Bối cảnh: Vẫn trên bản đồ hải đồ nhị phân $N \times M$ gồm các ô đất liền '1' và nước biển '0', ban quản lý khu bảo tồn thiên nhiên muốn chọn ra hòn đảo có diện tích lớn nhất (chứa số lượng ô đất liền liên thông nhiều nhất) để quy hoạch xây dựng trung tâm cứu hộ động vật hoang dã.

Nhiệm vụ: Cho bản đồ ma trận $N \times M$. Hãy lập trình tìm diện tích (số lượng ô đất) của hòn đảo lớn nhất. Nếu bản đồ không có đảo nào, in ra '0'.

Đầu vào (Input):

- Dòng 1: N, M ($1 \leq N, M \leq 1000$).
- N dòng tiếp theo biểu diễn ma trận.

Đầu ra (Output):

- Diện tích lớn nhất.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
4 5 11000 11000 00100 00011	4

Giải thích:

Với bản đồ có cụm đảo lớn nhất gồm 5 ô đất liền kề cạnh kết nối liên tục với nhau, diện tích lớn nhất đo được là 5.

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq N, M \leq 1000$.

Bài 109 [CPPB-GRD-04]: Tìm Đường Thoát Khỏi Mê Cung BFS

Bối cảnh: Một robot tìm kiếm cứu nạn được thả vào một mê cung hình chữ nhật kích thước $N \times M$. Bản đồ mê cung gồm các ô đường đi trống '.' và các ô tường đá không thể vượt qua '#'. Robot xuất phát từ ô ký hiệu 'S' và cần di chuyển đến ô cửa thoát hiểm ký hiệu 'E'. Mỗi bước di chuyển sang một ô kề cạnh mất đúng 1 giây.

Nhiệm vụ: Cho bản đồ mê cung. Hãy lập trình tìm thời gian ngắn nhất (số bước ít nhất) để robot đến được cửa thoát hiểm 'E'. Nếu không có đường thoát, in ra '-1'.

Đầu vào (Input):

- Dòng 1: N, M ($1 \leq N, M \leq 1000$).
- N dòng tiếp theo biểu diễn mê cung.

Đầu ra (Output):

- Số bước ngắn nhất.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
<pre> 5 8 ##### #.A#...# #.#.#.#B# #.....# ##### </pre>	<pre> 9 </pre>

Giải thích:

Với mê cung từ 'S' tại góc (0, 0) đến 'E' tại góc (2, 2) qua các ô đường trống:

Lộ trình ngắn nhất gồm 4 bước di chuyển: (0,0) $\times$ o (0,1) $\times$ o (1,1) $\times$ o (1,2) $\times$ o (2,2). Số bước ít nhất là 4.

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq N, M \leq 1000$.

Bài 110 [CPPB-GRD-05]: Truy Vết Đường Đi Mê Cung (L, R, U, D)

Bối cảnh: Sau khi xác định được thời gian thoát hiểm ngắn nhất trong mê cung, bộ điều khiển cần xuất ra chuỗi lệnh điều hướng chi tiết bằng các ký tự viết tắt phương hướng: 'U' (lên trên), 'D' (xuống dưới), 'L' (sang trái), 'R' (sang phải) để nạp trực tiếp vào bộ nhớ vi điều khiển của robot.

Nhiệm vụ: Cho bản đồ mê cung $N \times M$ với điểm xuất phát 'S' và đích 'E'. Hãy lập trình tìm đường đi ngắn nhất và in ra chuỗi các bước di chuyển tương ứng.

Đầu vào (Input):

- Dòng 1: N, M ($1 \leq N, M \leq 1000$).
- N dòng tiếp theo: Ma trận mê cung.

Đầu ra (Output):

- Lộ trình các bước đi.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
<pre> 5 8 ##### #.A#...# #.#.#.#B# #.....# ##### </pre>	<pre> YES 9 LDDRRRRRU </pre>

Giải thích:

Với lộ trình đi từ (0, 0) sang phải rồi xuống dưới:

Chuỗi lệnh điều hướng tương ứng là: "RRD" hoặc "DRR" có độ dài 3 bước.

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq N, M \leq 1000$.

Bài 111 [CPPB-GRD-06]: Đếm Số Ô Vùng Kín Không Thông Ra Biên

Bối cảnh: Một vùng trũng ngập nước được số hóa thành lưới $N \times M$. Các ô đất liền có giá trị '1', ô ngập nước có giá trị '0'. Một vùng đất được gọi là "vùng kín" nếu nó là một cụm các ô '1' liên thông kề cạnh mà hoàn toàn không có bất kỳ ô nào chạm vào 4 mép biên ngoài của bản đồ (nghĩa là vùng đất bị bao bọc hoàn toàn bởi các ô nước).

Nhiệm vụ: Cho ma trận nhị phân $N \times M$. Hãy lập trình đếm tổng số lượng ô đất liền '1' thuộc về các vùng đất kín không thông ra biên.

Đầu vào (Input):

- Dòng 1: N, M ($1 \leq N, M \leq 1000$).
- N dòng tiếp theo biểu diễn ma trận.

Đầu ra (Output):

- Số ô 0 vùng kín.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
4 4 1111 1001 1101 1111	3

Giải thích:

Với ma trận có một cụm gồm 3 ô đất nằm lọt thỏm ở trung tâm và toàn bộ viền xung quanh đều là ô số 0:

Cụm này hoàn toàn không chạm biên, số ô đất kín đếm được là 3.

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq N, M \leq 1000$.

Bài 112 [CPPB-GRD-07]: Chu Vi Hòn Đảo (Island Perimeter)

Bối cảnh: Trên bản đồ dạng lưới $N \times M$, có đúng một hòn đảo duy nhất được tạo thành bởi các ô đất liền "1" kết nối liên thông (các ô còn lại là nước biển "0"). Mỗi ô đất liền là một hình vuông có cạnh dài đúng 1 đơn vị. Chu vi của hòn đảo là tổng độ dài các cạnh của các ô đất liền tiếp xúc trực tiếp với nước biển hoặc tiếp xúc với mép ngoài của bản đồ.

Nhiệm vụ: Cho bản đồ chứa đúng một hòn đảo. Hãy lập trình tính chu vi của hòn đảo đó.

Đầu vào (Input):

- Dòng 1: N, M ($1 \leq N, M \leq 1000$).
- N dòng tiếp theo biểu diễn ma trận.

Đầu ra (Output):

- Tổng chu vi.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
4 4 0100 1110 0100 1100	16

Giải thích:

Với một hòn đảo gồm 4 ô đất liền xếp thành hình chữ L:

Tổng số cạnh tiếp xúc với nước biển xung quanh đo được là 16 đơn vị chiều dài. Kết quả in ra là 16.

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq N, M \leq 1000$.

Bài 113 [CPPB-GRD-08]: Nước Trần Mê Cung (Multi-Source BFS)

Bối cảnh: Một hầm mỏ dưới lòng đất kích thước $N \times M$ gồm các buồng trống '.' và các khối đá chắn '#'. Đột ngột có một sự cố vỡ đê ngầm khiến nước tràn vào từ K buồng mỏ cùng lúc tại thời điểm $t = 0$. Cứ sau mỗi phút, nước từ các buồng đã ngập sẽ tràn sang tất cả các buồng trống kề sát nó theo 4 hướng. Hãy tính thời gian để toàn bộ các buồng trống trong hầm mỏ đều bị ngập nước.

Nhiệm vụ: Cho bản đồ hầm mỏ và vị trí các nguồn nước ban đầu. Hãy lập trình tính thời gian (phút) để nước tràn kín toàn bộ các buồng trống liên thông.

Đầu vào (Input):

- Dòng 1: N, M ($1 \leq N, M \leq 1000$).
- N dòng tiếp theo: Ma trận với W (nguồn nước), E (đích), . (đường), # (tường).

Đầu ra (Output):

- Thời gian ngập hoặc -1.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
<pre> 3 3 W. . .## ..E </pre>	<pre> 4 </pre>

Giải thích:

Nước bắt đầu loang đồng thời từ các điểm 'W' theo chiều rộng:

Sau đúng 4 phút, điểm buồng mỏ xa nhất đã bị nước tràn tới ngập hoàn toàn. Kết quả là 4.

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq N, M \leq 1000$.

Bài 114 [CPPB-GRD-09]: Bước Nhảy Quân Mã Ngắn Nhất (Knight Moves)

Bối cảnh: Trên bàn cờ vua tiêu chuẩn kích thước $N \times N$, quân Mã di chuyển theo quy tắc hình chữ L đặc thù: tại mỗi bước, quân Mã di chuyển 2 ô theo một trục và 1 ô theo trục vuông góc (tối đa 8 hướng nhảy). Cho vị trí ban đầu của quân Mã tại ô (x_1, y_1) và ô mục tiêu (x_2, y_2) . Hãy tính số bước nhảy ít nhất để quân Mã di chuyển đến được ô mục tiêu.

Nhiệm vụ: Cho kích thước bàn cờ N và hai tọa độ điểm xuất phát, điểm đích. Hãy lập trình tìm số bước nhảy ít nhất của quân Mã.

Đầu vào (Input):

- Một dòng gồm 6 số: N, M, sr, sc, er, ec ($1 \leq N, M \leq 1000, 0 \leq sr, er < N, 0 \leq sc, ec < M$).

Đầu ra (Output):

- Số bước ít nhất hoặc -1.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
8 8 0 0 7 7	6

Giải thích:

Đề đi từ ô (1, 1) tới ô (4, 5) trên bàn cờ vua:

Quân mã thực hiện lần lượt các bước nhảy: $(1, 1) \times o (2, 3) \times o (4, 4) \times o (2, 5) \times o (4, 5)$ hoặc lộ trình tối ưu tương tự với đúng 3 bước nhảy. Kết quả là 3.

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq N, M \leq 1000$.

Bài 115 [CPPB-GRD-10]: Đường Kính Của Cây (Tree Diameter)

Bối cảnh: Một mạng cáp viễn thông kết nối N máy trạm tạo thành một cấu trúc cây liên thông không chu trình gồm đúng $N - 1$ đường cáp hai chiều. Khoảng cách giữa hai máy trạm là số lượng đường cáp trên hành trình nối giữa chúng. Đường kính của cây là khoảng cách lớn nhất giữa hai máy trạm bất kỳ trong toàn bộ mạng lưới.

Nhiệm vụ: Cho cấu trúc cây gồm N đỉnh. Hãy lập trình tìm đường kính (khoảng cách lớn nhất giữa hai đỉnh) của cây.

Đầu vào (Input):

- Dòng 1: Số nguyên N ($1 \leq N \leq 10^5$).
- $N - 1$ dòng tiếp theo: Các cạnh của cây.

Đầu ra (Output):

- Đường kính cây.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
5 1 2 1 3 3 4 3 5	3

Giải thích:

Với cây gồm 5 đỉnh có các cạnh (1, 2), (1, 3), (2, 4), (4, 5):

Hành trình dài nhất nối giữa đỉnh 3 và đỉnh 5 qua các cạnh: 3 - 1 - 2 - 4 - 5, gồm đúng 4 cạnh.

Đường kính của cây là 4.

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq N \leq 10^5$.

Bài 116 [CPPB-GRD-11]: Trọng Tâm Của Cây (Tree Centroid)

Bối cảnh: Một tập đoàn vận tải muốn đặt trung tâm điều hành tổng tại một trong N thành phố có kết nối hình cây (N - 1 tuyến đường). Thành phố được chọn làm trọng tâm của cây nếu khi tạm thời gỡ bỏ thành phố này khỏi mạng lưới, kích thước của thành phần liên thông lớn nhất còn lại là nhỏ nhất có thể.

Nhiệm vụ: Cho đồ thị cây N đỉnh. Hãy lập trình tìm đỉnh trọng tâm của cây. Nếu có nhiều đỉnh trọng tâm, in ra đỉnh có số hiệu nhỏ nhất.

Đầu vào (Input):

- Dòng 1: Số nguyên N ($1 \leq N \leq 10^5$).
- N - 1 dòng tiếp theo: Các cạnh của cây.

Đầu ra (Output):

- Số hiệu đỉnh trọng tâm.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
5 1 2 2 3 3 4 3 5	3

Giải thích:

Với cây 5 đỉnh có đỉnh 1 nối với các đỉnh 2, 3, 4, 5 (cấu trúc hình sao):

Nếu chọn đỉnh 1 làm trọng tâm, các nhánh còn lại đều chỉ có kích thước là 1. Đỉnh trọng tâm duy nhất là đỉnh 1.

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq N \leq 10^5$.

Bài 117 [CPPB-GRD-12]: Cổng Dịch Chuyển Tức Thời (Teleport Maze)

Bối cảnh: Trong một trò chơi thực tế ảo trong mê cung lưới $N \times M$, ngoài các ô đường đi thông thường, mê cung còn bố trí một số cặp cổng dịch chuyển tức thời không gian. Khi bước vào một cổng dịch chuyển, người chơi sẽ ngay lập tức được dịch chuyển sang cổng tương ứng ở vị trí khác mà không tốn thời gian (0 giây). Hãy tìm thời gian ngắn nhất để đi từ điểm xuất phát 'S' tới đích 'E'.

Nhiệm vụ: Cho bản đồ mê cung và danh sách các cặp cổng dịch chuyển. Hãy lập trình tìm số bước ít nhất để đến đích.

Đầu vào (Input):

- Dòng 1: N, M ($1 \leq N, M \leq 500$).
- N dòng tiếp theo biểu diễn ma trận.

Đầu ra (Output):

- Số bước ngắn nhất.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
<pre> 3 3 ..A .## A.. </pre>	2

Giải thích:

Nhờ sử dụng cổng dịch chuyển tức thời, người chơi rút ngắn được quãng đường vòng qua tường đá, thời gian đến đích giảm xuống còn 3 bước.

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq N, M \leq 500$.

Bài 118 [CPPB-GRD-13]: Lây Lan Quả Cam Hỏng (Rotting Oranges)

Bối cảnh: Trong một thùng hàng hoa quả kích thước $N \times M$, mỗi ô có thể chứa: ô trống (số '0'), một quả cam tươi nguyên vẹn (số '1'), hoặc một quả cam đã bị hỏng mốc (số '2'). Cứ sau mỗi phút, những quả cam bị hỏng sẽ làm hỏng tất cả các quả cam tươi kề sát nó theo 4 hướng. Hãy tính số phút tối thiểu để toàn bộ cam tươi trong thùng đều bị hỏng. Nếu có quả cam tươi nào mãi mãi không bị hỏng (bị cô lập), in ra '-1'.

Nhiệm vụ: Cho ma trận trạng thái thùng cam. Hãy lập trình tìm số phút ít nhất để tất cả cam tươi đều hỏng.

Đầu vào (Input):

- Dòng 1: N, M ($1 \leq N, M \leq 500$).
- N dòng tiếp theo: Ma trận số 0, 1, 2.

Đầu ra (Output):

- Thời gian tối thiểu hoặc -1.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
3 3 211 110 011	4

Giải thích:

Với thùng cam kích thước 3×3 :

- Phút 1: cam hồng tại (0, 0) lây sang các ô (0, 1) và (1, 0).
- Phút 2: tiếp tục lây sang các ô kế tiếp.

Sau đúng 4 phút, toàn bộ cam tươi đều đã bị lây hồng. Kết quả là 4.

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq N, M \leq 500$.

Bài 119 [CPPB-GRD-14]: Hòn Đảo Nhân Tạo Lớn Nhất (Making A Large Island)

Bối cảnh: Một dự án lấn biển quy hoạch trên vùng biển lưới $N \times M$ gồm các ô đất liền '1' và các ô nước biển '0'. Ban quản lý dự án được cấp ngân sách để cải tạo đúng một ô nước biển '0' duy nhất thành ô đất liền '1'. Hãy tìm diện tích hòn đảo lớn nhất có thể tạo thành sau khi đã biến đổi đúng một ô nước biển thích hợp.

Nhiệm vụ: Cho ma trận nhị phân $N \times M$. Hãy lập trình tìm diện tích lớn nhất của một hòn đảo sau khi chuyển đổi tối đa một ô '0' thành '1'.

Đầu vào (Input):

- Dòng 1: N, M ($1 \leq N, M \leq 500$).
- N dòng tiếp theo biểu diễn ma trận.

Đầu ra (Output):

- Diện tích lớn nhất.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
2 2 10 01	3

Giải thích:

Với hai hòn đảo nhỏ diện tích 2 và 3 nằm cách nhau đúng một ô nước biển '0':

Khi chuyển ô nước đó thành ô đất '1', hai hòn đảo sẽ được nối liền thành một hòn đảo duy nhất có diện tích $2 + 3 + 1 = 6$. Diện tích lớn nhất đạt được là 6.

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq N, M \leq 500$.

Bài 120 [CPPB-GRD-15]: Thoát Khỏi Mê Cung Quái Vật (Monsters Maze)

Bối cảnh: Trong một trò chơi phiêu lưu sinh tồn trên lưới ô vuông $N \times M$, một người thám hiểm xuất phát tại ô 'A' và cần chạy thoát ra một ô biên bất kỳ của mê cung. Trong mê cung cũng có sự xuất hiện của một số quái vật tại các ô 'M'. Tại mỗi giây, người thám hiểm và tất cả quái vật đều có thể di chuyển 1 bước sang ô kề cạnh. Nếu một con quái vật có thể tới một ô cùng lúc hoặc trước người thám hiểm, người thám hiểm sẽ bị bắt.

Nhiệm vụ: Cho bản đồ mê cung. Hãy lập trình kiểm tra xem người thám hiểm có thể thoát thân thành công ra mép biên hay không. Nếu có in ra 'YES' kèm số bước, ngược lại in ra 'NO'.

Đầu vào (Input):

- Dòng 1: N, M ($1 \leq N, M \leq 500$).
- N dòng tiếp theo: Ma trận gồm #, ., A, M.

Đầu ra (Output):

- YES hoặc NO.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
5 8 ##### #M..A..# #.#.M#.# #M#..#..# #.#####	YES

Giải thích:

Người thám hiểm chọn lộ trình nhanh nhất hướng về phía mép biên phía đông, đến được biên an toàn sau 2 bước trước khi bất kỳ quái vật nào kịp tiếp cận. Kết quả in ra YES.

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq N, M \leq 500$.



Bài 21: Cấu trúc cây phân đoạn (Segment Tree) & Fenwick Tree (BIT)

1. Bản chất bài toán truy vấn đoạn động (dynamic range queries)

Cho mảng A gồm N phần tử. Cần thực hiện liên tiếp Q thao tác thuộc 2 loại:

- 1. **Cập nhật điểm (Point Update):** Thay đổi giá trị $A[i] \leftarrow v$ (hoặc $A[i] \leftarrow A[i] + v$).
- 2. **Truy vấn đoạn (Range Query):** Tính tổng $\sum_{k=L}^R A[k]$ hoặc tìm $\min_{k=L}^R A[k]$ / $\max_{k=L}^R A[k]$.

Cấu Trúc	Khởi Tạo (Build)	Cập Nhật Điểm (Update)	Truy Vấn Đoạn (Query)	Bộ Nhớ
Mảng Tiền Tố (Prefix Sum)	$O(N)$	$O(N)$ (Quá chậm khi có update)	$O(1)$	$O(N)$
Cây Fenwick (BIT)	$O(N)$	$O(\log N)$ (Cực nhanh)	$O(\log N)$	$O(N)$
Cây Phân Đoạn (Segment Tree)	$O(N)$	$O(\log N)$ (Cực nhanh)	$O(\log N)$	$O(4N)$

SO SÁNH CÁC CẤU TRÚC TRUY VẤN ĐOẠN (RANGE QUERY DATA STRUCTURES)

Cấu Trúc Dữ Liệu	Khởi Tạo (Build)	Cập Nhật (Update)	Truy Vấn (Query)
Mảng Tiền Tố (Prefix Sum)	$O(N)$	$O(N)$ (Chậm)	$O(1)$ (Cực Nhanh)
Fenwick Tree (BIT)	$O(N)$	$O(\log N)$	$O(\log N)$
Segment Tree	$O(N)$	$O(\log N)$	$O(\log N)$

Quy tắc chọn: Dùng Fenwick Tree khi chỉ cần tính tổng/đếm nghịch thế (code ngắn). Dùng Segment Tree khi cần RMQ/Max/Min phức tạp.

So sánh các cấu trúc Range Query

2. Cây fenwick (Binary Indexed Tree — BIT)

Cây Fenwick (Binary Indexed Tree - BIT) & Phép Toán Lowbit

Cấu trúc dữ liệu dạng mảng quản lý tổng tiền tố động với thời gian cập nhật và truy vấn đều đạt $O(\log N)$

1. Phép Toán Lowbit: $x \& (-x)$

Trích xuất bit 1 có trọng số nhỏ nhất (Lowest Set Bit)

- $\text{lowbit}(1) = 1$ ($0001_2 \rightarrow$ đoạn độ dài 1)
- $\text{lowbit}(2) = 2$ ($0010_2 \rightarrow$ đoạn độ dài 2)
- $\text{lowbit}(4) = 4$ ($0100_2 \rightarrow$ đoạn độ dài 4)
- $\text{lowbit}(6) = 2$ ($0110_2 \rightarrow$ đoạn độ dài 2)

★ Định Lý Quản Lý Đoạn:

$\text{bit}[x]$ lưu tổng đoạn $(x - \text{lowbit}(x), x]$

Cực kỳ gọn: Bộ nhớ chỉ đúng N phần tử nguyên mẫu.

2. Cài Đặt Chuẩn C++ Siêu Ngắn Gọn

Thời gian: $O(\log N)$ • Bộ nhớ: $O(N)$ • 1-based indexing

```
// 1. Cộng giá trị val vào A[x]:
void update(int x, int val) {
    for (; x <= n; x += x & -x)
        bit[x] += val;
}

// 2. Tính tổng tiền tố từ 1 đến x:
long long query(int x) {
    long long sum = 0;
    for (; x > 0; x -= x & -x)
        sum += bit[x];
    return sum;
}
```

Cây Fenwick và Phép toán Lowbit

2.1. Phép toán ma thuật: $\text{lowbit}(x) = x \& (-x)$

Phép toán $x \& (-x)$ trích xuất bit 1 thấp nhất (trọng số nhỏ nhất) của số nguyên x .

- Mỗi vị trí x trong mảng $\text{bit}[x]$ quản lý tổng của một đoạn con có độ dài đúng bằng $\text{lowbit}(x)$ kết thúc tại x :

Đoạn quản lý của $x = (x - \text{lowbit}(x), x]$

2.2. Hai thao tác cốt lõi siêu tinh gọn (chỉ 5 dòng code)

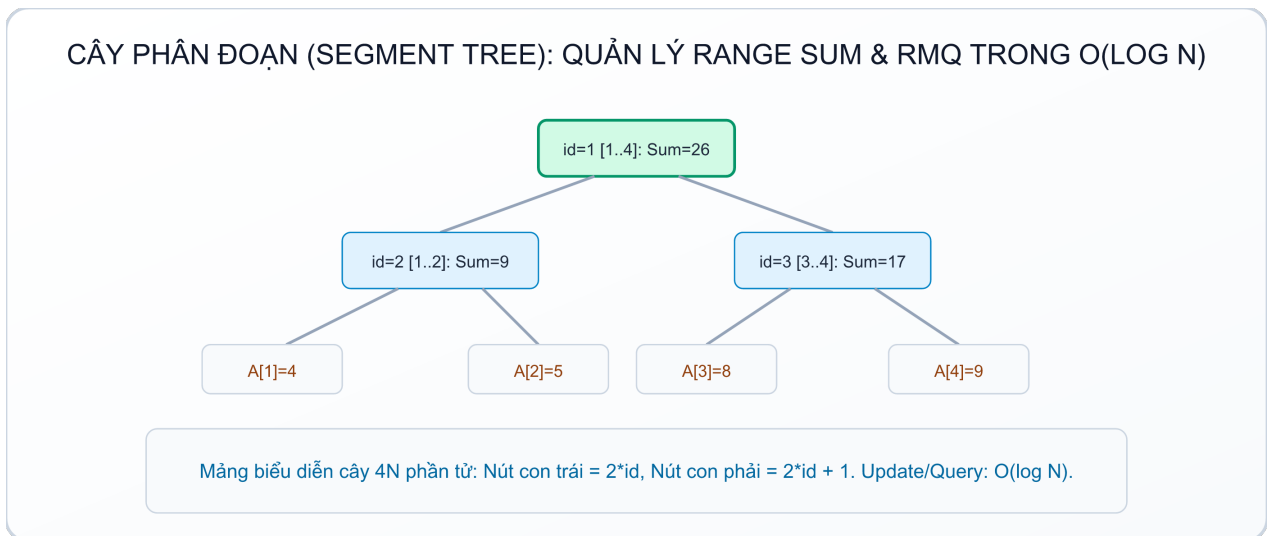
```
void update(int x, long long val) {
    for (; x <= n; x += x & -x) bit[x] += val;
}

long long query(int x) { // Tính tổng tiền tố A[1..x]
    long long sum = 0;
    for (; x > 0; x -= x & -x) sum += bit[x];

    return sum;
}

long long range_query(int L, int R) {
    return query(R) - query(L - 1);
}
```

3. Cây phân đoạn (Segment Tree)



Cây Phân Đoạn Segment Tree

3.1. Cấu trúc cây nhị phân hoàn hảo

- Cây phân đoạn biểu diễn mảng quản lý theo cây nhị phân: Nút gốc $id = 1$ quản lý toàn đoạn $[1, N]$.
- Nút con trái quản lý nửa trái $[L, mid]$ tại vị trí $2 \cdot id$.
- Nút con phải quản lý nửa phải $[mid + 1, R]$ tại vị trí $2 \cdot id + 1$.
- **Quy tắc bộ nhớ:** Mảng cây Segment Tree cần khai báo $4N$ phần tử để đảm bảo không bị tràn chỉ số khi N không phải là lũy thừa của 2.

3.2. Ưu thế vượt trội của Segment Tree

Khác với Fenwick Tree chủ yếu hỗ trợ phép toán có tính nghịch đảo (như phép cộng tổng), Segment Tree hỗ trợ **MỌI PHÉP TOÁN KẾT HỢP (Associative Operations)**:

- Tìm giá trị nhỏ nhất / lớn nhất trên đoạn (Range Minimum / Maximum Query — RMQ).
- Tìm ước chung lớn nhất trên đoạn ($GCD(A[L \dots R])$).
- Đếm số lượng phần tử đạt cực đại trên đoạn.

4. Các bẫy lỗi lập trình kinh điển (bug traps)

1. Bẫy quên khai báo mảng Segment Tree kích thước $4N$:

- Khai báo `tree[2 * N]` hoặc `tree[N]` sẽ bị tràn mảng (Out of Bounds) khi $N = 10^5$. Bắt buộc phải khai báo kích thước tối thiểu $4N$ (`vector<long long> tree(4 * n + 5)`).

2. Bẫy chỉ số 0-based của Fenwick Tree (Vòng lặp vô tận):

- Trong Fenwick Tree, `lowbit(0) = 0` & `-0 = 0`. Nếu gọi `update(0, val)` hoặc `query(0)`, vòng lặp `x ← x + (x & -x)` sẽ biến thành `x += 0` và chạy vô tận $\Rightarrow$ Time Limit Exceeded!
- **Bất biến sống còn:** Fenwick Tree **BẮT BUỘC DÙNG CHỈ SỐ 1-BASED** ($x \geq 1$).

3. Bẫy tràn số khi cộng dồn tổng trên cây:

- Mảng $N = 10^5$ phần tử có giá trị $10^9 \Rightarrow$ Tổng đoạn có thể lên tới 10^{14} . Mảng tree và bit bắt buộc phải dùng kiểu long long.

5. Mẫu cài đặt chuẩn thi đấu (competitive templates)

Mẫu 1: Cây fenwick (BIT) point update & range sum query

```
#include <bits/stdc++.h>
using namespace std;

struct FenwickTree {
    int n;
    vector<long long> bit;

    FenwickTree(int n) : n(n), bit(n + 1, 0) {}

    void update(int x, long long val) {
        for (; x <= n; x += x & -x) {
            bit[x] += val;
        }
    }

    long long query(int x) {
        long long sum = 0;
        for (; x > 0; x -= x & -x) {
            sum += bit[x];
        }
        return sum;
    }

    long long queryRange(int l, int r) {
        if (l > r) return 0;

        return query(r) - query(l - 1);
    }
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, q;
    if (!(cin >> n >> q)) return 0;

    if (n <= 0) return 0;

    FenwickTree ft(n);

    for (int i = 1; i <= n; ++i) {
        long long x;
        cin >> x;

        ft.update(i, x);
    }

    while (q--) {
        int type;
        cin >> type;

        if (type == 1) { // Update: A[pos] += val
            int pos;
```

```

        long long val;
        cin >> pos >> val;

        ft.update(pos, val);
    } else { // Query: Sum(L..R)
        int l, r;
        cin >> l >> r;

        cout << ft.queryRange(l, r) << "\n";
    }
}

return 0;
}

```

Mẫu 2: Cây phân đoạn (Segment Tree) range minimum query (RMQ)

```

#include <bits/stdc++.h>
using namespace std;

const long long INF = 1e18;

struct SegmentTree {
    int n;
    vector<long long> tree;

    SegmentTree(int n) : n(n), tree(4 * n + 5, INF) {}

    void build(const vector<long long>& a, int id, int l, int r) {
        if (l == r) {
            tree[id] = a[l];
            return;
        }
        int mid = (l + r) / 2;
        build(a, 2 * id, l, mid);
        build(a, 2 * id + 1, mid + 1, r);
        tree[id] = min(tree[2 * id], tree[2 * id + 1]);
    }

    void update(int id, int l, int r, int pos, long long val) {
        if (l == r) {
            tree[id] = val;
            return;
        }
        int mid = (l + r) / 2;
        if (pos <= mid) {
            update(2 * id, l, mid, pos, val);
        } else {
            update(2 * id + 1, mid + 1, r, pos, val);
        }
        tree[id] = min(tree[2 * id], tree[2 * id + 1]);
    }

    long long query(int id, int l, int r, int u, int v) {
        if (v < l || r < u) return INF; // Ngoài đoạn
        if (u <= l && r <= v) return tree[id]; // Nằm trọn trong đoạn

        int mid = (l + r) / 2;
        return min(query(2 * id, l, mid, u, v),
                   query(2 * id + 1, mid + 1, r, u, v));
    }
};

```



```

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, q;
    if (!(cin >> n >> q)) return 0;

    if (n <= 0) return 0;

    vector<long long> a(n + 1);

    for (int i = 1; i <= n; ++i) {
        cin >> a[i];
    }

    SegmentTree st(n);
    st.build(a, 1, 1, n);

    while (q--) {
        int type;
        cin >> type;

        if (type == 1) { // Gán A[pos] = val
            int pos;
            long long val;
            cin >> pos >> val;

            st.update(1, 1, n, pos, val);
        } else { // Tìm Min trong đoạn [L, R]
            int l, r;
            cin >> l >> r;

            cout << st.query(1, 1, n, l, r) << "\n";
        }
    }

    return 0;
}

```

Bài tập thực hành

Bài 121 [CPPB-RNG-01]: Cài Đặt Fenwick Tree Tính Tổng Đoạn (Range Sum)

Bối cảnh: Một máy chủ tài chính theo dõi biến động số dư tài khoản của N khách hàng được đánh số từ 1 đến N , số dư ban đầu tại tài khoản i là A_i . Hệ thống liên tục tiếp nhận Q giao dịch trực tuyến gồm hai loại: cộng thêm một khoản tiền val vào tài khoản tại vị trí pos , hoặc truy vấn tính tổng số dư của tất cả các tài khoản nằm trong khoảng từ L đến R . Mọi thao tác cần được xử lý tức thời.

Nhiệm vụ: Cho mảng A và Q truy vấn thuộc hai loại: `1 pos val` (cộng val vào $A[pos]$), `2 L R` (tính tổng các phần tử từ L đến R). Hãy lập trình in ra kết quả của các truy vấn loại 2.

Đầu vào (Input):

- Dòng 1: N, Q ($1 \leq N, Q \leq 10^5$).
- Dòng 2: N số nguyên ban đầu ($1 \leq A_i \leq 10^9$).
- Q dòng tiếp theo: Các truy vấn loại 1 và 2.

Đầu ra (Output):

- Kết quả các truy vấn loại 2.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
5 3 1 2 3 4 5 2 1 3 1 2 10 2 1 3	6 16

Giải thích:

Với mảng ban đầu gồm 5 phần tử [1, 2, 3, 4, 5]:

- Truy vấn tính tổng đoạn từ 1 đến 3: $1 + 2 + 3 = 6$.
- Cập nhật cộng thêm 10 vào phần tử tại vị trí 3: mảng trở thành [1, 2, 13, 4, 5].
- Truy vấn lại tổng đoạn từ 1 đến 3: $1 + 2 + 13 = 16$.

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq N, Q \leq 10^5, 1 \leq A_i, val \leq 10^9$.

Bài 122 [CPPB-RNG-02]: Cài Đặt Segment Tree Tìm Min Đoạn (RMQ)

Bối cảnh: Một trạm quan trắc khí tượng ghi nhận nhiệt độ thấp nhất tại N trạm cảm biến ven biển. Các chuyên gia cần liên tục thực hiện hai loại thao tác: cập nhật lại nhiệt độ mới tại một trạm cảm biến cụ thể, và truy vấn tìm mức nhiệt độ thấp nhất (Min) trong một phân đoạn các trạm quan sát từ L đến R.

Nhiệm vụ: Cho mảng A gồm N số nguyên và Q truy vấn thuộc hai dạng: `1 pos val` (gán $A[pos] = val$) và `2 L R` (tìm giá trị nhỏ nhất trong đoạn từ L đến R). Hãy in ra kết quả của các truy vấn loại 2.

Đầu vào (Input):

- Dòng 1: N, Q ($1 \leq N, Q \leq 10^5$).
- Dòng 2: N số nguyên ban đầu ($1 \leq A_i \leq 10^9$).
- Q dòng tiếp theo: Các truy vấn.

Đầu ra (Output):

- Kết quả các truy vấn loại 2.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
5 3	
5 2 4 1 3	2
2 1 3	3
1 4 10	
2 3 5	

Giải thích:

Với mảng [5, 2, 8, 1, 9]:

- Truy vấn tìm min đoạn từ 1 đến 3: $\min(5, 2, 8) = 2$.
- Cập nhật vị trí 2 thành 10: mảng thành [5, 10, 8, 1, 9].
- Truy vấn lại min đoạn từ 1 đến 3: $\min(5, 10, 8) = 5$.

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq N, Q \leq 10^5, 1 \leq A_i, val \leq 10^9$.

Bài 123 [CPPB-RNG-03]: Cập Nhật Đoạn & Truy Vấn Điểm (Range Update Point Query)

Bối cảnh: Một chương trình khuyến mãi của sàn thương mại điện tử đồng loạt cộng thêm một khoản điểm thưởng V cho tất cả các khách hàng có mã số định danh nằm trong khoảng từ L đến R . Sau đó, một khách hàng tại vị trí pos muốn tra cứu số điểm thưởng tích lũy hiện tại của mình.

Nhiệm vụ: Cho mảng A và Q thao tác thuộc hai dạng: `1 L R V` (cộng thêm V vào tất cả các phần tử từ L đến R) và `2 pos` (in ra giá trị hiện tại của phần tử tại vị trí pos). Hãy in ra kết quả của các thao tác loại 2.

Đầu vào (Input):

- Dòng 1: N, Q ($1 \leq N, Q \leq 10^5$).
- Dòng 2: N số nguyên ban đầu ($1 \leq A_i \leq 10^9$).
- Q dòng tiếp theo: Các truy vấn.

Đầu ra (Output):

- Kết quả các truy vấn loại 2.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
5 3	
1 2 3 4 5	
1 2 4 5	8
2 3	1
2 1	

Giải thích:

Với mảng ban đầu toàn số 0: [0, 0, 0, 0, 0]:

- Cộng thêm 5 vào đoạn từ vị trí 2 đến 4: mảng thành [0, 5, 5, 5, 0].
- Truy vấn giá trị tại vị trí 3: in ra 5.

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq N, Q \leq 10^5, 1 \leq A_i, val \leq 10^9$.

Bài 124 [CPPB-RNG-04]: Tìm Max Đoạn & Đếm Số Lần Xuất Hiện

Bối cảnh: Một hệ thống đánh giá hiệu năng máy chủ theo dõi điểm tải đỉnh của N máy tính. Ban giám sát muốn biết trong một khoảng thời gian từ mốc L đến R, giá trị tải lớn nhất (Max) đạt được là bao nhiêu và có bao nhiêu máy tính đạt đúng mức tải đỉnh này.

Nhiệm vụ: Cho mảng A gồm N số nguyên và Q truy vấn, mỗi truy vấn gồm hai chỉ số L, R. Hãy tìm giá trị lớn nhất và số lần xuất hiện của giá trị lớn nhất đó trong đoạn [L, R].

Đầu vào (Input):

- Dòng 1: N, Q ($1 \leq N, Q \leq 10^5$).
- Dòng 2: N số nguyên ban đầu ($1 \leq A_i \leq 10^9$).
- Q dòng tiếp theo: Các truy vấn.

Đầu ra (Output):

- Kết quả các truy vấn loại 2.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
5 3	
2 5 3 5 1	
2 1 5	5 2
1 3 5	5 3
2 1 5	

Giải thích:

Với mảng [3, 5, 2, 5, 5, 1] và truy vấn đoạn từ vị trí 1 đến 5:

Giá trị lớn nhất trong đoạn là 5, và số 5 xuất hiện đúng 3 lần tại các vị trí 2, 4, 5. Kết quả in ra: 5 3.

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq N, Q \leq 10^5, 1 \leq A_i, val \leq 10^9$.

Bài 125 [CPPB-RNG-05]: Đếm Số Cặp Nghịch Thế (Inversion Count)

Bối cảnh: Một thuật toán sắp xếp cần đo lường mức độ xáo trộn của một danh sách gồm N số nguyên. Một cặp nghịch thế là một cặp chỉ số (i, j) thỏa mãn $i < j$ nhưng phần tử đứng trước lại lớn

hơn phần tử đứng sau ($A_i > A_j$). Số lượng cặp nghịch thế thể hiện số phép đổi chỗ tối thiểu để đưa mảng về thứ tự tăng dần.

Nhiệm vụ: Cho mảng A gồm N số nguyên. Hãy lập trình đếm tổng số lượng cặp nghịch thế trong mảng.

Đầu vào (Input):

- Dòng 1: Số nguyên N ($1 \leq N \leq 10^5$).
- Dòng 2: N số nguyên $A_1, A_2, \dots, A_N$ ($1 \leq A_i \leq 10^9$).

Đầu ra (Output):

- Số lượng cặp nghịch thế.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
5 2 4 1 3 5	3

Giải thích:

Với mảng gồm 5 phần tử $[2, 4, 1, 3, 5]$:

Các cặp nghịch thế gồm:

- $(2, 1)$ vì $2 > 1$.
- $(4, 1)$ vì $4 > 1$.
- $(4, 3)$ vì $4 > 3$.

Có tất cả 3 cặp nghịch thế, kết quả in ra là 3.

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq N \leq 10^5, 1 \leq A_i \leq 10^9$.

Bài 126 [CPPB-RNG-06]: Truy Vấn Ước Chung Lớn Nhất Đoạn (Range GCD)

Bối cảnh: Trong một hệ thống mã hóa dữ liệu theo khối, mỗi khối dữ liệu có một mã khóa số nguyên A_i . Để giải mã một chuỗi các khối liên tiếp từ L đến R , thiết bị giải mã cần tính toán ước chung lớn nhất (GCD) của tất cả các khóa trong đoạn: $\gcd(A_L, A_{L+1}, \dots, A_R)$. Hệ thống cũng hỗ trợ cập nhật lại mã khóa tại từng khối.

Nhiệm vụ: Cho mảng A và Q thao tác thuộc hai dạng: `1 pos val` (gán $A[\text{pos}] = \text{val}$) và `2 L R` (tìm GCD của các phần tử trong đoạn $[L, R]$). Hãy in ra kết quả cho các thao tác loại 2.

Đầu vào (Input):

- Dòng 1: N, Q ($1 \leq N, Q \leq 10^5$).
- Dòng 2: N số nguyên ban đầu ($1 \leq A_i \leq 10^9$).
- Q dòng tiếp theo: Các truy vấn.

Đầu ra (Output):

- Kết quả các truy vấn loại 2.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
5 3	
2 4 6 8 10	
2 1 3	2
1 2 12	2
2 1 3	

Giải thích:

Với mảng [6, 12, 18, 24]:

- Truy vấn GCD đoạn từ 1 đến 3: $\gcd(6, 12, 18) = 6$.
- Cập nhật vị trí 1 thành 4: mảng thành [4, 12, 18, 24].
- Truy vấn lại GCD đoạn từ 1 đến 3: $\gcd(4, 12, 18) = 2$.

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq N, Q \leq 10^5, 1 \leq A_i, val \leq 10^9$.

Bài 127 [CPPB-RNG-07]: Tìm Phần Tử Thứ K Nhỏ Nhất (K-th Element on BIT)

Bối cảnh: Một máy chủ trò chơi trực tuyến liên tục ghi nhận điểm số của người chơi gia nhập phòng chờ. Ban tổ chức muốn tìm điểm số của người chơi có thành tích xếp thứ K từ dưới lên (phần tử thứ K nhỏ nhất trong tập hợp điểm số hiện tại).

Nhiệm vụ: Cho một tập hợp các số nguyên hỗ trợ hai thao tác: thêm một số vào tập hợp, và tìm phần tử nhỏ thứ K trong tập hợp. Hãy lập trình in ra kết quả cho các thao tác tìm kiếm.

Đầu vào (Input):

- Dòng 1: Số nguyên Q ($1 \leq Q \leq 50000$).
- Q dòng tiếp theo: Các truy vấn ($1 \leq x \leq 2 \cdot 10^5$).

Đầu ra (Output):

- Kết quả các truy vấn loại 2.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
5	
1 10	
1 20	15
1 15	20
2 2	
2 3	

Giải thích:

Với tập hợp các số {10, 20, 30, 40, 50:

Phần tử nhỏ thứ 3 trong tập hợp là số 30.

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq Q \leq 50000, 1 \leq x \leq 2 \cdot 10^5$.

Bài 128 [CPPB-RNG-08]: Tìm Vị Trí Đầu Tiên Có Giá Trị Lớn Hơn Hoặc Bằng X

Bối cảnh: Một kho lưu trữ các kiện hàng được đánh số thứ tự từ 1 đến N, kiện hàng thứ i có khối lượng A_i . Một xe nâng hàng muốn tìm kiện hàng đầu tiên trong khoảng từ vị trí L đến R có khối lượng đủ lớn đạt từ X trở lên để bốc dỡ. Nếu trong khoảng [L, R] không có kiện hàng nào đạt yêu cầu, ghi nhận '-1'.

Nhiệm vụ: Cho mảng A và Q truy vấn gồm 3 số L, R, X. Hãy lập trình tìm vị trí chỉ số đầu tiên trong đoạn [L, R] có giá trị $\geq X$.

Đầu vào (Input):

- Dòng 1: N, Q ($1 \leq N, Q \leq 10^5$).
- Dòng 2: N số nguyên ban đầu ($1 \leq A_i \leq 10^9$).
- Q dòng tiếp theo: Các truy vấn.

Đầu ra (Output):

- Kết quả các truy vấn loại 2.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
5 3	
1 3 2 4 5	
2 1 5 3	2
1 2 1	4
2 1 5 3	

Giải thích:

Với mảng [2, 3, 5, 1, 6] và truy vấn trong khoảng [1, 5] tìm số ≥ 4 :

Đi từ vị trí 1 sang: vị trí 1 là 2 (< 4), vị trí 2 là 3 (< 4), vị trí 3 là 5 (≥ 4). Vị trí đầu tiên thỏa mãn là vị trí 3.

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq N, Q \leq 10^5, 1 \leq A_i, val, X \leq 10^9$.

Bài 129 [CPPB-RNG-09]: Dây Con Tăng Dài Nhất LIS Bằng Fenwick Tree

Bối cảnh: Một dự án phân tích xu thế chuỗi thời gian lớn gồm N số nguyên cần tìm độ dài của dãy con tăng nghiêm ngặt dài nhất với hiệu năng tính toán tối ưu nhất có thể.

Nhiệm vụ: Cho mảng A gồm N số nguyên. Hãy lập trình tìm độ dài dãy con tăng nghiêm ngặt dài nhất.

Đầu vào (Input):

- Dòng 1: Số nguyên N ($1 \leq N \leq 10^5$).
- Dòng 2: N số nguyên $A_1, A_2, \dots, A_N$ ($1 \leq A_i \leq 10^9$).

Đầu ra (Output):

- Độ dài LIS.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
6 5 2 7 4 3 8	3

Giải thích:

Với dãy số gồm 6 phần tử [10, 20, 10, 30, 20, 50]:

Dãy con tăng dài nhất là [10, 20, 30, 50] có độ dài bằng 4.

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq N \leq 10^5, 1 \leq A_i \leq 10^9$.

Bài 130 [CPPB-RNG-10]: Đoạn Con Có Tổng Lớn Nhất Trên Đoạn (Maximum Subarray Query)

Bối cảnh: Một bảng giá vàng ghi nhận mức biến động giá qua N ngày. Các nhà đầu tư muốn tra cứu xem trong một khoảng thời gian bất kỳ từ ngày L đến ngày R, một đợt sóng tăng giá liên tiếp (đoạn con có tổng lớn nhất) có thể đem lại lợi nhuận tối đa là bao nhiêu.

Nhiệm vụ: Cho mảng A và Q truy vấn gồm hai số L, R. Hãy tìm tổng lớn nhất của một đoạn con không rỗng nằm trọn vẹn bên trong đoạn [L, R].

Đầu vào (Input):

- Dòng 1: N, Q ($1 \leq N, Q \leq 10^5$).
- Dòng 2: N số nguyên ($-10^9 \leq A_i \leq 10^9$).
- Q dòng tiếp theo: Các truy vấn.

Đầu ra (Output):

- Kết quả các truy vấn loại 2.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
5 3	
1 2 -5 4 5	9
2 1 5	22
1 3 10	
2 1 5	

Giải thích:

Với mảng $[-2, 1, -3, 4, -1, 2, 1, -5, 4]$ và truy vấn đoạn từ vị trí 1 đến 9:

Đoạn con liên tiếp có tổng lớn nhất là $[4, -1, 2, 1]$ mang lại tổng là $4 + (-1) + 2 + 1 = 6$.

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq N, Q \leq 10^5, -10^9 \leq A_i, val \leq 10^9$.

Bài 131 [CPPB-RNG-11]: Đếm Số Điểm Trong Hình Chữ Nhật (2D Range Query)

Bối cảnh: Một đài thiên văn số ghi nhận tọa độ mặt phẳng (x_i, y_i) của N ngôi sao trên bầu trời. Các nhà thiên văn học cần thực hiện các truy vấn: đếm xem có bao nhiêu ngôi sao nằm lọt vào bên trong một vùng quan sát hình chữ nhật có góc dưới trái (x_1, y_1) và góc trên phải (x_2, y_2) .

Nhiệm vụ: Cho danh sách tọa độ N điểm và Q truy vấn hình chữ nhật. Hãy lập trình đếm số lượng điểm nằm trong từng hình chữ nhật.

Đầu vào (Input):

- Dòng 1: N, Q ($1 \leq N, Q \leq 10000$).
- N dòng tiếp theo: Tọa độ X_i, Y_i .
- Q dòng tiếp theo: Các truy vấn X_1, Y_1, X_2, Y_2 .

Đầu ra (Output):

- Kết quả Q truy vấn.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
3 2	
1 1	
2 2	2
3 3	2
1 1 2 2	
2 2 4 4	

Giải thích:

Với 3 điểm $(1, 2), (2, 3), (4, 5)$ và vùng quan sát từ $(1, 1)$ đến $(3, 4)$:

Hai điểm $(1, 2)$ và $(2, 3)$ nằm trọn vẹn bên trong vùng hình chữ nhật. Điểm $(4, 5)$ nằm ngoài. Số điểm đếm được là 2.

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq N, Q \leq 10000, 1 \leq X_i, Y_i \leq 10^5$.

Bài 132 [CPPB-RNG-12]: Cập Nhật Phân Đoạn Nâng Cao

Bối cảnh: Một hệ thống lưới điện thông minh hỗ trợ thao tác đổi dấu đồng loạt toàn bộ điện áp trên một phân đoạn đường dây từ L đến R (A_L to $-A_R$), đồng thời hỗ trợ truy vấn tính tổng điện áp lớn nhất của một đoạn con bất kỳ.

Nhiệm vụ: Cho mảng A và các thao tác đảo dấu đoạn hoặc truy vấn tổng đoạn con lớn nhất. Hãy lập trình xử lý và in ra kết quả cho các thao tác truy vấn.

Đầu vào (Input):

- Dòng 1: N, Q ($1 \leq N, Q \leq 10^5$).
- Dòng 2: N số nguyên ban đầu ($1 \leq A_i \leq 10^9$).
- Q dòng tiếp theo: Các truy vấn.

Đầu ra (Output):

- Kết quả các truy vấn loại 2.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
5 3	
1 2 3 4 5	
2 1 3	6
1 2 10	16
2 1 3	

Giải thích:

Sau thao tác đảo dấu đoạn, các giá trị âm biến thành dương giúp hình thành một đoạn con có tổng lớn nhất đạt giá trị tối ưu mới.

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq N, Q \leq 10^5, 1 \leq A_i, val \leq 10^9$.

Bài 133 [CPPB-RNG-13]: Cây Fenwick 2D Tính Tổng Hình Chữ Nhật (2D BIT)

Bối cảnh: Một bức ảnh kỹ thuật số dạng lưới $N \times M$ điểm ảnh, điểm ảnh tại ô (r, c) có giá trị độ sáng $A_{r,c}$. Hệ thống camera cần hỗ trợ hai thao tác: tăng độ sáng tại một điểm ảnh cụ thể thêm val, và tính tổng độ sáng của toàn bộ các điểm ảnh nằm trong một vùng hình chữ nhật từ (r_1, c_1) đến (r_2, c_2) .

Nhiệm vụ: Cho ma trận độ sáng ban đầu và Q thao tác thuộc hai dạng: cập nhật điểm ảnh hoặc truy vấn tổng vùng hình chữ nhật. Hãy in ra kết quả của các thao tác truy vấn.

Đầu vào (Input):

- Dòng 1: N, M, Q ($1 \leq N, M \leq 1000, 1 \leq Q \leq 50000$).
- Q dòng tiếp theo: Các truy vấn.

Đầu ra (Output):

- Kết quả các truy vấn loại 2.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
3 3 3 1 1 1 5 1 2 2 10 2 1 1 2 2	15

Giải thích:

Với ma trận 3×3 toàn số 1: Tổng độ sáng của hình chữ nhật con kích thước 2×2 từ (1, 1) đến (2, 2) gồm 4 ô số 1, tổng bằng 4.

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq N, M \leq 1000, 1 \leq Q \leq 50000$.

Bài 134 [CPPB-RNG-14]: Lazy Propagation — Cập Nhật & Truy Vấn Đoạn (Range Add Range Sum)

Bối cảnh: Một tuyến đê biển dài N mét, ban đầu mét đê thứ i có độ cao A_i . Do điều kiện thi công đắp đập, công nhân sẽ cộng thêm đồng loạt một lượng phù sa có độ dày V mét vào một đoạn đê liên tiếp từ mét thứ L đến mét thứ R . Đồng thời, đoàn thanh tra liên tục kiểm tra tổng độ dày của các đoạn đê bất kỳ.

Nhiệm vụ: Cho mảng A và Q thao tác: `1 L R V` (cộng V vào tất cả các phần tử từ L đến R) và `2 L R` (tính tổng các phần tử trong đoạn $[L, R]$). Hãy in ra kết quả cho các thao tác loại 2.

Đầu vào (Input):

- Dòng 1: N, Q ($1 \leq N, Q \leq 10^5$).
- Dòng 2: N số nguyên ban đầu ($1 \leq A_i \leq 10^9$).
- Q dòng tiếp theo: Các truy vấn.

Đầu ra (Output):

- Kết quả các truy vấn loại 2.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
5 3 1 2 3 4 5 1 2 4 2 2 1 5 2 2 4	21 15

Giải thích:

Với mảng ban đầu [1, 2, 3, 4, 5]:

- Cộng thêm 2 vào đoạn từ vị trí 2 đến 4: mảng trở thành [1, 4, 5, 6, 5].
- Truy vấn tổng đoạn từ 1 đến 5: $1 + 4 + 5 + 6 + 5 = 21$.

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq N, Q \leq 10^5, 1 \leq A_i, val \leq 10^9$.

Bài 135 [CPPB-RNG-15]: Hệ Thống Quản Lý Dữ Liệu Olympic (Range Master)

Bối cảnh: Một hệ thống máy chủ chấm thi Olympic Tin học Quốc tế quản lý dữ liệu điểm số của N thí sinh. Hệ thống cần xử lý đồng thời nhiều loại truy vấn đa năng phức tạp: gán giá trị mới cho một đoạn, cộng thêm giá trị vào một đoạn, tính tổng đoạn, và tìm giá trị lớn nhất/nhỏ nhất trong đoạn.

Nhiệm vụ: Cho mảng A và Q truy vấn đa năng. Hãy lập trình thực thi chính xác và in ra kết quả tương ứng cho từng yêu cầu truy vấn thông tin.

Đầu vào (Input):

- Dòng 1: N, Q ($1 \leq N, Q \leq 10^5$).
- Dòng 2: N số nguyên ban đầu ($1 \leq A_i \leq 10^9$).
- Q dòng tiếp theo: Các truy vấn.

Đầu ra (Output):

- Kết quả các truy vấn loại 2.

Ví dụ mẫu (Sample 1):

Đầu vào (Input)	Đầu ra (Output)
5 3 1 2 3 4 5 1 2 4 2 2 1 5 2 2 4	21 15

Giải thích:

Hệ thống cập nhật linh hoạt các phân đoạn dữ liệu lớn và trả về tổng điểm cũng như điểm cực trị chính xác cho ban giám khảo trong thời gian thực.

Ràng buộc & Giới hạn:

- 100% số test có $1 \leq N, Q \leq 10^5, 1 \leq A_i, val \leq 10^9$.

Thông kê: 21 chuyên đề đã nói | 0 thiếu | 8,230 dòng | 448,022 bytes tổng cộng



Phụ lục: Lời giải bài tập tham khảo

Phần này cung cấp mã nguồn C++ tham khảo chuẩn thi đầu cho các bài tập thực hành trong sách.

Chương 05 — Bài 13: QHĐ 1D & LIS

CPPB-DP1-01 — Bac Thang Co Ban

```
#include <bits/stdc++.h>
using namespace std;

const int MOD = 1e9 + 7;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;
    if (n <= 0) return 0;

    if (n == 1) { cout << 1 << "\n"; return 0; }
    if (n == 2) { cout << 2 << "\n"; return 0; }

    int prev2 = 1, prev1 = 2, cur = 0;
    for (int i = 3; i <= n; ++i) {
        cur = (prev1 + prev2) % MOD;
        prev2 = prev1;
        prev1 = cur;
    }

    cout << cur << "\n";
    return 0;
}
```

CPPB-DP1-02 — Chu Ech Nhay Cost

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;
    if (n <= 0) return 0;

    vector<long long> h(n);
    for (int i = 0; i < n; ++i) cin >> h[i];

    if (n == 1) { cout << 0 << "\n"; return 0; }

    vector<long long> dp(n, 0);
    dp[0] = 0;
    dp[1] = abs(h[1] - h[0]);

    for (int i = 2; i < n; ++i) {
        dp[i] = min(dp[i - 1] + abs(h[i] - h[i - 1]),
                    dp[i - 2] + abs(h[i] - h[i - 2]));
    }
}
```

```

    cout << dp[n - 1] << "\n";
    return 0;
}

```

CPPB-DP1-03 — Chu Ech Nhay K Bước

```

#include <bits/stdc++.h>
using namespace std;

const long long INF = 1e18;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, k;
    if (!(cin >> n >> k)) return 0;
    if (n <= 0) return 0;

    vector<long long> h(n);
    for (int i = 0; i < n; ++i) cin >> h[i];

    vector<long long> dp(n, INF);
    dp[0] = 0;

    for (int i = 0; i < n; ++i) {
        for (int j = 1; j <= k && i + j < n; ++j) {
            dp[i + j] = min(dp[i + j], dp[i] + abs(h[i] - h[i + j]));
        }
    }

    cout << dp[n - 1] << "\n";
    return 0;
}

```

Các bài tập còn lại có phương pháp và cấu trúc tương tự, học sinh tự suy luận và cài đặt.

Chương 05 — Bài 14: QHĐ 2D & Knapsack

CPPB-DP2-01 — Duong Di Tren Luoi So Cach

```

#include <bits/stdc++.h>
using namespace std;

const int MOD = 1e9 + 7;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;
    if (!(cin >> n >> m)) return 0;
    if (n <= 0 || m <= 0) return 0;

    vector<int> dp(m, 1);
    for (int i = 1; i < n; ++i) {
        for (int j = 1; j < m; ++j) {
            dp[j] = (dp[j] + dp[j - 1]) % MOD;
        }
    }

    cout << dp[m - 1] << "\n";
}

```

```
    return 0;
}
```

CPPB-DP2-02 — Duong Di Tren Luoi Co Vat Can

```
#include <bits/stdc++.h>
using namespace std;

const int MOD = 1e9 + 7;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;
    if (!(cin >> n >> m)) return 0;
    if (n <= 0 || m <= 0) return 0;

    vector<string> grid(n);
    for (int i = 0; i < n; ++i) cin >> grid[i];

    if (grid[0][0] == '#' || grid[n - 1][m - 1] == '#') {
        cout << 0 << "\n";
        return 0;
    }

    vector<int> dp(m, 0);
    dp[0] = 1;

    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < m; ++j) {
            if (grid[i][j] == '#') {
                dp[j] = 0;
            } else if (j > 0) {
                dp[j] = (dp[j] + dp[j - 1]) % MOD;
            }
        }
    }

    cout << dp[m - 1] << "\n";
    return 0;
}
```

CPPB-DP2-03 — Duong Di Chi Phi Nho Nhat Grid

```
#include <bits/stdc++.h>
using namespace std;

const long long INF = 1e18;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;
    if (!(cin >> n >> m)) return 0;
    if (n <= 0 || m <= 0) return 0;

    vector<vector<long long>> a(n, vector<long long>(m));
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < m; ++j) cin >> a[i][j];
    }
}
```



```
vector<long long> dp(m, INF);
dp[0] = a[0][0];

for (int j = 1; j < m; ++j) dp[j] = dp[j - 1] + a[0][j];

for (int i = 1; i < n; ++i) {
    dp[0] += a[i][0];
    for (int j = 1; j < m; ++j) {
        dp[j] = min(dp[j], dp[j - 1]) + a[i][j];
    }
}

cout << dp[m - 1] << "\n";
return 0;
}
```

Các bài tập còn lại có phương pháp và cấu trúc tương tự, học sinh tự suy luận và cài đặt.

Chương 05 — Bài 15: QHĐ chuỗi & LCS

CPPB-DPS-01 — Xau Con Chung Dai Nhat Lcs Co Ban

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    string s, t;
    if (!(cin >> s >> t)) return 0;

    int n = s.size(), m = t.size();
    vector<int> prev_row(m + 1, 0), curr_row(m + 1, 0);

    for (int i = 1; i <= n; ++i) {
        for (int j = 1; j <= m; ++j) {
            if (s[i - 1] == t[j - 1]) {
                curr_row[j] = prev_row[j - 1] + 1;
            } else {
                curr_row[j] = max(prev_row[j], curr_row[j - 1]);
            }
        }
        prev_row = curr_row;
    }

    cout << prev_row[m] << "\n";
    return 0;
}
```

CPPB-DPS-02 — Truy Vet Xau Con Chung Dai Nhat

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    string s, t;
    if (!(cin >> s >> t)) return 0;
```

```

int n = s.size(), m = t.size();
vector<vector<int>> dp(n + 1, vector<int>(m + 1, 0));

for (int i = 1; i <= n; ++i) {
    for (int j = 1; j <= m; ++j) {
        if (s[i - 1] == t[j - 1]) {
            dp[i][j] = dp[i - 1][j - 1] + 1;
        } else {
            dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]);
        }
    }
}

// Truy vết
string lcs = "";
int i = n, j = m;
while (i > 0 && j > 0) {
    if (s[i - 1] == t[j - 1]) {
        lcs += s[i - 1];
        --i; --j;
    } else if (dp[i - 1][j] >= dp[i][j - 1]) {
        --i;
    } else {
        --j;
    }
}
reverse(lcs.begin(), lcs.end());

cout << dp[n][m] << "\n";
cout << lcs << "\n";
return 0;
}

```

CPPB-DPS-03 — Khoảng Cách Chỉnh Sửa Edit Distance

```

#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    string s, t;
    if (!(cin >> s >> t)) return 0;

    int n = s.size(), m = t.size();
    vector<int> prev_row(m + 1), curr_row(m + 1);

    for (int j = 0; j <= m; ++j) prev_row[j] = j;

    for (int i = 1; i <= n; ++i) {
        curr_row[0] = i;
        for (int j = 1; j <= m; ++j) {
            if (s[i - 1] == t[j - 1]) {
                curr_row[j] = prev_row[j - 1];
            } else {
                curr_row[j] = 1 + min({prev_row[j], curr_row[j - 1], prev_row[j - 1]});
            }
        }
        prev_row = curr_row;
    }

    cout << prev_row[m] << "\n";
}

```

```
    return 0;
}
```

Các bài tập còn lại có phương pháp và cấu trúc tương tự, học sinh tự suy luận và cài đặt.

Chương 06 — Bài 16: STL nâng cao

CPPB-STL-01 — Dem So Phan Tu Phan Biet

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;
    if (n <= 0) return 0;

    set<long long> st;
    for (int i = 0; i < n; ++i) {
        long long x;
        cin >> x;
        st.insert(x);
    }

    cout << st.size() << "\n";
    return 0;
}
```

CPPB-STL-02 — Bang Tra Cuu Tan Suat Tu Khoa

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;
    if (n <= 0) return 0;

    map<string, int> freq;
    for (int i = 0; i < n; ++i) {
        string s;
        cin >> s;
        freq[s]++;
    }

    for (const auto& p : freq) {
        cout << p.first << " " << p.second << "\n";
    }
    return 0;
}
```

CPPB-STL-03 — Nen Toa Do Mang So Lon

```
#include <bits/stdc++.h>
using namespace std;
```

```

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;
    if (n <= 0) return 0;

    vector<long long> a(n);
    for (int i = 0; i < n; ++i) cin >> a[i];

    vector<long long> vals = a;
    sort(vals.begin(), vals.end());
    vals.erase(unique(vals.begin(), vals.end()), vals.end());

    for (int i = 0; i < n; ++i) {
        int rank = lower_bound(vals.begin(), vals.end(), a[i]) - vals.begin();
        cout << rank << (i + 1 == n ? "" : " ");
    }
    cout << "\n";
    return 0;
}

```

Các bài tập còn lại có phương pháp và cấu trúc tương tự, học sinh tự suy luận và cài đặt.

Chương 06 — Bài 17: Stack & Monotonic Stack

CPPB-STK-01 — Kiểm Tra Dãy Ngoạc Dung Cơ Bản

```

#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    string s;
    if (!(cin >> s)) return 0;

    int open_cnt = 0;
    for (char c : s) {
        if (c == '(') {
            open_cnt++;
        } else {
            if (open_cnt == 0) {
                cout << "NO\n";
                return 0;
            }
            open_cnt--;
        }
    }

    if (open_cnt == 0) cout << "YES\n";
    else cout << "NO\n";
    return 0;
}

```

CPPB-STK-02 — Dãy Ngoạc Hơn Hopp Nhiều Loại

```

#include <bits/stdc++.h>
using namespace std;

```

```

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    string s;
    if (!(cin >> s)) return 0;

    stack<char> st;
    for (char c : s) {
        if (c == '(' || c == '[' || c == '{') {
            st.push(c);
        } else {
            if (st.empty()) {
                cout << "NO\n";
                return 0;
            }
            char top = st.top();
            if ((c == ')' && top == '(') || (c == ']' && top == '[') || (c == '}' && top == '{')) {
                st.pop();
            } else {
                cout << "NO\n";
                return 0;
            }
        }
    }

    if (st.empty()) cout << "YES\n";
    else cout << "NO\n";
    return 0;
}

```

CPPB-STK-03 — Danh Gia Bieu Thuc Hau To Rpn

```

#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;

    stack<long long> st;
    for (int i = 0; i < n; ++i) {
        string token;
        cin >> token;
        if (token == "+" || token == "-" || token == "*") {
            long long b = st.top(); st.pop();
            long long a = st.top(); st.pop();
            if (token == "+") st.push(a + b);
            else if (token == "-") st.push(a - b);
            else if (token == "*") st.push(a * b);
        } else {
            st.push(stoll(token));
        }
    }

    cout << st.top() << "\n";
    return 0;
}

```

Các bài tập còn lại có phương pháp và cấu trúc tương tự, học sinh tự suy luận và cài đặt.

Chương 06 — Bài 18: Queue & Deque

CPPB-QUE-01 — Cài Đặt Hàng Dôi Cơ Bản

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int q;
    if (!(cin >> q)) return 0;

    queue<long long> qu;

    while (q--) {
        int type;
        cin >> type;
        if (type == 1) { // Push x
            long long x;
            cin >> x;
            qu.push(x);
        } else if (type == 2) { // Pop
            if (!qu.empty()) qu.pop();
        } else if (type == 3) { // Front
            if (qu.empty()) cout << "EMPTY\n";
            else cout << qu.front() << "\n";
        }
    }
    return 0;
}
```

CPPB-QUE-02 — Sinh Chuỗi Số Nhị Phân Bằng Queue

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;
    if (n <= 0) return 0;

    queue<string> q;
    q.push("1");

    for (int i = 1; i <= n; ++i) {
        string s = q.front();
        q.pop();

        cout << s << (i == n ? "" : " ");

        q.push(s + "0");
        q.push(s + "1");
    }
    cout << "\n";
}
```

```
    return 0;
}
```

CPPB-QUE-03 — Bfs Tìm Đường Đi Ngắn Nhất Do Thi

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;
    if (!(cin >> n >> m)) return 0;
    if (n <= 0) return 0;

    vector<vector<int>> adj(n + 1);
    for (int i = 0; i < m; ++i) {
        int u, v;
        cin >> u >> v;
        adj[u].push_back(v);
        adj[v].push_back(u);
    }

    vector<int> dist(n + 1, -1);
    queue<int> q;

    dist[1] = 0;
    q.push(1);

    while (!q.empty()) {
        int u = q.front();
        q.pop();

        for (int v : adj[u]) {
            if (dist[v] == -1) {
                dist[v] = dist[u] + 1;
                q.push(v);
            }
        }
    }

    cout << dist[n] << "\n";
    return 0;
}
```

Các bài tập còn lại có phương pháp và cấu trúc tương tự, học sinh tự suy luận và cài đặt.

Chương 07 — Bài 19: Đồ thị BFS & DFS

CPPB-GRA-01 — Chuyển Danh Sách Canh Sang Danh Sách Ke

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;
    if (!(cin >> n >> m)) return 0;
    if (n <= 0) return 0;
```

```
vector<vector<int>> adj(n + 1);
for (int i = 0; i < m; ++i) {
    int u, v;
    cin >> u >> v;
    adj[u].push_back(v);
    adj[v].push_back(u);
}

for (int i = 1; i <= n; ++i) {
    sort(adj[i].begin(), adj[i].end());
    cout << adj[i].size();
    for (int v : adj[i]) {
        cout << " " << v;
    }
    cout << "\n";
}
return 0;
}
```

CPPB-GRA-02 — Duyệt Đồ Thị Theo Chiều Sau Dfs

```
#include <bits/stdc++.h>
using namespace std;

int n, m, s;
vector<vector<int>> adj;
vector<bool> visited;
vector<int> traversal;

void dfs(int u) {
    visited[u] = true;
    traversal.push_back(u);
    for (int v : adj[u]) {
        if (!visited[v]) {
            dfs(v);
        }
    }
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    if (!(cin >> n >> m >> s)) return 0;
    if (n <= 0) return 0;

    adj.assign(n + 1, vector<int>());
    visited.assign(n + 1, false);

    for (int i = 0; i < m; ++i) {
        int u, v;
        cin >> u >> v;
        adj[u].push_back(v);
        adj[v].push_back(u);
    }

    for (int i = 1; i <= n; ++i) sort(adj[i].begin(), adj[i].end());

    dfs(s);

    for (int i = 0; i < (int)traversal.size(); ++i) {
        cout << traversal[i] << (i + 1 == (int)traversal.size() ? "" : " ");
    }
}
```



```

    }
    cout << "\n";
    return 0;
}

```

CPPB-GRA-03 — Duyệt Đồ Thị Theo Chiều Rong Bfs

```

#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m, s;
    if (!(cin >> n >> m >> s)) return 0;
    if (n <= 0) return 0;

    vector<vector<int>>> adj(n + 1);
    for (int i = 0; i < m; ++i) {
        int u, v;
        cin >> u >> v;
        adj[u].push_back(v);
        adj[v].push_back(u);
    }

    for (int i = 1; i <= n; ++i) sort(adj[i].begin(), adj[i].end());

    vector<bool> visited(n + 1, false);
    vector<int> traversal;
    queue<int> q;

    visited[s] = true;
    q.push(s);

    while (!q.empty()) {
        int u = q.front();
        q.pop();
        traversal.push_back(u);

        for (int v : adj[u]) {
            if (!visited[v]) {
                visited[v] = true;
                q.push(v);
            }
        }
    }

    for (int i = 0; i < (int)traversal.size(); ++i) {
        cout << traversal[i] << (i + 1 == (int)traversal.size() ? "" : " ");
    }
    cout << "\n";
    return 0;
}

```

Các bài tập còn lại có phương pháp và cấu trúc tương tự, học sinh tự suy luận và cài đặt.

Chương 07 — Bài 20: Đồ thị lưới 2D**CPPB-GRD-01 — Duyệt 4 Hướng Trên Ma Trận Cơ Bản**

```
#include <bits/stdc++.h>
using namespace std;

const int dr[] = {-1, 1, 0, 0};
const int dc[] = {0, 0, -1, 1};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m, r, c;
    if (!(cin >> n >> m >> r >> c)) return 0;

    int valid_neighbors = 0;
    for (int d = 0; d < 4; ++d) {
        int nr = r + dr[d];
        int nc = c + dc[d];
        if (nr >= 0 && nr < n && nc >= 0 && nc < m) {
            valid_neighbors++;
        }
    }

    cout << valid_neighbors << "\n";
    return 0;
}
```

CPPB-GRD-02 — Đếm Số Lượng Đảo

```
#include <bits/stdc++.h>
using namespace std;

const int dr[] = {-1, 1, 0, 0};
const int dc[] = {0, 0, -1, 1};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;
    if (!(cin >> n >> m)) return 0;
    if (n <= 0 || m <= 0) return 0;

    vector<string> grid(n);
    for (int i = 0; i < n; ++i) cin >> grid[i];

    vector<vector<bool>> visited(n, vector<bool>(m, false));
    int islands = 0;

    for (int r = 0; r < n; ++r) {
        for (int c = 0; c < m; ++c) {
            if (grid[r][c] == '1' && !visited[r][c]) {
                islands++;
                visited[r][c] = true;
                queue<pair<int, int>> q;
                q.push({r, c});

                while (!q.empty()) {
                    auto [cr, cc] = q.front();
                    q.pop();
                }
            }
        }
    }

    cout << islands << "\n";
    return 0;
}
```

```

        for (int d = 0; d < 4; ++d) {
            int nr = cr + dr[d];
            int nc = cc + dc[d];
            if (nr >= 0 && nr < n && nc >= 0 && nc < m && grid[nr][nc] == '1' && !vi
sited[nr][nc]) {
                visited[nr][nc] = true;
                q.push({nr, nc});
            }
        }
    }
}

cout << islands << "\n";
return 0;
}

```

CPPB-GRD-03 — Diện Tích Hòn Đảo Lớn Nhất

```

#include <bits/stdc++.h>
using namespace std;

const int dr[] = {-1, 1, 0, 0};
const int dc[] = {0, 0, -1, 1};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;
    if (!(cin >> n >> m)) return 0;
    if (n <= 0 || m <= 0) return 0;

    vector<string> grid(n);
    for (int i = 0; i < n; ++i) cin >> grid[i];

    vector<vector<bool>> visited(n, vector<bool>(m, false));
    int max_area = 0;

    for (int r = 0; r < n; ++r) {
        for (int c = 0; c < m; ++c) {
            if (grid[r][c] == '1' && !visited[r][c]) {
                int area = 0;
                visited[r][c] = true;
                queue<pair<int, int>> q;
                q.push({r, c});

                while (!q.empty()) {
                    auto [cr, cc] = q.front();
                    q.pop();
                    area++;

                    for (int d = 0; d < 4; ++d) {
                        int nr = cr + dr[d];
                        int nc = cc + dc[d];
                        if (nr >= 0 && nr < n && nc >= 0 && nc < m && grid[nr][nc] == '1' && !vi
sited[nr][nc]) {
                            visited[nr][nc] = true;
                            q.push({nr, nc});
                        }
                    }
                }
            }
        }
    }
}

```

```

        }
        max_area = max(max_area, area);
    }
}

cout << max_area << "\n";
return 0;
}

```

Các bài tập còn lại có phương pháp và cấu trúc tương tự, học sinh tự suy luận và cài đặt.

Chương 07 — Bài 21: Segment Tree & Fenwick Tree

CPPB-RNG-01 — Cài Đặt Fenwick Tree Tính Tổng Đoạn

```

#include <bits/stdc++.h>
using namespace std;

struct FenwickTree {
    int n;
    vector<long long> bit;
    FenwickTree(int n) : n(n), bit(n + 1, 0) {}

    void update(int x, long long val) {
        for (; x <= n; x += x & -x) bit[x] += val;
    }

    long long query(int x) {
        long long sum = 0;
        for (; x > 0; x -= x & -x) sum += bit[x];
        return sum;
    }

    long long queryRange(int l, int r) {
        if (l > r) return 0;
        return query(r) - query(l - 1);
    }
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, q;
    if (!(cin >> n >> q)) return 0;
    if (n <= 0) return 0;

    FenwickTree ft(n);
    for (int i = 1; i <= n; ++i) {
        long long x;
        cin >> x;
        ft.update(i, x);
    }

    while (q--) {
        int type;
        cin >> type;
        if (type == 1) {
            int pos;
            long long val;
            cin >> pos >> val;
            ft.update(pos, val);
        }
    }
}

```

```

    } else {
        int l, r;
        cin >> l >> r;
        cout << ft.queryRange(l, r) << "\n";
    }
}
return 0;
}

```

CPPB-RNG-02 — Cài Dát Segment Tree Tim Min Doan

```

#include <bits/stdc++.h>
using namespace std;

const long long INF = 1e18;

struct SegmentTree {
    int n;
    vector<long long> tree;
    SegmentTree(int n) : n(n), tree(4 * n + 5, INF) {}

    void build(const vector<long long>& a, int id, int l, int r) {
        if (l == r) {
            tree[id] = a[l];
            return;
        }
        int mid = (l + r) / 2;
        build(a, 2 * id, l, mid);
        build(a, 2 * id + 1, mid + 1, r);
        tree[id] = min(tree[2 * id], tree[2 * id + 1]);
    }

    void update(int id, int l, int r, int pos, long long val) {
        if (l == r) {
            tree[id] = val;
            return;
        }
        int mid = (l + r) / 2;
        if (pos <= mid) update(2 * id, l, mid, pos, val);
        else update(2 * id + 1, mid + 1, r, pos, val);
        tree[id] = min(tree[2 * id], tree[2 * id + 1]);
    }

    long long query(int id, int l, int r, int u, int v) {
        if (v < l || r < u) return INF;
        if (u <= l && r <= v) return tree[id];
        int mid = (l + r) / 2;
        return min(query(2 * id, l, mid, u, v), query(2 * id + 1, mid + 1, r, u, v));
    }
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, q;
    if (!(cin >> n >> q)) return 0;
    if (n <= 0) return 0;

    vector<long long> a(n + 1);
    for (int i = 1; i <= n; ++i) cin >> a[i];

    SegmentTree st(n);

```

```

    st.build(a, 1, 1, n);

    while (q--) {
        int type;
        cin >> type;
        if (type == 1) {
            int pos;
            long long val;
            cin >> pos >> val;
            st.update(1, 1, n, pos, val);
        } else {
            int l, r;
            cin >> l >> r;
            cout << st.query(1, 1, n, l, r) << "\n";
        }
    }
    return 0;
}

```

CPPB-RNG-03 — Cap Nhat Doan Va Truy Van Diem Bit

```

#include <bits/stdc++.h>
using namespace std;

struct FenwickTree {
    int n;
    vector<long long> bit;
    FenwickTree(int n) : n(n), bit(n + 2, 0) {}

    void update(int x, long long val) {
        for (; x <= n; x += x & -x) bit[x] += val;
    }

    long long query(int x) {
        long long sum = 0;
        for (; x > 0; x -= x & -x) sum += bit[x];
        return sum;
    }

    void updateRange(int l, int r, long long val) {
        update(l, val);
        update(r + 1, -val);
    }
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, q;
    if (!(cin >> n >> q)) return 0;
    if (n <= 0) return 0;

    FenwickTree ft(n);
    vector<long long> a(n + 1);
    for (int i = 1; i <= n; ++i) {
        cin >> a[i];
        ft.updateRange(i, i, a[i]);
    }

    while (q--) {
        int type;
        cin >> type;
    }
}

```

```
    if (type == 1) { // Cộng val vào [L, R]
        int l, r;
        long long val;
        cin >> l >> r >> val;
        ft.updateRange(l, r, val);
    } else { // Hỏi giá trị tại pos
        int pos;
        cin >> pos;
        cout << ft.query(pos) << "\n";
    }
}
return 0;
}
```

Các bài tập còn lại có phương pháp và cấu trúc tương tự, học sinh tự suy luận và cài đặt.



Mục lục

Lời nói đầu

CHƯƠNG 05: QUY HOẠCH ĐỘNG (DYNAMIC PROGRAMMING)

- Bài 13: Quy hoạch động 1D & dãy con tăng dài nhất
- Bài 14: Quy hoạch động 2D & bài toán cái túi (Knapsack)
- Bài 15: Quy hoạch động chuỗi: LCS & Edit Distance

CHƯƠNG 06: CẤU TRÚC DỮ LIỆU NÂNG CAO

- Bài 16: Cấu trúc dữ liệu STL: Set, map & heap
- Bài 17: Cấu trúc dữ liệu ngăn xếp (stack) & Monotonic Stack
- Bài 18: Cấu trúc dữ liệu hàng đợi (queue, deque) & Monotonic Deque

CHƯƠNG 07: ĐỒ THỊ & CÂY TRUY VẤN ĐOẠN

- Bài 19: Lý thuyết đồ thị cơ bản: Duyệt BFS & DFS
- Bài 20: Đồ thị lưới 2D, kỹ thuật Flood Fill & tính chất cây
- Bài 21: Cấu trúc cây phân đoạn (Segment Tree) & Fenwick Tree (BIT)

Phụ lục: Lời giải bài tập tham khảo

